NestSelf authentication and authorization: implementation guide

How identity, sessions, authorization, the data layer and the clients fit together, where each piece lives in the repository, and how to work on it. The companion `quick-guide.md` gives the one-page version and the reasoning; `docs/tasks/auth/auth-strategy.md` is the full specification (requirement ids such as `D-06` or `AUTHZ-R-61` below refer to it).

Last updated 2026-10-31 on branch `feat/auth-p2`, after auth phase 4, pilot readiness and auth debts (design: `docs/superpowers/specs/2026-10-30-auth-phase-4-pilot-readiness-design.md`; plan: `docs/superpowers/plans/2026-10-30-auth-phase-4-pilot-readiness.md`; status per task: `docs/tasks/auth/phase-2-readiness.md`, "Phase 4 status"). Phase 4 closed the SSO follow-ups that would bite a first real employer (throttles, federated step-up, the unverified OIDC email, the binding connection, federated users on mobile, certificate expiry for metadata-URL providers, the Cognito custom domain) and the auth debts that needed no owner decision; section 21 lists what is left and why, and section 23 describes the shared mailer it added. Before that: auth phase 3, enterprise SSO (plan: `docs/superpowers/plans/2026-10-25-auth-phase-3-sso.md`; section 22), auth phase 2 (plan: `docs/superpowers/plans/2026-10-24-auth-phase-2.md`; status per task: `docs/tasks/auth/phase-2-readiness.md`), the organization owner model (plan: `docs/superpowers/plans/2026-10-24-auth-owner-model.md`) and the UI-completeness plan (`docs/superpowers/plans/2026-10-25-auth-ui-completeness.md`), which gave every auth capability of the API a screen in the web apps. The screenshots are of the real screens, taken by `pnpm docs:auth:screenshots` against the stub API and the seeded sample people (section 19).

Contents

1. The system in one picture
2. Running it locally
3. Repository map
4. The request pipeline
5. Identity provider layer
6. Sign-in flows
7. Sessions, cookies and CSRF
8. Mobile bearer tokens
9. Principal, grants and the authorization engine
10. Data layer: Drizzle, repositories and row-level security
11. Organization and operations administration
12. Audit log
13. WebSockets
14. Background jobs
15. Web clients
16. Mobile client
17. Infrastructure and the Cognito rollout
18. Configuration reference
19. Testing
20. Recipes
21. Known gaps and next steps
22. Enterprise SSO
23. Email

1. The system in one picture



Three ideas carry the whole design:

- **Identity is not authorization.** Cognito (or the local fake) only proves who the caller is. What they may do comes from `grants`, `assignments` and `memberships` in PostgreSQL, resolved per request into a `Principal` and evaluated by the pure `authorize()` function. Tokens carry no roles (D-02).
- **Browsers never see tokens.** Web clients hold an opaque session cookie; the API keeps the Cognito tokens encrypted in Redis (Backend-for-Frontend, D-06). Mobile holds Cognito tokens itself and calls the API with a bearer token (D-07).

- **Default deny, twice.** Every route must declare `@Public()`, `@Authenticated()` or `@Authorize()`, enforced by a CI test. Tenant tables also carry PostgreSQL row-level security policies, so a query that forgets its organization filter returns nothing (D-19, D-37).

2. Running it locally

Local development uses `AUTH_MODE=stub`: a fake identity provider in memory, an in-memory key-value store unless `REDIS_URL` is set, and a real PostgreSQL.

```
pnpm install
pnpm db:up                                  # Postgres 17 and Redis 7 via Docker Compose
cp apps/api/.env.example apps/api/.env      # defaults are fine for stub mode
pnpm --filter @nestself/api db:migrate      # apply apps/api/drizzle/*.sql
pnpm --filter @nestself/api db:seed         # fictional fixtures from packages/sample-data
pnpm --filter @nestself/api dev             # http://localhost:4000, Swagger at /api/docs
pnpm --filter @nestself/api worker:dev      # optional: the nightly jobs (needs REDIS_URL)
pnpm --filter @nestself/api job:run grants.reconcile   # run one job once, no queue
```

The seed creates the sample organization, org units, people, grants and an `external_identities` row per person. The fake identity provider is pre-loaded with the same people (`apps/api/src/identity/identity.module.ts`), so sign-in works out of the box:

Sample person	Email	Population	How to sign in
Jovan Petrović	`user_jovan@example.com`	employee	Email code; read it from the API log line `Fake identity provider: OTP for ... is 123456`, or set `FAKE_IDP_FIXED_OTP=123456`
Ivana Jovanović	`user_ivana@example.com`	employer HR admin and employer owner	Email code. The seed gives her `employer_owner` next to `hr_admin` (`EXTRA_SAMPLE_TEMPLATES` in `seed-lib.ts`), which is what opens `/hr/access`
Stefan Marković	`user_stefan@example.com`	team manager	Email code
Nikola Đorđević	`user_nikola@example.com`	operations staff	Password `Nestself-dev-1234!`, then authenticator enrolment on first sign-in
Milica Ristić	`user_milica@example.com`	expert	Password `Nestself-dev-1234!`, then authenticator enrolment
Dragan Podrašćanin	`user_dragan@example.com`	employer HR admin	Email code. Added 2026-10-24 as the developer account linked to the real dev pool; see section 17

Codes that NestSelf mails itself (the step-up of a federated user and the confirm-email step of an SSO sign-in, section 6) are never written to the log: without SES the logging mailer records only the template, the locale and the recipient's domain (section 23). To try those flows locally set `FAKE_IDP_FIXED_OTP=123456`, which in stub mode fixes those codes as well (`NestselfOtpService.generateCode`).

For TOTP in development use any authenticator app with the QR code, or compute codes with `computeTotp(secret)` from `apps/api/src/identity/totp.ts`. The web apps run in sample mode by default (`VITE_DATA_SOURCE=sample`); set `VITE_DATA_SOURCE=api` in `apps/workspace/.env.local` or `apps/admin/.env.local` to use the real sign-in screens against the API. Set `SESSION_COOKIE_INSECURE_DEV=true` in the API env so the session cookie works on plain `http://localhost`.

`pnpm db:up` publishes Redis on `6379:6379`, so it fails when a local Redis or Valkey already owns that port. A machine that already runs PostgreSQL and Redis outside Docker needs only `DATABASE_URL` and `REDIS_URL` pointed at them; skip `db:up` entirely.

The worker boots the same `AppModule` as the API, so it needs the API's whole configuration (database, identity provider settings, session encryption key) plus `REDIS_URL`: the `sessions.reconcile-ttl` job revokes refresh tokens through `SessionService`.

`db:seed` grants each sample person only the templates mapped from their legacy role (plus `employee` for every member, plus Ivana's `employer_owner`). Run against a database seeded before 2026-10-24, it also deletes the over-broad sample grants the older seed created, when every person received every template without a legacy role. It stops early when a sample person's `external_identities` row was relinked to a real pool subject, because the upsert then collides on the row id; seed a fresh database instead.

Switching to a real Cognito pool is described in section 17.

3. Repository map

Path	What lives there
`apps/api/src/auth/`	HTTP sign-in layer: `auth.controller.ts`, `authentication.guard.ts`, the sign-in services, challenge store, throttle, bearer verifiers, `step-up.service.ts` and `step-up-throttle.ts`, `nestself-otp.service.ts` (the email codes NestSelf issues itself, section 6), `id-token-verifier.ts` (the mobile ID token of `bind-token`, section 8), `me-response.builder.ts` (fills `effectiveActions`), `access-revocation.service.ts`, `profile.service.ts`
`apps/api/src/authz/`	Authorization: `authorize.ts` engine and helpers, `authorization.guard.ts`, `authorization.service.ts` (`require()`, `effectiveActions()`), `step-up-freshness.ts`, `throttles.ts`, `authorize.decorator.ts`, `public.decorator.ts`, `authenticated.decorator.ts`, `resolvers.ts`, `context-loader.ts`, golden decision table, `default-deny.test.ts`
`apps/api/src/principal/`	`principal-loader.ts` (user → `Principal` data with Redis caching), `cache-gateway.ts` (circuit breaker), `invalidation-subscriber.ts`
`apps/api/src/session/`	Redis sessions: `session.service.ts`, `session-cookie.ts`, `csrf.guard.ts`, `session-policy.ts` (lifetimes), `token-cipher.ts`
`apps/api/src/identity/`	`identity-provider.service.ts` (abstract contract), `cognito-identity-provider.ts` + helpers, `fake-identity-provider.ts` + `fake-user-store.ts`, `totp.ts`

Path	What lives there
`apps/api/src/redis/`	`KeyValueStore` contract, Redis and in-memory implementations, `redis-keys.ts` (every key and TTL), `redis-health.indicator.ts`, `key-value-throttler-storage.ts` (throttler counters in Redis, section 4)
`apps/api/src/db/`	Drizzle: `db.service.ts`, `schema/*.ts`, `repositories/*.ts`, `seed/` (`seed-lib.ts`, `backfill-grants.ts`), `migrate.ts`, `like-pattern.ts` (escaped prefix searches)
`apps/api/drizzle/`	SQL migrations and drizzle-kit snapshots
`apps/api/src/audit/`	`audit-writer.ts`, `audit-events.ts` (action catalogue and categories), `pino-redaction.ts`
`apps/api/src/invitations/`, `members/`, `org-units/`, `users/`	Organization and operations administration endpoints; `ops-invitations.controller.ts` and `ops-members.controller.ts` serve the `/ops/orgs/:orgId/*` mirrors
`apps/api/src/grants/`	Grant administration: `grants.controller.ts` (`/orgs/:orgId/grants`), `approval-authorities.controller.ts`, `ops-grants.controller.ts` (`/ops/grants`, `/ops/orgs/:orgId/grants`, `/ops/users/:userId/access`), `grants.service.ts`, `approval-authorities.service.ts`, `grant-rules.ts` (template, scope, condition and delegation-bound rules), `grant-mapper.ts`
`apps/api/src/professional/`	Confidential-domain reads under `/professional/*`; `professional-records.ts` holds the resource shape and the response views shared with emergency reads
`apps/api/src/emergency/`	Emergency access request, approval, denial, revocation and record reads under `/ops/emergency-access/*`
`apps/api/src/sso/`	Enterprise SSO connection management (section 22): `sso-connections.controller.ts` (`/ops/sso-connections`), `sso-connections.service.ts` (lifecycle, domains), `provider-name.ts`, `dns-txt-resolver.ts` (authoritative and fake TXT lookups), `metadata-fetcher.ts` and `public-address.ts` (the guarded metadata fetch, section 22.11), `fake-metadata-fetcher.ts`, `sso.module.ts`
`apps/api/src/auth/sso-callback.service.ts`, `federated-binding.service.ts`, `sso-bind.service.ts`, `pending-bind-store.ts`, `discover.service.ts`, `fake-sso-authorize.controller.ts`, `constants.ts`	Federated sign-in: start, callback and identity resolution; the binding paths the callback, the confirm-email step and the mobile `bind-token` share; the pending binding after an unverified OIDC email; home-realm discovery; the stub-only fake authorize hop; the public email domains
`apps/api/src/email/`	The `Mailer` port, the templates in four locales, the SES and logging adapters, `mask-email.ts` (section 23)
`apps/api/src/identity/cognito-sso-ops.ts`, `fake-sso-store.ts`, `saml-metadata.ts`	Cognito provider and app-client operations, token exchange and ID-token verification; the fake provider's SSO state; SAML certificate expiry
`apps/api/src/ws/`	Socket.IO adapter, gateway, handshake authentication, invalidation
`apps/api/src/jobs/`, `apps/api/src/worker/`	`maintenance-jobs.service.ts` (system jobs), `job-scheduler.service.ts` (BullMQ schedules), `job-authorization.ts` (wrapper for principal-bound processors); `worker/main.ts` is the worker entrypoint
`apps/api/src/common/`	`api-error.ts` (error envelope), exception filter, request metadata, Zod pipe, `app-throttler.guard.ts`, `redirect-on-throttle.decorator.ts` and `throttler-options.ts` (section 4)
`apps/api/src/config/env.ts`	Zod schema of every environment variable, `AUTH_MODE` rules
`apps/api/test/`	Integration suites against a real PostgreSQL (`*.e2e.spec.ts`, `db/schema.spec.ts`)
`apps/api/docs/auth-core.md`	Route table and module map kept current with the code
`packages/contracts/src/`	Shared Zod schemas and types: `auth.ts`, `principal.ts`, `authorization.ts`, `actions.ts`, `templates.ts`, `org-admin.ts` (grants, approval authorities), `ops-admin.ts` (platform grants, user access, emergency access)
`packages/api-client/src/auth.ts`, `org-admin.ts`	Typed client for every auth endpoint (cookie or bearer mode); `createOrgAdminApi(client, { kind: 'org' | 'ops' })` for the grant administration routes
`apps/workspace/src/auth/`, `apps/admin/src/auth/`	`AuthProvider`, sign-in screens (with `screens/sso-bind-screen.tsx`, the confirm-email step), error mapping, `step-up-dialog.tsx`, `with-step-up.ts`, `route-auth.ts` (route guards wait for the session), `session-unavailable.tsx` (section 15)
`apps/workspace/src/features/access/`, `apps/admin/src/features/access/`	Access-management screens (section 15); the workspace's `member-picker.tsx` searches the members
`apps/{workspace,admin}/src/features/account/`, `features/context/`	Settings (display name, language, sessions, authenticator), the header account menu and the context picker (section 7)
`apps/workspace/src/features/people/`, `features/org-units/`, `features/org/`	HR People (members, invitations, import) and Org units; `features/org/use-org-units.ts` owns the org-units query both these and the access screen read (section 11)
`apps/admin/src/features/organizations/`, `features/people/`, `features/users/`, `features/emergency/`	Operations: organizations list and detail, an organization's people, suspend and reinstate, emergency access (sections 11 and 11.3)
`apps/admin/src/features/sso/`, `features/organizations/organization-picker.tsx`	The admin SSO connections page and its organisation search (section 22.8)
`apps/workspace/src/features/cases/`	Expert area (`/cases`) and the participant's conversations (`/conversations`) (section 11.2)
`apps/api/src/organizations/`	`GET /ops/organizations` and `GET /ops/organizations/:orgId` for the admin organizations screens

Path	What lives there
`packages/api-client/src/professional.ts`	`createProfessionalApi`: conversation lists and reads, intake reads
`apps/mobile/src/auth/`	Cognito public client (PKCE, email OTP), secure token store, `AuthProvider`, `sso.ts` (`finishSsoSignIn`: the API binds a federated sign-in first), `sign-in-error.ts`
`infra/cdk/lib/auth-stack.ts`, `infra/cdk/lib/triggers/`, `lib/config.ts`, `lib/compute-stack.ts`	Cognito user pool, app clients, Lambda triggers, secrets, the custom auth domain; stage settings; the API environment and the CloudFront-only load balancer (section 17)
`packages/sample-data/src/fixtures/`	Fictional people, org units, credentials, `identities.ts` (fake pool subjects), `grants.ts` (`SAMPLE_GRANTS`), `case.ts` (`SAMPLE_CASE`: one conversation and one intake), `sso-connections.ts` (`SAMPLE_SSO_CONNECTIONS`)

3.1 Screens for each auth capability

Every screen below works against the API (`VITE_DATA_SOURCE=api`) and, in sample mode, over `SAMPLE_*` fixtures with every change disabled and a note saying so. The gates are conveniences; the API enforces every action (section 4). Failures render translated copy (`packages/i18n/locales/*/auth.json`, `workspace.json`, `admin.json`) with an `unknown` fallback, and every step-up action runs through `withStepUp`.

Capability	App and route	Shown when	API	Feature fold
Display name, language	Workspace and admin `/settings`	Signed in	`PATCH /auth/me/display-name`, `PATCH /auth/me/locale`	`features/a`
Sessions, sign out of one, sign out everywhere	`/settings`, Security	Signed in (not in sample mode)	`GET /auth/sessions`, `DELETE /auth/sessions/:id`, `POST /auth/sign-out-all`, `POST /auth/sign-out`	`features/a`
Authenticator for a signed-in user	`/settings`, Security	`auth.method === 'password_totp'`	`POST /auth/me/totp/enrol`, `POST /auth/totp/confirm`	`features/a`
Sign out	Header account menu, both apps	Signed in	`POST /auth/sign-out`	`features/a`
Context switch	Header context picker, both apps	More than one context available	`POST /auth/context`, then `GET /auth/me`	`features/c`
Members: list, move, end membership	Workspace `/hr/people`, Members	`program.overview` (move `people.move`, end `people.deactivate`)	`GET` (`program.overview` or `access.preview`), `PATCH /orgs/:orgId/members/:id`, `POST .../end`	`features/p`
Invitations: list, invite, import, revoke	Workspace `/hr/people`, Invitations	`people.invite` or `people.import`	`GET`, `POST /orgs/:orgId/invitations`, `POST .../import`, `DELETE .../:id`	`features/p`
Org units: add, rename, move	Workspace `/hr/org-units`	`org_units.manage`	`GET` (`program.overview` or `access.preview`), `POST /orgs/:orgId/org-units`, `PATCH .../:id`, `POST .../:id/move`	`features/c`
Grants, delegation, preview, ownership (HR side)	Workspace `/hr/access`	`access.preview` (the route sits outside the `_hr` layout since phase 4, so an employer owner without an HR template reaches it)	`/orgs/:orgId/grants/*`, the member search `GET /orgs/:orgId/members?search=`	`features/a`
Organizations list and detail	Admin `/organizations`, `/organizations/:orgId`	Staff; list needs `client.configure` at platform scope	`GET /ops/organizations`, `GET /ops/organizations/:orgId`	`features/c`
Owner and transfer (staff)	Admin `/organizations/:orgId` Overview	`access.preview`, transfer `access.grant`	`GET /ops/orgs/:orgId/grants`, `POST .../transfer-ownership`	`features/c`
An organization's people	Admin `/organizations/:orgId/people`	`client.configure` (end `people.deactivate`, invitations `people.onboard`)	`/ops/orgs/:orgId/members`, `/ops/orgs/:orgId/invitations`	`features/p`
Grants (staff side), user access lookup	Admin `/access`, `/organizations/:orgId/access`	`access.preview`	`/ops/grants`, `/ops/orgs/:orgId/grants`, `/ops/users/:userId/access`	`features/a`
Suspend and reinstate	Admin user lookup result	`people.deactivate`; not offered for one's own account	`POST /ops/users/:userId/suspend`, `.../reinstate`	`features/u`
Emergency access: request, approve, deny, revoke	Admin `/emergency`	`audit.read` (list), request `access.emergency.request`, decide `access.emergency.approve`	`/ops/emergency-access/*`	`features/e`
Emergency record view	Admin `/emergency`, Open record	The caller's own active access	`GET /ops/emergency-access/:id/records/:type/:id`	`features/e`

Capability	App and route	Shown when	API	Feature fold
Assigned cases (expert)	Workspace `/cases`, `/cases/:conversationId`	`principal.expert` or `conversation.read.assigned`	`GET /professional/conversations`, `GET /professional/conversations/:id`, `GET /professional/intake/:id`	`features/c`
My conversations (participant)	Workspace `/conversations`, `/conversations/:id`	`conversation.participate`, not in the expert context	`GET /professional/me/conversations`, `GET /professional/me/conversations/:id`	`features/c`
SSO connections: create, edit, domains, verify, test link, activate, suspend, reset to draft	Admin `/sso`	`sso.manage`	`/ops/sso-connections/*`	`features/s`
Federated sign-in and its outcomes	Workspace and admin `/sign-in` (redirect line, `?reason=sso_denied`, `?reason=sso_rate_limited`, `?reason=sso_email_unverified&bind=<handle>`: the confirm-email step)	Discovery answers `sso`	`POST /auth/discover`, `GET /auth/sso/start`, `GET /auth/sso/callback`, `POST /auth/sso/bind/start`, `/verify`, `/resend`	`auth/scree` `auth/scree`
Step-up before a sensitive action	A dialog over the screen that asked, both apps	The API answers 403 `step_up_required`	`POST /auth/step-up/start`, `/resend`, `/verify`	`auth/step-` `auth/with-`
A session that cannot be loaded	Any signed-in route, both apps	`GET /auth/me` fails with anything but a 401, or takes longer than 10 seconds	`GET /auth/me`	`auth/sessi` `auth/route`

Not covered by a screen: approval authorities (`/orgs/:orgId/approval-authorities`), changing a grant's conditions or validity (`PATCH /orgs/:orgId/grants/:id`), and everything on mobile, which keeps sign-in (federated sign-in included, section 8), settings and sign-out (section 21).

4. The request pipeline

Four global guards run in this order for every HTTP request (`apps/api/src/app.module.ts`):

1. **`AuthenticationGuard`** (`src/auth/authentication.guard.ts`). Skips `@Public()` routes. Otherwise it reads the session cookie or the `Authorization: Bearer` header, resolves a `Principal` and attaches `request.principal`, `request.session` (web only), `request.authMeta` and `request.csrfToken`. Missing or invalid credentials end the request with 401.
2. **`CsrfGuard`** (`src/session/csrf.guard.ts`). Only when a session cookie is present and the method is not safe: it checks `Sec-Fetch-Site`, the `Origin` header against the allowed surfaces and the double-submit token (`X-CSRF-Token` header must equal the CSRF cookie). WebSocket contexts are skipped. Failure is 403 `csrf_rejected`.
3. **`AuthorizationGuard`** (`src/authz/authorization.guard.ts`). Skips `@Public()` and `@Authenticated()` routes. For `@Authorize(action, resolver)` it calls the resolver to derive the `Resource`, builds the context with `ContextLoader`, calls `authorize()`, writes an `access.denied` audit event on denial (the engine's reason is recorded there, never returned to the client) and answers 404 when the caller has no visibility into the organization and 403 otherwise (AUTHZ-R-61). A route may list several actions, `@Authorize([a, b], resolver)`, meaning any of them: the guard evaluates them in order, stops at the first that allows and applies that action's step-up class; when none allows, the denial is audited under the first action (`actionAttempted`) with the whole list in `details.alternatives`, which only a route with alternatives writes (`AuthorizeMetadata.actions` in `src/authz/authorize.decorator.ts`; an empty list throws when the route is declared). `GET /orgs/:orgId/members` and `GET /orgs/:orgId/org-units` accept `program.overview` or `access.preview`, so an employer owner without an HR template can use the access page (section 11.1). Any principal holding a staff template counts as having visibility, so an operations caller denied in an organization they are not assigned to gets 403 `forbidden`, not 404 (the confidential reads answer 404 for every denial, section 11.2). When the decision allows and the action has a step-up class, the guard then checks that the authentication is recent enough and answers 403 `step_up_required` otherwise (section 9, Step-up). A handler that carries no decorator at all is denied at runtime as well as by the CI test.
4. **`AppThrottlerGuard`** (`src/common/app-throttler.guard.ts`, the NestJS `ThrottlerGuard` plus the SSO redirect). 60 requests per minute per client IP by default (`src/common/throttler-options.ts`); a route's `@Throttle(...)` replaces that budget, with the values and trackers in `src/authz/throttles.ts`:
   - grant, approval-authority, emergency-access and SSO-connection writes: `WRITE_THROTTLE`, 10 per minute per route and client IP;
   - `GET /auth/sso/start`: `SSO_START_THROTTLE`, 60 per minute per client IP and `connection` value (`ssoStartTracker`; the raw value is cut to 32 characters);
   - `GET /auth/sso/callback`: `SSO_CALLBACK_THROTTLE`, 120 per minute per client IP;
   - the three step-up routes: `SESSION_THROTTLE`, 10 per minute per session, for a bearer per token family (`sessionTracker`, hashed);
   - the SSO bind routes and `bind-token`: `SSO_BIND_THROTTLE`, 10 per minute per client IP;
   - the invitation preview and accept routes: 10 per minute.

   The native sign-in routes (`/auth/discover`, `/auth/email-otp/*`, `/auth/password`, `/auth/challenge`, `/auth/totp/*`, `/auth/password-reset/*`) carry no route throttle of their own and share the default budget; wrong codes are bounded by `SignInThrottle` per address (section 6). With `REDIS_URL` set the counters live in Redis (`KeyValueThrottlerStorage`, `src/redis/key-value-throttler-storage.ts`: a fixed window per key, blocking as the memory storage does), so every API task counts against one budget; without it each process counts on its own. A route marked `@RedirectOnThrottle()` (`src/common/redirect-on-throttle.decorator.ts`: the SSO start and callback) is answered with a bodyless 302 to the workspace `/sign-in?reason=sso_rate_limited` instead of the JSON 429 (section 22.3). The client IP is `req.ip`, which Express takes from `X-Forwarded-For` when `main.ts` sets `trust proxy` to `TRUST_PROXY_HOPS` (above 0; 2 in AWS, section 18); IPv6 addresses count per /64 (`clientIp`).

Decorators (`src/authz/`):

Decorator	Meaning
`@Public()`	No authentication. Sign-in, invitation preview and accept, health, leads.
`@Authenticated()`	A valid session or bearer token, no action check. `/auth/me`, sessions, sign-out, context switch, profile.
`@Authorize(ACTIONS['x.y'], resolver)`, `@Authorize([ACTIONS['x.y'], ACTIONS['z.w']], resolver)`	Full grant-based check; a list means any of the actions (guard 3 above). The resolver returns `{ type, id?, orgId?, ... }`; use `orgResourceFromContext(type)` for `/orgs/:orgId/*` routes (it enforces that the active context is that organization, API-R-21), `operationsResource(type, idFn)` for `/ops/*` routes (session required, operations context) and `operationsOrgResource(type)` for `/ops/orgs/:orgId/*` routes (the same checks, plus the route organization in the resource so the engine applies the `assigned_clients` scope through the CLIENT assignment).
`@CurrentPrincipal()`	Parameter decorator returning `request.principal`.

`src/authz/default-deny.test.ts` boots the application and fails when any controller handler or gateway message handler lacks one of the three decorators. It runs with the normal test suite, so an undecorated route cannot reach `main`.

Errors use one envelope (`src/common/api-error.ts`, `api-exception.filter.ts`): `{ status, code, message, requestId, details? }`. Codes the clients map: `unauthenticated`, `session_expired`, `csrf_rejected`, `forbidden`, `not_found`, `context_mismatch`, `session_required`, `invalid_code`, `invalid_credentials`, `invalid_challenge`, `expired_code`, `too_many_attempts`, `rate_limited`, `validation`, `conflict`, `step_up_required`, `step_up_unavailable`, `delegation_exceeds_boundary`, `subject_not_member`, `template_inactive`, `missing_conditions`, `self_approval_denied`, `emergency_not_active`, `owner_exists`, `cannot_suspend_self`, `cannot_end_self`, `sso_connection_state`, `domain_blocklisted`, `domain_unverified`, `sso_email_unverified`, `sso_denied`, `unavailable` (`AUTH_ERROR_CODES` in `packages/contracts/src/auth.ts`); the invitation routes add `duplicate_invitation` and `already_member`, and an import names its refused rows with the codes of section 11. A 429 carries `details.retryAfter` (seconds) where the wait is known: a resend cooldown or a bind start inside its minute.

5. Identity provider layer

`IdentityProviderService` (`src/identity/identity-provider.service.ts`) is an abstract class used as the DI token. It covers everything the API needs from the identity provider: start and answer the email OTP challenge, password sign-in and the `SOFTWARE_TOKEN_MFA`, `NEW_PASSWORD_REQUIRED` and `MFA_SETUP` challenges, TOTP association and verification, refresh and revocation, global sign-out, enable, disable, create, delete and find users, forgot-password. Sign-in services depend only on this contract, so swapping the provider never touches them.

- **`CognitoIdentityProvider`** (`AUTH_MODE=cognito`): AWS SDK v3 calls with the confidential web client (`SECRET_HASH` on every call). User-management calls live in `cognito-user-ops.ts`; `cognito-error-map.ts` turns Cognito exceptions into `IdentityProviderError` codes.
- **`FakeIdentityProvider`** (`AUTH_MODE=stub`): in-memory users, sessions, OTP codes and refresh-token families with the same challenge chain as Cognito (`fake-user-store.ts`). It refuses to construct when `NODE_ENV=production`, logs email codes, honours `FAKE_IDP_FIXED_OTP`, and is pre-seeded with the sample people in development.

Both return `AuthOutcome`, either `{ kind: 'tokens', tokens }` or `{ kind: 'challenge', challenge, session, username }`.

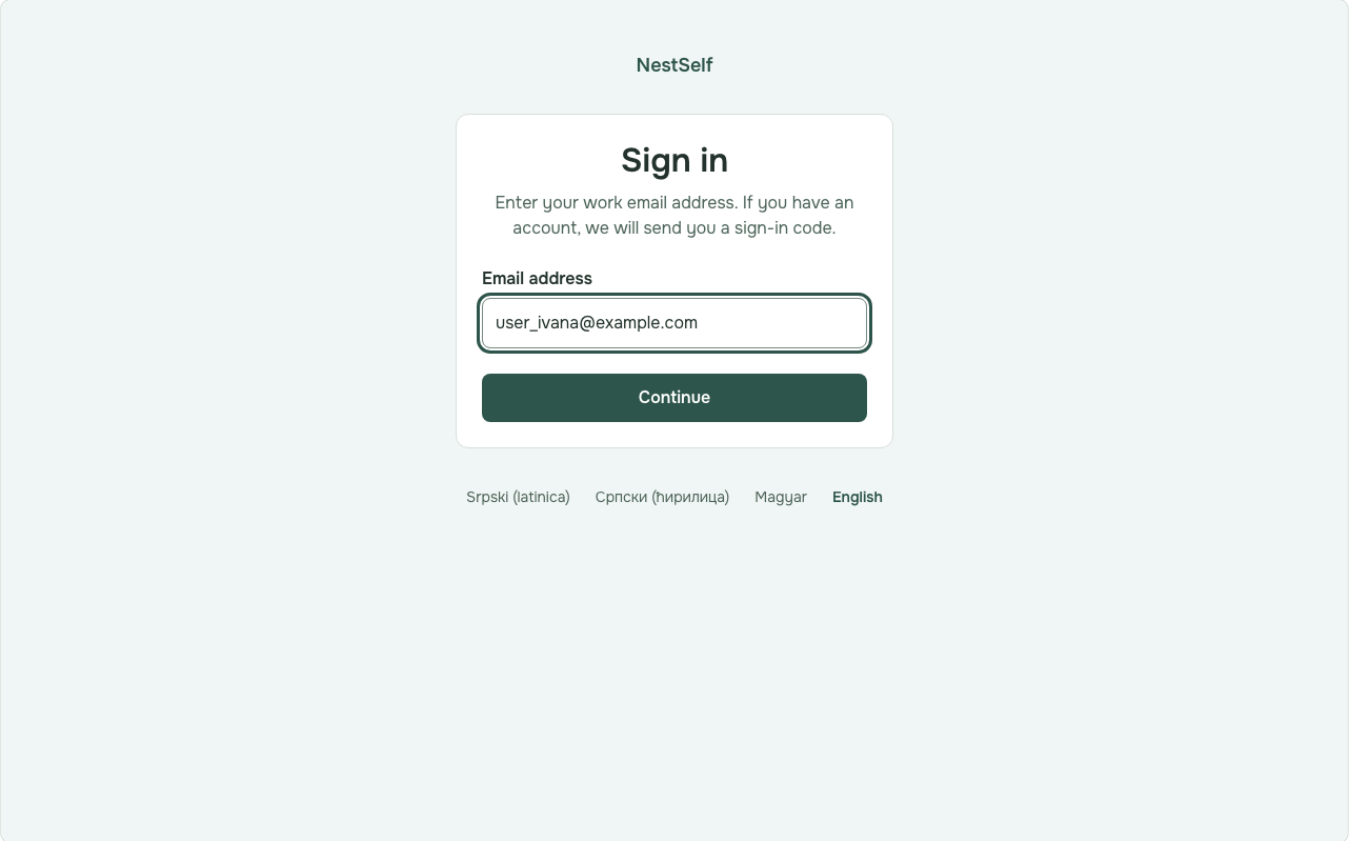
6. Sign-in flows

All flows start at `POST /auth/discover` (`discover.service.ts`), which answers `email_otp`, `password` or `sso` without revealing whether the address exists. Interactive steps are stitched together with **challenge tokens**: `ChallengeStore` (`src/auth/challenge-store.ts`) stores the provider session under `challenge:{sha256(token)}` for 5 minutes and consumes it on every use (`GETDEL`, so two concurrent answers cannot both reach the provider). Failed attempts count in `SignInThrottle` (`throttle:otp:{email}`): five failures lock the identifier for 15 minutes with 429 `too_many_attempts`. Unknown addresses go through the same timing (`neutral-delay.ts`) and receive a dummy challenge so responses do not reveal existence.

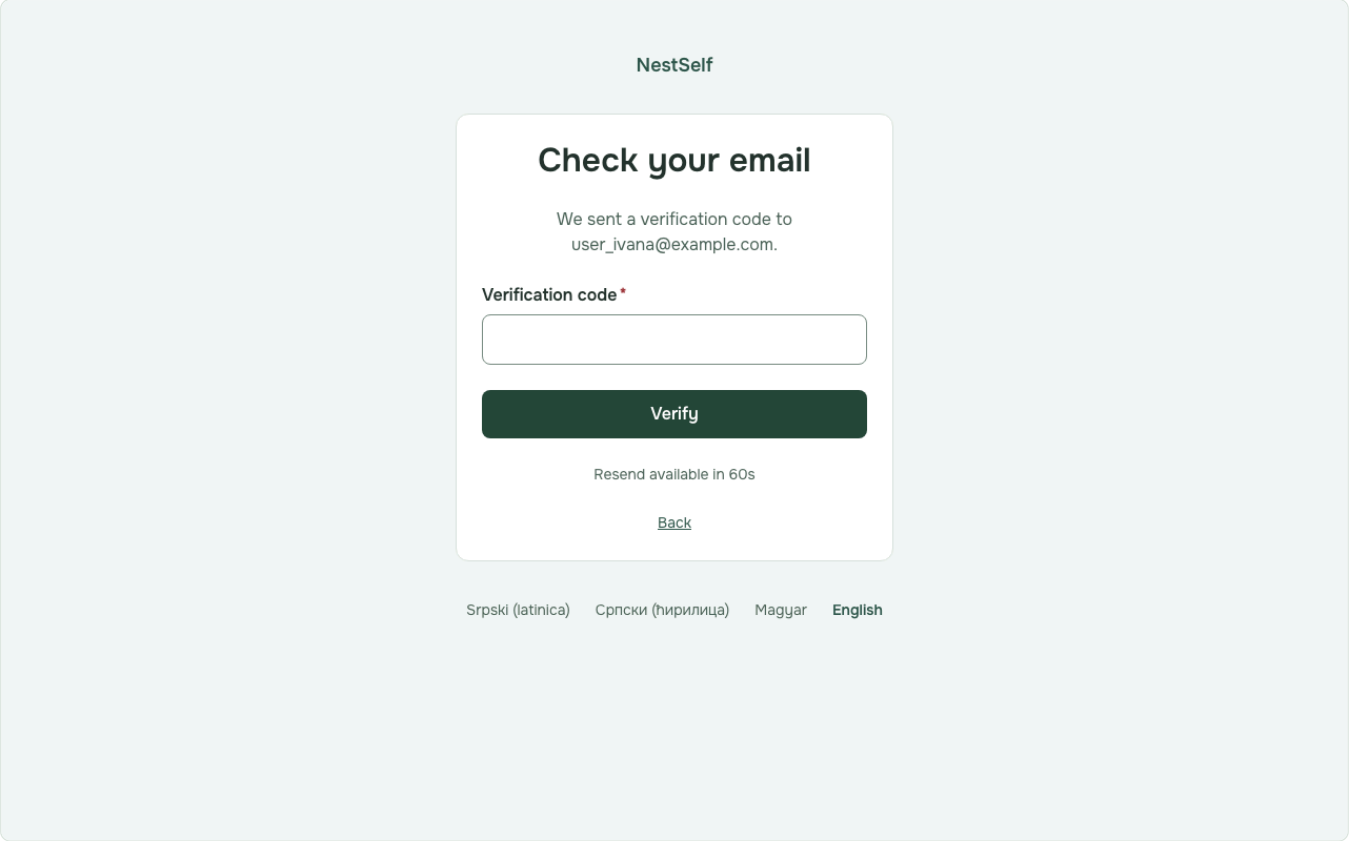
Cognito's EMAIL_OTP code is eight digits, unlike the six-digit codes it issues for sign-up confirmation, forgot-password and authenticator apps, and unlike the stub provider's `FAKE_IDP_FIXED_OTP`. `EMAIL_OTP_MIN_LENGTH` and `EMAIL_OTP_MAX_LENGTH` in `packages/contracts/src/auth.ts` hold that range; `EmailOtpVerifyRequestSchema`, `OtpInput` and the mobile field all read it. Before 2026-10-24 every one of them hard-coded six, so the input truncated the real code, the user could not type the last two digits, and Cognito answered `CodeMismatchException`.

A wrong email code can be retyped, up to three times per challenge. `emailOtpVerify` and the step-up `verify` consume the challenge before asking the provider. When the provider answers `invalid_code` (Cognito's `CodeMismatchException`, which keeps its own `Session` alive), `retainAfterWrongCode` in `sign-in.service.ts` puts the record back under the same token with `attempts + 1` through `ChallengeStore.put()`, and the API answers 401 `invalid_code`; the user corrects the code without requesting a new one. `put` keeps the deadline stamped at creation (`expiresAt`), so retries never extend the five minutes. The third wrong code (`MAX_EMAIL_OTP_ATTEMPTS = 3`) is not put back: the next attempt with that token answers 401 `invalid_challenge` and the screens ask for a new code. Any other provider error ends the challenge at once. Dummy challenges for unknown addresses follow the same budget so the answers stay identical. `SignInThrottle` still counts every wrong code, across challenges, and locks the address after five. Each wrong code is counted before the record is handed back, and the record that triggers the lock is not handed back; a challenge retained earlier answers 429 `too_many_attempts` once the address is locked (checked after `consume` for dummy and real challenges alike), so a lock cannot be outrun with a token that still has tries left.

Employees: email code. `POST /auth/email-otp/start` with `{ email, locale? }` → `{ challengeToken }` (`locale` is one of `en`, `sr-Latn`, `sr-Cyrl`, `hu`; the API forwards it to the provider as `ClientMetadata.locale`, which the CustomMessage trigger uses for the email's language when the user has no `custom:locale` attribute; the web screens send the language they are shown in and the mobile app sends its resolved device locale directly to Cognito); `POST /auth/email-otp/verify` with `{ challengeToken, code }` → tokens → `IdentityResolutionService.resolveIdentityAndCreateSession()` maps the token `sub` to an `external_identities` row, loads the principal, creates the Redis session, sets the cookies and returns `{ principal, effectiveActions }`.

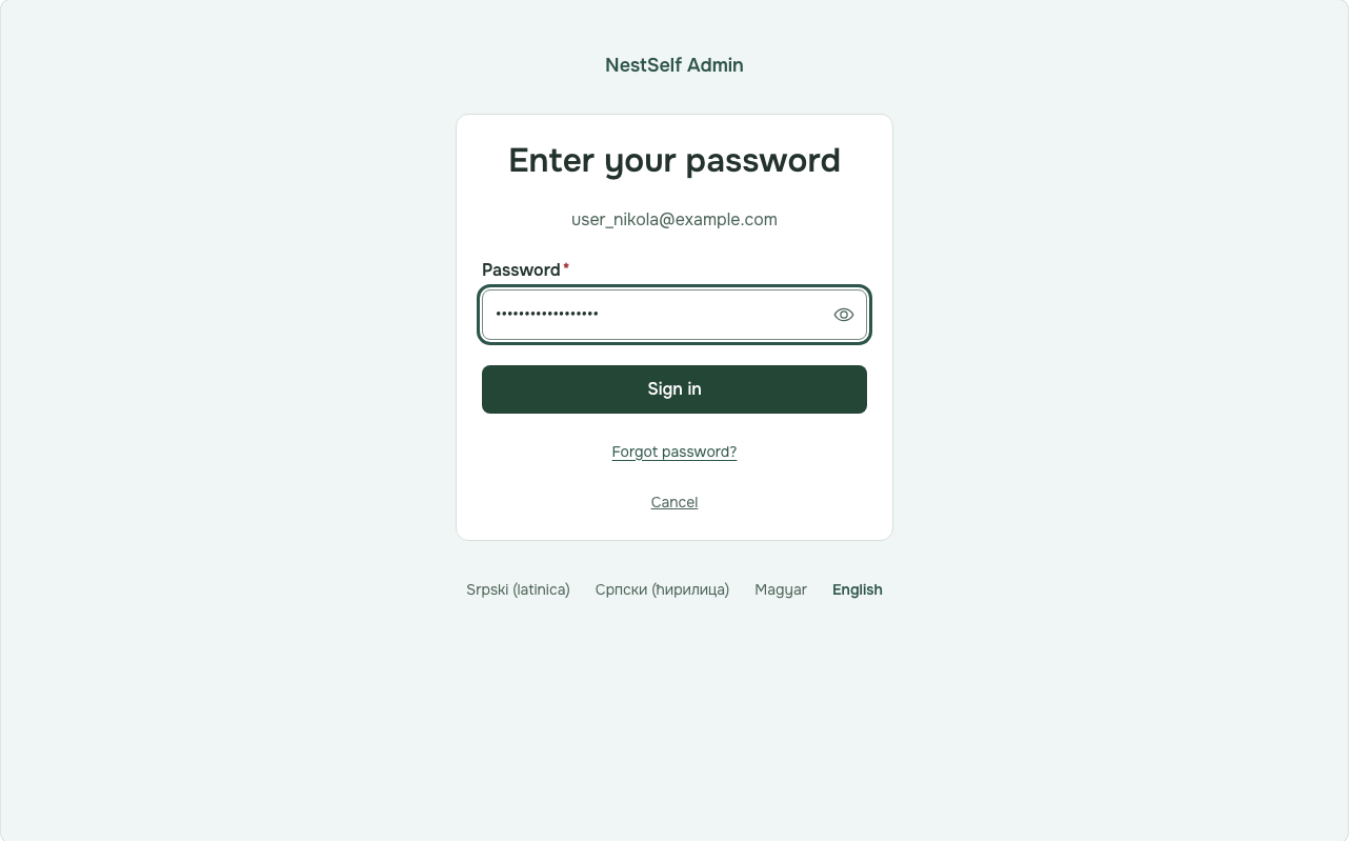


Workspace sign-in, first step: the address decides the method (`POST /auth/discover`).

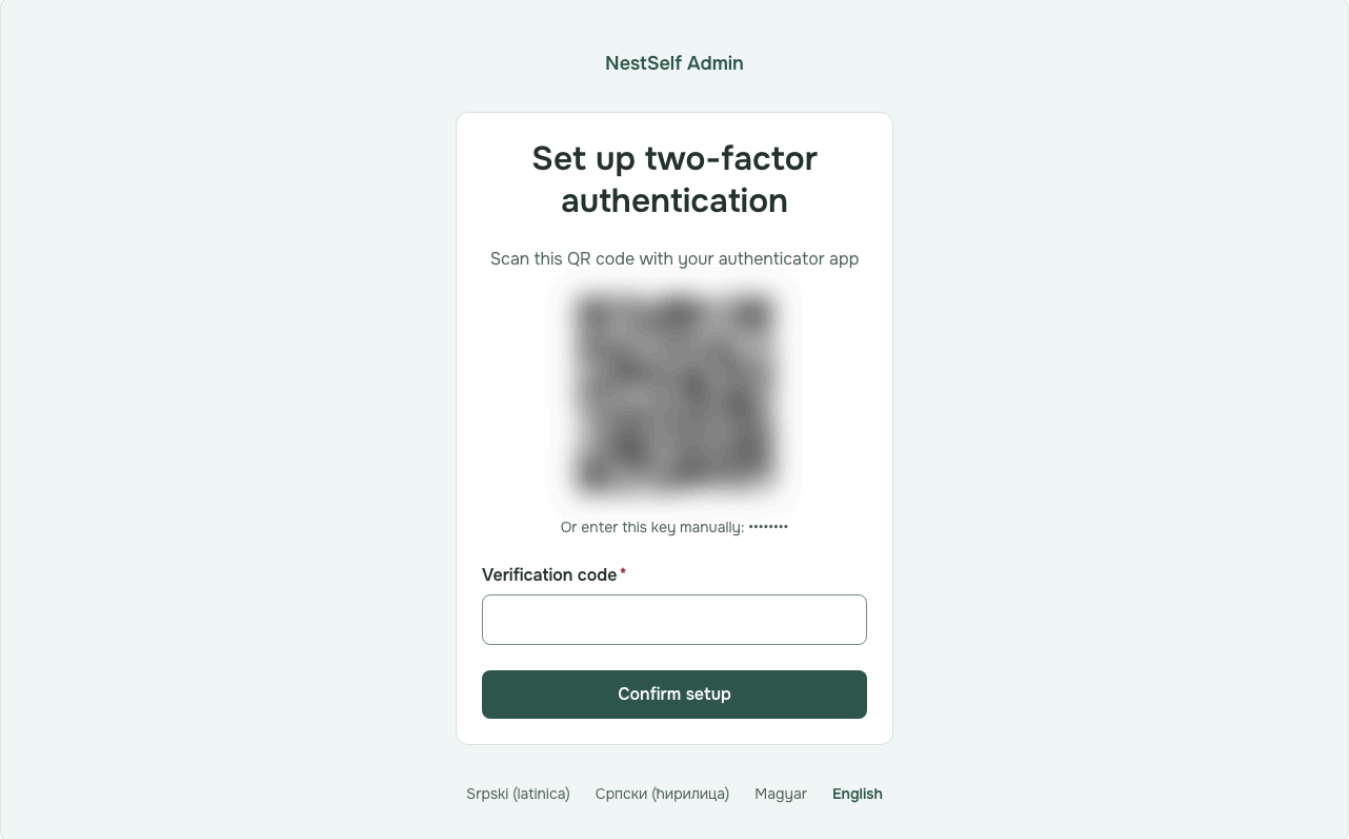


The email-code step. The field takes the code length range from `@nestself/contracts` (eight digits from Cognito).

Experts and staff: password and authenticator. `POST /auth/password` returns either the signed-in principal or `{ step, challengeToken }` where `step` is `new_password` , `totp` or `totp_enrol` . `POST /auth/challenge` with `{ challengeToken, response }` answers one step and returns either the next `{ step, challengeToken }` (200) or the principal. For `totp_enrol` the client first calls `POST /auth/totp/enrol` with the challenge token to receive the `otpauth://` URI and a rotated challenge token, then `POST /auth/challenge` with the authenticator code; the API verifies the software token and completes `MFA_SETUP` in one step (`sign-in.service.ts` , `totp.service.ts`). `POST /auth/totp/confirm` is the signed-in variant for adding an authenticator later.



Admin sign-in for staff and experts: the password step after the identifier.



First sign-in: enrolment shows the `otpauth://` URI as a QR code and as a manual key, then takes the first authenticator code (`totp_enrol`). Later sign-ins show only the code field. Both carry the authenticator secret, so the screenshot blurs the QR code and masks the key.

Password reset. `POST /auth/password-reset/start` and `/confirm` (`password-reset.service.ts`) wrap the provider's forgot-password flow with neutral responses.

Invitation acceptance. `GET /invitations/:token/preview` returns only the organization name; `POST /invitations/accept` (`src/invitations/invitation-activation.service.ts`) validates the hashed token, creates or links the user, ensures the identity-provider user and the `external_identities` row, creates the ACTIVE membership and the template grants in one transaction, then starts the right sign-in method: an email code for employees or a temporary-password flow

for staff and experts. The request may carry `locale` (`InvitationAcceptRequestSchema`), which the email code of the employee path is written in; both web invite screens send the page's language (`routes/_public/invite.$token.tsx`). For an address on a verified domain of the inviting organization's `ACTIVE` SSO connection the accept writes nothing and answers `{ method: 'sso', ssoStartUrl }` (section 22.4).

SSO. When discovery answers `sso`, the browser goes to `GET /auth/sso/start` and comes back through the callback (section 22.4). An OIDC sign-in whose address the provider did not confirm continues on the sign-in page with an email code that NestSelf sends (the confirm-email step, section 22.4); the mobile app binds a federated sign-in through `POST /auth/sso/bind-token` (section 8).

Codes NestSelf issues itself. Cognito sends the native email codes. Two flows have no native pool user to send one to, the step-up of a federated user (section 9) and the confirm-email step of a pending SSO binding (section 22.4), so `NestselfOtpService` (`src/auth/nestself-otp.service.ts`) issues them:

- The code has `EMAIL_OTP_MAX_LENGTH` (8) digits from `crypto.randomInt`. `issue` mails it through the `Mailer` (section 23) first and only then stores the challenge at `otp:{purpose}:{sha256(token)}` (`purpose` is `step_up` or `sso_bind`) for `CHALLENGE_TTL` (300 s), so a refused mail leaves nothing behind. The record keeps `sha256(token:code)`, never the code, and the purpose is part of the key, so a token opens only the flow it was issued for. The answer is the challenge token and the masked address (`maskEmail`, section 9).
- `verify` allows three wrong codes (`MAX_EMAIL_OTP_ATTEMPTS`): 401 `invalid_code` keeps the challenge with its deadline, the third answers 429 `too_many_attempts` and ends it, an unknown or expired token is 401 `invalid_challenge`, and a success consumes it.
- `resend` mails a new code under the same token, the same deadline and the same attempt count, at most once per `OTP_RESEND_COOLDOWN_SECONDS` (60 s) counted from the last mail, the first one included (`otp_resend:{purpose}:{hash}`; 429 `rate_limited` with `details.retryAfter`).
- `verify` and `resend` take the record out of the store (`GETDEL`) and put it back for the time it has left, so parallel guesses cannot exceed the attempt budget and a resend cannot revive a consumed challenge; a request that finds the record taken answers `invalid_challenge`. A resend keeps the record out of the store while its mail is in flight and puts the previous record back when the provider refuses it.
- A provider refusal (`MailDeliveryError`) answers 503 `unavailable`. In stub mode `FAKE_IDP_FIXED_OTP` fixes the code, as it does for the fake identity provider. Nothing logs an address, a code or a token.

Every sign-in writes `auth.sign_in.succeeded` or `auth.sign_in.failed` through `AuditWriter`.

7. Sessions, cookies and CSRF

`SessionService` (`src/session/session.service.ts`) stores one JSON record per session at `session:{sha256(sessionId)}` and indexes a user's sessions in `user_sessions:{userId}`. The record (`session.types.ts`) holds the user id, surface (`workspace` or `admin`), population, active context, authentication metadata, lifetimes, a hashed IP and user-agent family, the CSRF token, and the Cognito tokens encrypted with AES-256-GCM (`token-cipher.ts`, key `SESSION_ENCRYPTION_KEY`, dual-key rotation via `SESSION_ENCRYPTION_KEY_PREVIOUS`).

Cookies (`session-cookie.ts`): `__Host-nestself_session` and `__Host-nestself_csrf`, `Secure`, `HttpOnly` for the session, `SameSite=Lax`, `Path=/`. `SESSION_COOKIE_INSECURE_DEV=true` drops the `__Host-` prefix and `Secure` for plain-HTTP localhost. The CSRF cookie is readable by JavaScript on purpose: the API client copies it into `X-CSRF-Token`.

Lifetimes (`session-policy.ts`) depend on population and context, for example employee personal 30 minutes idle and 12 hours absolute, employee organization 60 minutes and 8 hours, expert 8 hours and 7 days, staff 60 minutes and 8 hours; `SESSION_LIFETIME_OVERRIDES` (JSON) adjusts them per environment within the 30-day idle and 90-day absolute caps. Each request extends the idle window; the absolute expiry never moves. The session id is regenerated at sign-in and again after a completed step-up (section 9).

No ETags on API responses. `main.ts` calls `set('etag', false)` on the Express instance. Express otherwise emits a weak ETag for every JSON response, the browser revalidates with `If-None-Match`, and the API answers `304 Not Modified` with no body. `@nestself/api-client` treats anything outside 2xx as a failure (`if (!response.ok)`), so a page refresh turned a live session into an `ApiError` with status 304 and bounced the user to `/sign-in` while the Redis session was still healthy. These responses are per-principal and already `Cache-Control: no-store`; they must never be revalidated.

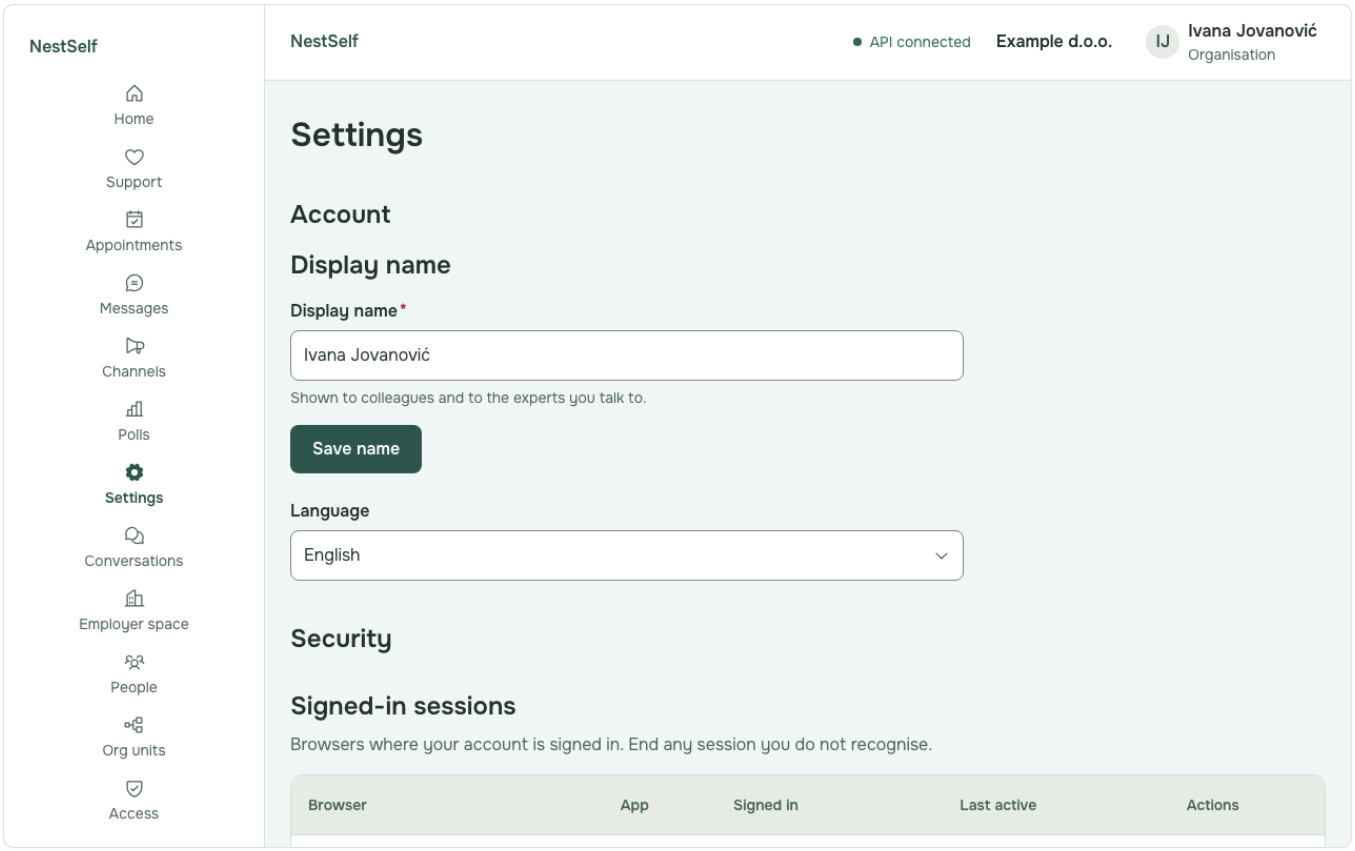
Access-token refresh happens lazily: `ensureFreshAccessToken` refreshes through the identity provider under `lock:session:{hash}` so concurrent requests do not race; a failed refresh destroys the session.

Revocation paths: `POST /auth/sign-out` (current session), `POST /auth/sign-out-all` (every session, Cognito global sign-out), `DELETE /auth/sessions/:id` (another device), and `AccessRevocationService.revokeAllAccess()` (`src/auth/access-revocation.service.ts`), which membership end and suspension call: destroy sessions, `globalSignOut` for every external identity, publish the principal invalidation. It never throws; it reports what failed.

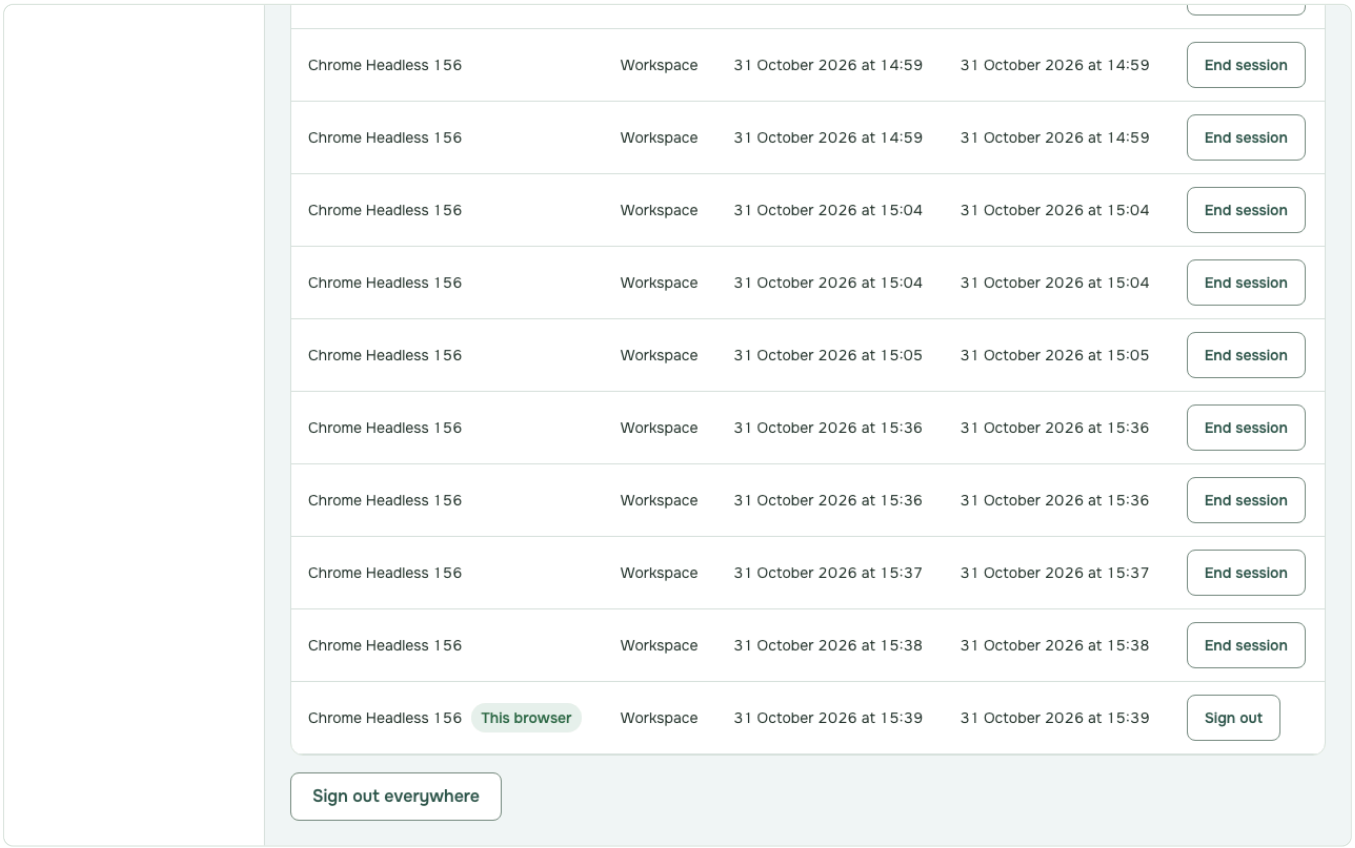
Context switch (`POST /auth/context`, `session-endpoints.service.ts`) changes the active context (`personal`, `organization`, `expert`, `operations`) after checking the principal actually holds it; it never extends the absolute expiry.

Account security screens. `/settings` in both web apps (`features/account/`) has three parts. *Account*: the display name (`PATCH /auth/me/display-name`, at most `DISPLAY_NAME_MAX_LENGTH` = 120 characters from `@nestself/contracts`) and the language, saved to the account and then applied to the screen. *Security*: the signed-in sessions (`GET /auth/sessions`), with the current browser marked; ending another session calls `DELETE /auth/sessions/:id`, while the row of the current browser reads "Sign out" and calls `POST /auth/sign-out`, because the API refuses to delete the current session with 409. "Sign out everywhere" calls `POST /auth/sign-out-all`, which keeps the caller's own session, and then `POST /auth/sign-out`, so this browser ends too and lands on `/sign-in`; if the first call fails the user stays on the page with the message. Staff and experts who sign in with a password also see the authenticator section: `POST /auth/me/totp/enrol` (refused with 409 `totp_already_enrolled` when an authenticator exists) and `POST /auth/totp/confirm`; the secret never outlives the screen (`gcTime: 0` on both mutations and a reset on cancel and unmount). *Preferences*: the theme. The header of both apps carries the account menu (name, current context, Settings, Sign out).

The display-name field shows the saved name until the user types, also on a deep link or a reload of `/settings`, where the principal arrives after the first render (fixed 2026-10-25; the field used to stay empty).



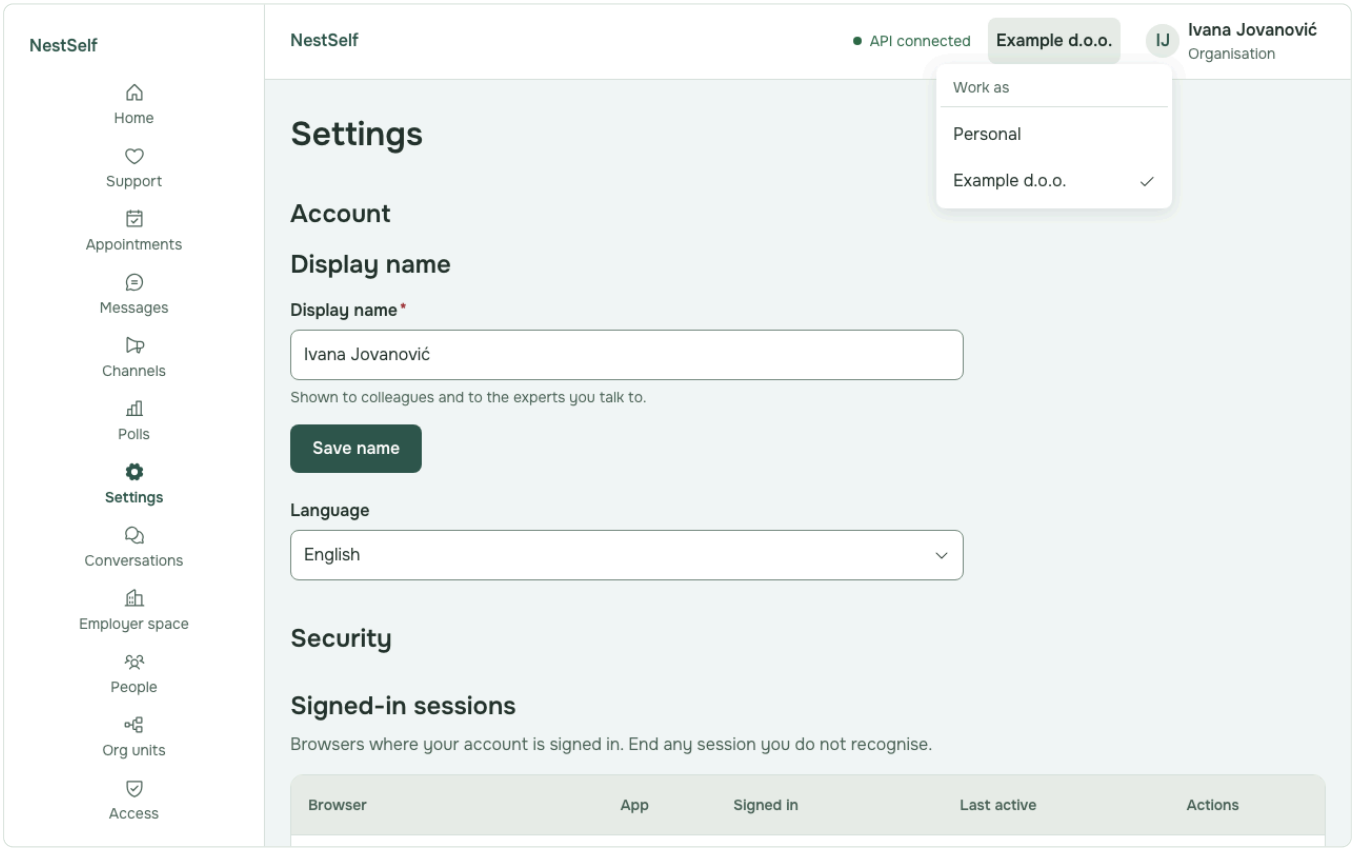
Workspace `/settings` for Ivana Jovanović: display name and language are saved to her account.



The end of the sessions list: one row per signed-in browser, "This browser" on the current one, End session on the others and Sign out everywhere below. The other rows are sign-ins of earlier screenshot runs, which the stub API keeps in the developer's Redis.

Context picker. `features/context/context-picker.tsx` sits in the header of both apps, before the account menu, whenever the principal has more than one context: Personal, each organization with an ACTIVE membership (by `orgName`, which `/auth/me` memberships carry since 2026-10-25), Expert when `principal.expert` is set and Operations when the principal holds a staff template. Choosing one calls `POST /auth/context` and then `GET /auth/me`, so `effectiveActions` and the gated navigation change without a reload; the workspace then opens `/` (or `/cases` for Expert), the admin app `/`. On `invalid_context` or `org_context_expired` the picker refreshes the principal, so an ended membership leaves the menu. Failures show a toast with translated

copy. The on-mount context hooks of the people, access, operations and expert screens switch at most once per mount, so they never undo a choice made in the picker.



The context picker open for Ivana: Personal or her organization, Example d.o.o. (current).

8. Mobile bearer tokens

Mobile apps authenticate against Cognito directly and send `Authorization: Bearer <access token>`. `AuthenticationGuard` passes the token to the `BEARER_VERIFIER` provider (`src/auth/bearer-verifier.ts`): `CognitoBearerVerifier` verifies signature and claims with `aws-jwt-verify` (`tokenUse: 'access'`, the mobile client id) and rejects tokens whose refresh family is listed in `revoked_family:{originJti}`; `StubBearerVerifier` accepts HS256 tokens signed with `JWT_SECRET` in stub mode. The guard then maps `sub` to the user through `external_identities`, records the family in `user_mobile_families:{userId}` and refreshes `mobile_family:{originJti}` (30-day TTL, AUTHZ-R-64) and assembles the principal with the default context for the user. Bearer callers cannot use `/ops/*` routes (API-R-28) and get 403 `session_required`. The nightly `mobile.prune-stale-families` job (section 14) denylists families whose last-seen key has expired.

Cognito access tokens carry `auth_time` but not the method, so `CognitoBearerVerifier` reports `email_otp` with no second factor for a native user. That is accurate: the mobile app client only permits the email-code first factor (`USER_AUTH` with `EMAIL_OTP`) and passwords and authenticators are web-only. Bearer callers step up through the same `/auth/step-up/*` endpoints as the browser, never through claims; the result is stored under `stepup:{tokenFamily}` for 30 minutes (`STEP_UP_BEARER_TTL`). The mobile app has no step-up screen yet (section 21).

Federated users on the bearer path. Both bearer verifiers pass the pool `username`. A federated pool user's username is `<cognitoProviderName>_<IdP subject>` (provider names hold no `_`), a native one's is the email address. When the part before the first `_` is the provider name of an existing connection, `AuthenticationGuard.resolveBearerUser` treats the bearer as federated: it looks the identity up under `cognito:<provider>` by `sub` and lets it through only on path 1 exactly as the SSO callback runs it (`FederatedBindingService.existingIdentityUsable`: the identity was bound through that very connection and its user is an `ACTIVE` member of the connection's organization, section 22.4). Otherwise the answer is 401 `unauthenticated`, with no native fallback, and `auth.sign_in.failed { reason: 'sso_provider_mismatch' }` for the connection's organization. That row is written once per connection and pool user in `BEARER_MISMATCH_AUDIT_TTL` (600 s), keyed by `bearer_mismatch_audited:{connectionId}:{sha256(sub)}`, because the guard runs before any throttle and a refused token would otherwise write a row per request; when the store cannot answer, the row is written anyway (the dedupe fails open, the 401 never does). A native user whose address starts with `<provider name>_` is refused the same way: the prefix rule fails safe. A federated principal records `method: 'sso'` and `secondFactor: true`, the second factor being delegated to the employer's IdP as on the web (3.4.9).

Binding a federated sign-in from the app. The app signs in through the Cognito hosted UI directly, so the API's callback, and with it every binding rule, never runs for it. After its own PKCE exchange and before anything else the app presents the ID token to `POST /auth/sso/bind-token` (`{ idToken }`, `@Public()`, 204, `SSO_BIND_THROTTLE`). `ID_TOKEN_VERIFIER` (`src/auth/id-token-verifier.ts`) checks it: with `AUTH_MODE=cognito` `aws-jwt-verify` against the pool's keys, `token_use = id` and the audience `COGNITO_MOBILE_CLIENT_ID` (the web client's ID tokens never leave the API and are refused); in stub mode the stub bearer verifier's signature check with `token_use: 'id'` required. A token that fails is 401 `unauthenticated`, keys that cannot be fetched 503 `unavailable`. `SsoBindService.bindToken` (`src/auth/sso-bind.service.ts`) then applies the callback's rules: an existing identity takes path 1 (204, or 403 `sso_denied` with `sso_provider_mismatch` and nothing deleted); a new binding needs an `ACTIVE` connection (no staff test on mobile), an address on one of its verified domains and, for OIDC, `email_verified === true` unless the connection's claim switch is off. An unconfirmed OIDC address answers 403 `sso_email_unverified` and keeps the pool user without opening a pending binding: the person confirms the address once on the web, and the app's next sign-in takes path 1. Nothing to bind deletes the orphan and answers 403 `sso_denied`. No audit row or log line carries the ID token.

9. Principal, grants and the authorization engine

Shape. `Principal` (`packages/contracts/src/auth.ts`, `principal.ts`): `userId`, `displayName`, `initials`, `locale`, `memberships[]` (id, orgId, orgUnitId, status, dates), `staffTemplates[]`, `expert` (id and approved categories), `context`, `auth` (method, time, second factor, last step-up), `sessionId`, `tokenFamily`. It carries no grants; those are loaded per decision.

Loading and caching. `PrincipalLoader` (`src/principal/principal-loader.ts`) reads through the repositories and caches in Redis with jittered TTLs: `principal:{userId}` 30 s, `grants:{userId}:{orgId}` 60 s, `assignments:{userId}` 60 s, `org_tree:{orgId}` 5 min. `CacheGateway` wraps Redis with a circuit breaker (three failures open it for 5 s) and a small in-memory LRU fallback. Writes that change access call `invalidateUser(userId, orgIds)`, which deletes the keys and publishes on the `invalidate` channel; `InvalidationSubscriber` on every API instance drops its local copies, and the WebSocket layer re-authorizes connected sockets.

Delegated grants carry their ancestors. When the subject holds a delegated grant, `loadGrants` also loads its ancestor chain (`AuthorizationRepository.findDelegationAncestors`, a recursive walk over `parent_grant_id` in system context, revoked rows included) into the same `grants:{userId}:{orgId}` entry, because `validateDelegation` needs the whole chain to decide. A change to an ancestor reaches that entry only through the delegatee's own invalidation, so every path that revokes or adjusts a grant calls `PrincipalLoader.invalidateDelegatees(grantIds)`, which walks the tree downwards (`findDelegationDescendants`) and invalidates every subject below: grant revoke and update, membership end and the `grants.reconcile` job. It runs after the commit; if it fails, delegatees keep stale copies until the grants TTL ends.

Vocabulary. `ACTIONS` (`packages/contracts/src/actions.ts`) is the closed catalogue of dotted action strings (`people.invite`, `program.overview`, `conversation.read.assigned` …). `PERMISSION_TEMPLATES` (`templates.ts`) are the 17 named starting sets (`platform_owner`, `client_operations`, `hr_admin`, `team_manager`, `employee`, `expert` …) with default actions, scope type, conditions and step-up actions. A **grant** gives a user a list of actions within a scope (`own`, `org_unit` with optional descendants, `organization`, `platform`, `assigned_cases` …) with optional Zod-validated conditions (spending ceiling, categories, confidential approver, time window), a validity window and, for delegated grants, a parent and depth limit of 3.

Deciding. `authorize(principal, action, resource, ctx)` (`src/authz/authorize.ts`) is pure and ordered: user state (suspended or deactivated denies everything), context validity (`context.ts`), confidential hard deny, assignments, grants with scope matching through the closure-table ancestor set (`scope.ts`), conditions (`conditions.ts`), validity window, self-approval rule, delegation envelope (`delegation.ts`). It returns `{ allowed, grantIds, assignmentIds, denialReason, explanation[] }`; `explain.ts`, `authorize-many.ts` and `effective-access.ts` build on it. `ContextLoader` (`context-loader.ts`) assembles the `AuthorizationContext` (grants, assignments, ancestor ids, organization settings, emergency access for confidential actions, which is never cached). Step 5 matches only grants whose `subjectUserId` is the principal's own; the ancestors loaded for a delegated grant serve `validateDelegation` and never lend their actions to the delegatee. The golden table `golden-table.json` (150 tuples, including the `CONF-DENY-01` to `06` confidential hard-deny rows and the `DELEG-ALLOW-01`, `DELEG-DENY-01` to `03` delegation rows) and fast-check property tests (`authorize.property.test.ts`) pin the behaviour: no cross-organization allow, delegation never exceeds the delegator, expired never allows, confidential never allowed by administrative scopes.

Using it outside a guard. `AuthorizationService.require(principal, action, resource)` is for service-layer checks whose resource needs row data a static resolver cannot load (the confidential domain and emergency record reads, section 11). It audits denials like the guard, maps them to 403 or 404 the same way (or always to the 404 of a missing record with `{ denyAs: 'not_found' }`, which the confidential reads pass, section 11.2), and applies the same step-up check. `effectiveActions(principal)` fills `effectiveActions` in every `MeResponse` through `MeResponseBuilder` (`src/auth/me-response.builder.ts`): sign-in completion, `GET /auth/me`, `POST /auth/context` (for the new context) and the two profile updates. Before 2026-10-24 every response carried an empty list, so every action-gated screen was unreachable in api mode. An action counts when any current grant allows it, and an `org_unit` grant is evaluated at its own unit, so team-scoped templates such as `team_manager` appear. Clients get the same answer client-side as a hint through `can()` in `packages/contracts/src/principal.ts`; it is never the enforcement.

Step-up. `STEP_UP_CLASS_BY_ACTION` (`packages/contracts/src/authorization.ts`) puts sensitive actions into five classes, each with a maximum age in `STEP_UP_MAX_AGE_SECONDS`:

Class	Maximum age	Actions
`emergency`	5 min	`access.emergency.request`, `access.emergency.approve`
`restricted`	15 min	`conversation.read.assigned`, `notes.read.supervision`, `records.read.emergency`
`export`	15 min	`report.export`, `data.export.own`
`financial`	15 min	`spend.approve`, `funding.allocate`, `budget.allocate`
`administration`	30 min	`access.grant`, `access.revoke`, `access.delegate`, `people.deactivate`, `sso.manage`, `session.revoke.any`

`assertStepUpFresh` (`src/authz/step-up-freshness.ts`) runs after an allow decision, both in `AuthorizationGuard` and in `AuthorizationService.require()`. It takes the latest of `auth.time`, `auth.lastStepUpAt` and, for bearer callers, the `stepup:{tokenFamily}` record; when that is older than the class allows it throws 403 `step_up_required` with `details: { stepUpClass, maxAge, method: 'email_otp' }` (true for every caller since phase 4: federated users step up with an email code too). A caller who signed in again after an old step-up counts from the new sign-in. Denied callers keep their 403 or 404 and are never asked to step up. The `restricted` class is enforced as a plain 15-minute maximum age, not as "first access per session" as the strategy words it; the stricter reading can be added without a contract change.

`StepUpService` (`src/auth/step-up.service.ts`) serves three `@Authenticated()` routes, each limited by `SESSION_THROTTLE` to 10 requests per minute per session (per token family for a bearer):

- **`POST /auth/step-up/start`** (200, `{ method: 'email_otp', challengeToken, ssoStartUrl: null, emailMasked }`). A native user, one whose `cognito` identity still has a pool user, gets Cognito's EMAIL_OTP in the principal's locale (`sr-Latn` when the stored locale is not one of the four); the challenge lives in the `ChallengeStore` with `stepUp.userId`. Anyone else with a federated identity gets a code NestSelf issues (`NestselfOtpService`, purpose `step_up`, template `step_up_code`, section 6), mailed to the `email` of that `external_identities` row, the address the employer's IdP asserted on a verified domain, never `users.email`; the challenge is bound to the user and to the session (by its hash) or the token family that asked. With several federated identities the one bound to an `ACTIVE` connection wins, else the most recently seen, else the first. A principal with no identity a code can go to gets 403 `step_up_unavailable` (it used to be a bare `forbidden`). `emailMasked` is the address as `maskEmail` (`src/email/mask-email.ts`) writes it: the first character, three stars, the domain (`i***@acme.example`). A principal with neither a session nor a token family, so nothing to record a step-up on, gets 401 `session_required` from all three routes.
- **`POST /auth/step-up/verify`** (`{ challengeToken, response }`, 204). It first checks the per-user lock, `StepUpThrottle` (`src/auth/step-up-throttle.ts`, `throttle:step_up:{userId}`): five wrong codes of either kind within 15 minutes of the first lock the user's step-up until that window ends (429 `too_many_attempts`), and a completed step-up clears the count. Then it dispatches on the token: a `ChallengeStore` record is the Cognito kind, anything else

goes to the NestSelf store under `step_up` only. A challenge minted for another user is refused with 401 `invalid_challenge`, audited as `auth.step_up.failed` with `reason: 'foreign_challenge'` (never the token or the code) and spent; a sign-in challenge is refused as unknown, and `/auth/email-otp/verify` refuses step-up challenges the same way, so a step-up code can never mint a new session. A wrong code follows the three-attempt rule of section 6; the third wrong NestSelf code answers 401 `invalid_challenge` (ask for a new code), not the lock's 429. On success a session caller gets `markStepUp`, then `regenerate`: the 204 carries a new session cookie and CSRF token. A bearer caller gets `stepup:{tokenFamily}` with a 30-minute TTL. The throwaway refresh token of a Cognito re-authentication is revoked, and `auth.step_up.completed { method: 'email_otp', issuer: 'cognito' | 'nestself' }` or `auth.step_up.failed` is audited.

- **`POST /auth/step-up/resend`** (`{ challengeToken }`, 200, the start answer). A new code at most once per `OTP_RESEND_COOLDOWN_SECONDS` (60 s) counted from the previous mail, the first one included (429 `rate_limited` with `details.retryAfter`). The Cognito kind starts a new provider challenge under a new token and spends the old one; the NestSelf kind keeps its token. Neither resets the wrong codes counted on the challenge, a provider failure leaves the code sent before working, and another user's challenge of either kind is refused, audited as `foreign_challenge` and not resent. A locked user can neither start nor resend.

Federated users step up with an email code, not an IdP round trip (SSO-R-17, phase 4 design 4.1). Cognito forwards neither `prompt=login` nor SAML `ForceAuthn` to the employer's IdP, and its `auth_time` is the moment Cognito completed the federation, which is "now" whenever the IdP answers silently from its own session; a fresh `auth_time` therefore cannot prove that the user re-authenticated. The round trip can be reconsidered only if the live pool shows that Cognito carries the IdP's own authentication time (a check on the T-3.4 list, section 22.13).

Every class steps up with an email code, staff and experts who signed in with a password and an authenticator included, and the `emergency` class included, for which the strategy asks for an authenticator. Cognito's `USER_AUTH` flow cannot issue a `SOFTWARE_TOKEN_MFA` challenge without a password first, so an authenticator-only step-up would mean retyping the password. This deviation is an open owner decision (section 21). Section 15 shows the dialog.

10. Data layer: Drizzle, repositories and row-level security

Schema (`src/db/schema/`): `identity.ts` (users, external_identities with `sso_connection_id`, the connection a federated identity was bound through), `organization.ts` (organizations with the `organizations_status_check` over the four contract values, org_units, org_unit_closure, memberships, legacy_teams and the `teams` view), `authorization.ts` (permission_templates, grants, assignments, approval_authorities, emergency_access), `professional.ts` (cases, conversations, conversation_messages, intake_records), `invitations.ts` (invitations, sso_connections with `oidc_email_verified_claim_required`, sso_domains), `audit.ts` (audit_events, retention_policies), `content.ts` (experts, credentials, content, channels, polls, leads, event requests), `relations.ts` for `db.query.*`. Since migration `0005` neither `users` nor `memberships` carries a legacy role: what a person may do comes from grants only. Conventions: snake_case columns (`casing: 'snake_case'`), text ids with `gen_random_uuid()::text` defaults, `timestamptz(3)`, `updated_at` maintained by Drizzle.

Row-level security. Tables with an organization column (`org_units`, `memberships`, `grants`, `approval_authorities`, `invitations`, `sso_connections`, `channels`, `event_requests`, `cases`, `conversations`, `intake_records`) declare a `pgPolicy` whose predicate is `coalesce(current_setting('app.current_org_id', true), '') = '' OR org_id = current_setting('app.current_org_id', true)` (platform-wide grants with a null organization stay visible). Tables that reach their organization only through a join use `tenantPredicateVia()` with a correlated `EXISTS`: `posts` through `channels`, `polls`, `poll_options` and `poll_votes` through `legacy_teams`, `org_unit_closure` through `org_units` (migration `0002_rls_legacy_content_closure.sql`), `conversation_messages` through `conversations` (migration `0003`) and `sso_domains` through `sso_connections` (migration `0006`). Every one of the eighteen tenant tables has RLS enabled and forced (`test/db/schema.spec.ts` lists them). `emergency_access` is platform-level and has no policy. `DbService.withOrgScope(orgId, fn)` runs `fn` in a transaction after `select set_config('app.current_org_id', $1, true)`; everything inside sees and writes only that organization. `DbService.withTransaction(fn)` is the same without the setting, for platform writes such as platform grants and emergency access. Code that runs without the setting (principal loading, seeds, jobs) is in system context and sees all rows. Policies bind only roles that are neither superuser nor table owner: migrations run as the owner, the API connects as a login that is a member of `nestself_app` (Docker: `nestself_api` created by `docker/initdb/01-app-role.sql`). A local Homebrew superuser is not restricted; `test/db/schema.spec.ts` proves the policies with a non-superuser login.

Repositories (`src/db/repositories/`): `identity`, `authorization` (grants, assignments, delegation walks, emergency access), `approval-authorities`, `professional` (inserts the case before a conversation or intake record), `audit`, `leads`, `invitations`, `memberships`, `org-units`, `organizations`, `sso`. Prefix searches (members by name or email, organizations by name or slug) escape `%`, `_` and `\` through `likePrefix` (`src/db/like-pattern.ts`), so a typed wildcard matches only itself. They are the only place with Drizzle query builders; every method accepts an optional `tx` so services can compose them inside `withOrgScope`. Services depend on repositories, which is why unit tests mock plain methods and app-level tests override `DbService` with `{ ping, onModuleInit, onModuleDestroy }`.

Migrations. `pnpm --filter @nestself/api db:generate` diffs the schema against `drizzle/meta` and writes a new SQL file; `db:generate -- --custom --name=<name>` creates an empty file for hand-written SQL; `db:migrate` applies pending files (tracked in `drizzle.__drizzle_migrations`); `db:seed` upserts the fixtures. `0001_audit_partitions_rls_roles.sql` holds what drizzle-kit cannot express: monthly range partitions and `audit_events_ensure_partitions()`, the append-only triggers, the `nestself_app` role and its privileges, `FORCE ROW LEVEL SECURITY`. `0002_rls_legacy_content_closure.sql` adds the join-based policies above, forces them, and makes `audit_events_ensure_partitions()` `SECURITY DEFINER` so the application role can create partitions from the nightly job. Never let drizzle-kit alter `audit_events` directly; write a custom migration.

Phase 2 added three migrations. `0003_professional_domain.sql` creates the confidential-domain tables with their policies and forced RLS. `0004_emergency_access_request.sql` makes `emergency_access.approved_by_user_id` nullable and adds `approved_at` and `requested_duration_minutes` (default 240), so a request can wait for its approver. `0005_drop_legacy_roles.sql` drops `users.platform_role`, `memberships.role` and the `org_role` and `platform_role` enum types (D-36); a hand-written `DO $$` guard at its top raises, naming the first user, when a legacy role still lacks its backfilled grant. On a database that still holds legacy roles, run `pnpm --filter @nestself/api db:backfill-grants` before `db:migrate` applies `0005`, from any migration state at or after `0001` (Drizzle's migrator cannot stop at `0004`; it applies every pending migration), then `db:migrate`; if anything is unbackfilled the guard refuses the drop (`apps/api/README.md`, "Legacy role backfill"). A fresh database needs no backfill. `memberships.team_id`, `legacy_teams` and the community tables stay until the community feature is retired, and the contracts keep `OrgRole` and `PlatformRole` for sample mode.

Phase 3 added `0006_sso_domains_rls.sql` (section 22.1). Phase 4 added four, each generated from the schema and, where noted, edited by hand:

- `0007_cases.sql`: the `cases(id, org_id)` table (`unique (id, org_id)`, tenant policy, forced), filled from the `(case_id, org_id)` pairs `conversations` and `intake_records` already used, then the composite foreign keys `conversations_case_org_fk` and `intake_records_case_org_fk`. A case id that two organizations share stops the migration at the foreign keys; correct those rows first.
- `0008_external_identities_sso_connection.sql`: `external_identities.sso_connection_id` (foreign key to `sso_connections`, `on delete restrict`, indexed), backfilled for every federated identity `cognito:<provider name>` from the connection that holds that provider name. The native `cognito` identities stay null, as does an identity whose provider name no connection holds. A federated identity linked by code older than the binding-connection change after this migration ran keeps a null binding and is refused by path 1 (section 22.4) until the same `UPDATE` is run again (section 22.15).

- `0009_sso_connections_oidc_claim.sql`: `sso_connections.oidc_email_verified_claim_required boolean not null default true`, so every existing connection keeps requiring the claim (section 22.4).
- `0010_organizations_status_check.sql`: `organizations_status_check`, `status in ('ONBOARDING','ACTIVE','SUSPENDED','ENDED')`. A row with any other value stops the migration; correct it first.

`test/db/schema.spec.ts` applies all eleven migrations, checks the checks and foreign keys, and migrates a database with phase 3 rows to `0006` first, then through `0007` and `0008`, to prove the two backfills.

11. Organization and operations administration

All `/orgs/:orgId/*` routes require the active context to be that organization (`orgResourceFromContext`). All `/ops/*` routes require a web session in the operations context (`operationsResource`, or `operationsOrgResource(type)` when the route names an organization). Writes run inside `withOrgScope` (or `withTransaction` for platform rows) and audit through `AuditWriter`.

Area	Routes and actions	Files
Invitations	`GET /orgs/:orgId/invitations` (pending, cursor), `POST /orgs/:orgId/invitations` and `DELETE .../:invitationId` (`people.invite`), `POST .../import` (`people.import`); public `GET /invitations/:token/preview`, `POST /invitations/accept`	`src/invitations/` (`invitations.service.ts`, `invitation-activation.service.ts`, `invitation-token.ts`, `invitation-mailer.ts`, a thin caller of the `Mailer` port, section 23)
Invitations for assigned clients	`GET /ops/orgs/:orgId/invitations`, `POST` and `DELETE .../:invitationId` (`people.onboard`), `POST .../import` (`people.import`)	`src/invitations/ops-invitations.controller.ts`
Organizations, operations side	`GET /ops/organizations` (`client.configure` at platform scope; cursor, name or slug prefix search), `GET /ops/organizations/:orgId` (`client.configure` through `operationsOrgResource`, so a `client_operations` user opens an assigned client)	`src/organizations/`
Org units	`GET /orgs/:orgId/org-units` (`program.overview` or `access.preview`), `POST`, `PATCH .../:unitId`, `POST .../:unitId/move` (`org_units.manage`)	`src/org-units/` (`closure-moves.ts` holds the pure closure-table algebra)
Members	`GET /orgs/:orgId/members` (`program.overview` or `access.preview`; cursor pagination, filters, `search` matching the start of the name or the email, at most 200 characters), `PATCH .../:membershipId` (`people.move`), `POST .../:membershipId/end` (`people.deactivate`; one's own membership 409 `cannot_end_self`)	`src/members/`
Members of assigned clients	`GET /ops/orgs/:orgId/members` (`client.configure`), `POST .../:membershipId/end` (`people.deactivate`)	`src/members/ops-members.controller.ts`
Grants	`GET /orgs/:orgId/grants` (`access.preview`; `?subjectUserId=&includeRevoked=`), `GET .../preview?userId=` (`access.preview`), `POST` and `PATCH .../:grantId` (`access.grant`), `POST .../:grantId/revoke` (`access.revoke`), `POST .../delegate` (`access.delegate`), `POST .../transfer-ownership` (`access.grant`, caller must hold the owner grant)	`src/grants/`
Approval authorities	`GET /orgs/:orgId/approval-authorities` (`access.preview`), `POST` and `PATCH .../:id` (`access.grant`)	`src/grants/approval-authorities.*`
Grants, operations side	`GET` and `POST /ops/grants` (platform grants; `access.preview`, `access.grant`), `POST /ops/grants/:grantId/revoke` (`access.revoke`); `GET /ops/orgs/:orgId/grants`, `GET .../preview`, `POST`, `POST .../:grantId/revoke`, `POST .../transfer-ownership` (`access.grant`); `GET /ops/users/:userId/access` (`access.preview`)	`src/grants/ops-grants.controller.ts`
Users, operations side	`POST /ops/users/:userId/suspend`, `POST .../reinstate` (`people.deactivate`)	`src/users/`
Confidential domain	`GET /professional/conversations` (the caller's assigned conversations) and `GET /professional/me/conversations` (the participant's), metadata only, cursor; `GET /professional/conversations/:id` and `GET /professional/intake/:id` (`conversation.read.assigned`), `GET /professional/me/conversations/:id` (`conversation.participate`); `@Authenticated()`, authorized in the service (the lists run the engine per row, without audit or step-up)	`src/professional/`
Emergency access	`GET /ops/emergency-access` (`audit.read`, optional `?userId=`), `POST` (`access.emergency.request`), `POST .../:id/approve`, `.../:id/deny`, `.../:id/revoke` (`access.emergency.approve`), `GET .../:id/records/:recordType/:recordId` (`@Authenticated()`, `records.read.emergency` in the service)	`src/emergency/`

The routes for assigned clients use different actions from their `/orgs` twins because the `client_operations` template holds `people.onboard` and `client.configure`, while `people.invite` and `program.overview` are HR actions (strategy 4.4). The ops invitation routes validate bodies with `ZodValidationPipe` and the `/orgs` invitation routes with a local helper, so their 400 bodies differ slightly.

Invitation tokens are 32 random bytes; only the SHA-256 hash is stored, with a 7-day expiry. An invitation carries only an active template of the `organization` or `employee` workspace (`findInvitableTemplate` in `src/grants/grant-rules.ts`): create answers 400 `validation` with `details.templateKey` for an operations or expert template, import refuses such rows with `invalid_template` and creates the rest, and acceptance refuses a stored invitation with another template as `invalid_invitation`, so an invitation can never mint a staff or platform grant. The invitation mail (template `invitation`, section 23) goes out after the transaction that created the invitation has committed; a mail that fails is logged and the invitation stands, so an administrator can revoke it and invite again. Without `SES_FROM_EMAIL` nothing is sent and the logging mailer records only the template, the locale and the recipient's domain; outside production the response of a single create (not of an import) then carries `devToken`, from which the link `{WORKSPACE_URL}/invitations/{devToken}` can be built by hand. Every AWS stage runs with `NODE_ENV=production` (section 18), so no `devToken` leaves a developer machine.

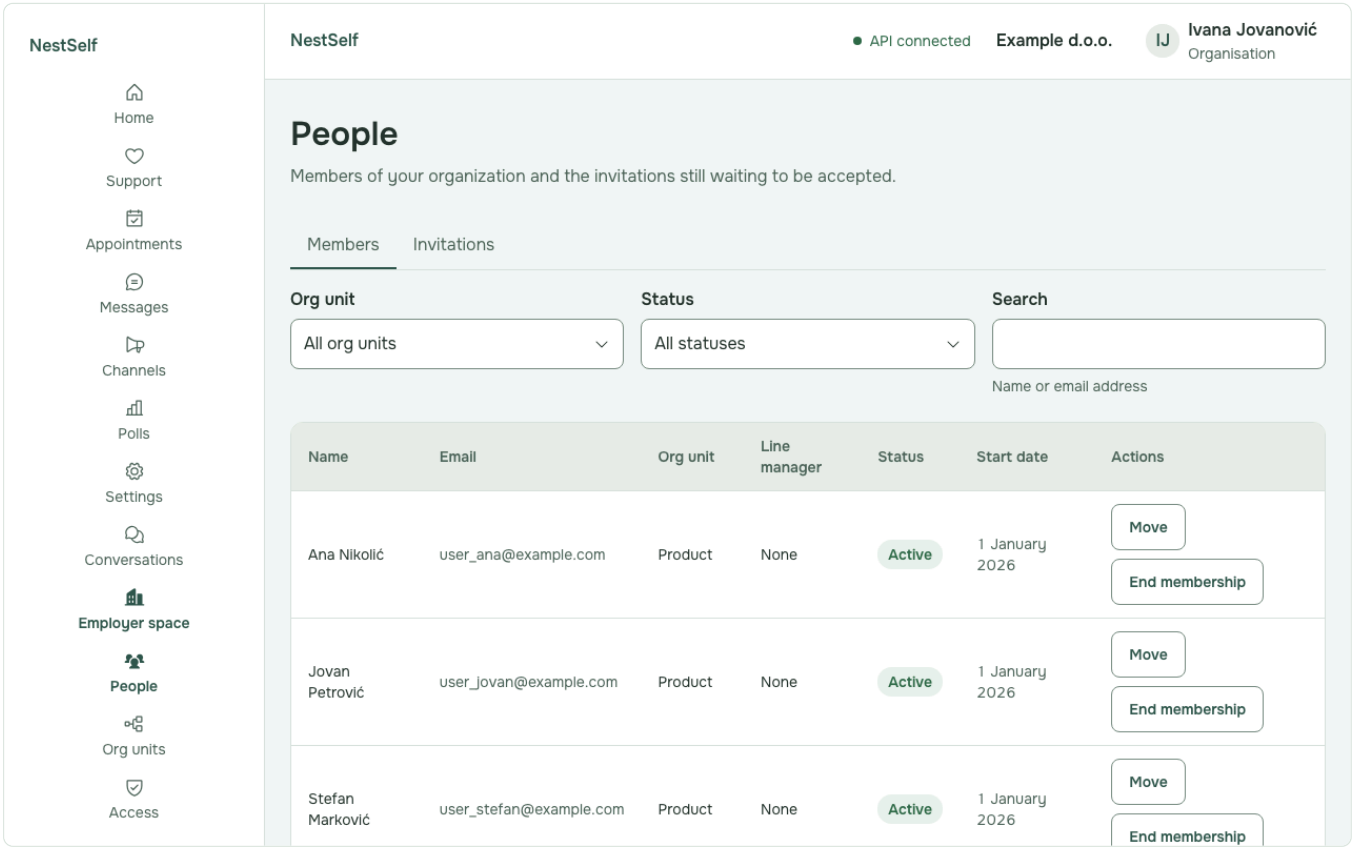
Import buckets (`InvitationsService.import` , `InvitationImportRowReason` in `packages/contracts/src/org-admin.ts`). Up to 500 rows; each lands in exactly one bucket, decided by the first check it fails, so the three add up to the rows sent:

- `created` : a new invitation, audited (`invitation.created` with the batch id) in the import's transaction.
- `skipped` (a count): the address already has a pending invitation or is an active member, so there is nothing to do.
- `errors` (in the order of the rows sent, `{ email, reason }`): `invalid_template` , `duplicate_in_batch` , `sso_domain_claimed` (the domain belongs to another organization's `ACTIVE` connection), `org_unit_outside_org` , `owner_exists` (an `employer_owner` row while the organization has an owner). The contract's reason enum also lists `duplicate_invitation` and `already_member` , but the import counts such rows as skipped and never names them; the single create answers them as 409 codes.

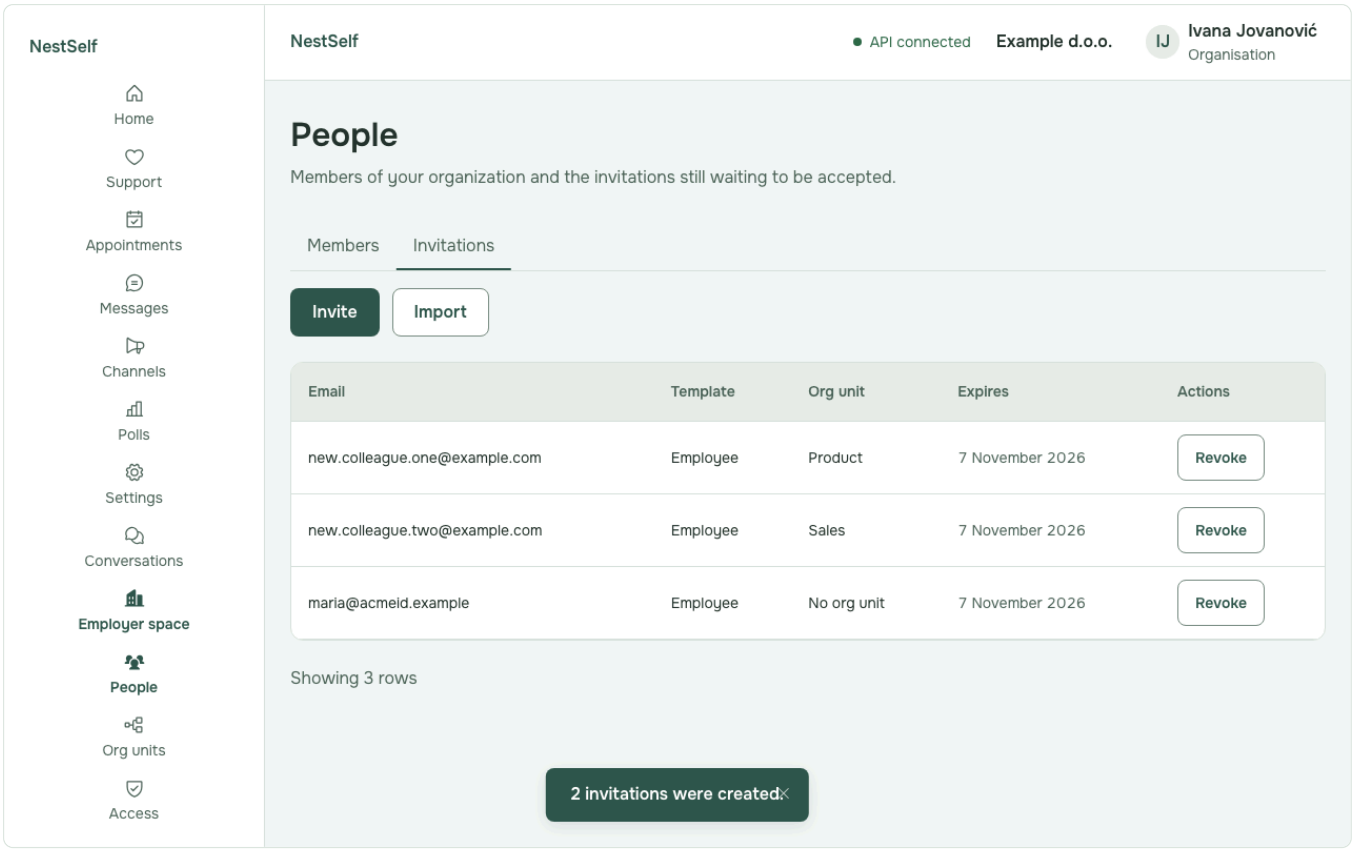
A repeated address marks its later rows `duplicate_in_batch` and processes the first. The first row claims the address even when a later check refuses it, so a valid later row for the same address is then `duplicate_in_batch` too (accepted for now; claiming at the first processable row would be friendlier). The invitation mails go out after the commit, for the created rows only, all at once and without retry (section 21).

Membership end sets `ENDED` , revokes the user's grants scoped to the organization and the whole delegation tree below them (`revokeGrantTree` : roots record `membership_ended` , descendants `parent_revoked`), audits first, invalidates every delegatee, then calls `revokeAllAccess` (SESS-R-22, SESS-R-23). Suspension sets `users.suspended_at` , disables the identity-provider user, audits `user.suspended` (SECURITY tier) and revokes all access; `authorize()` denies suspended users on the next request anyway.

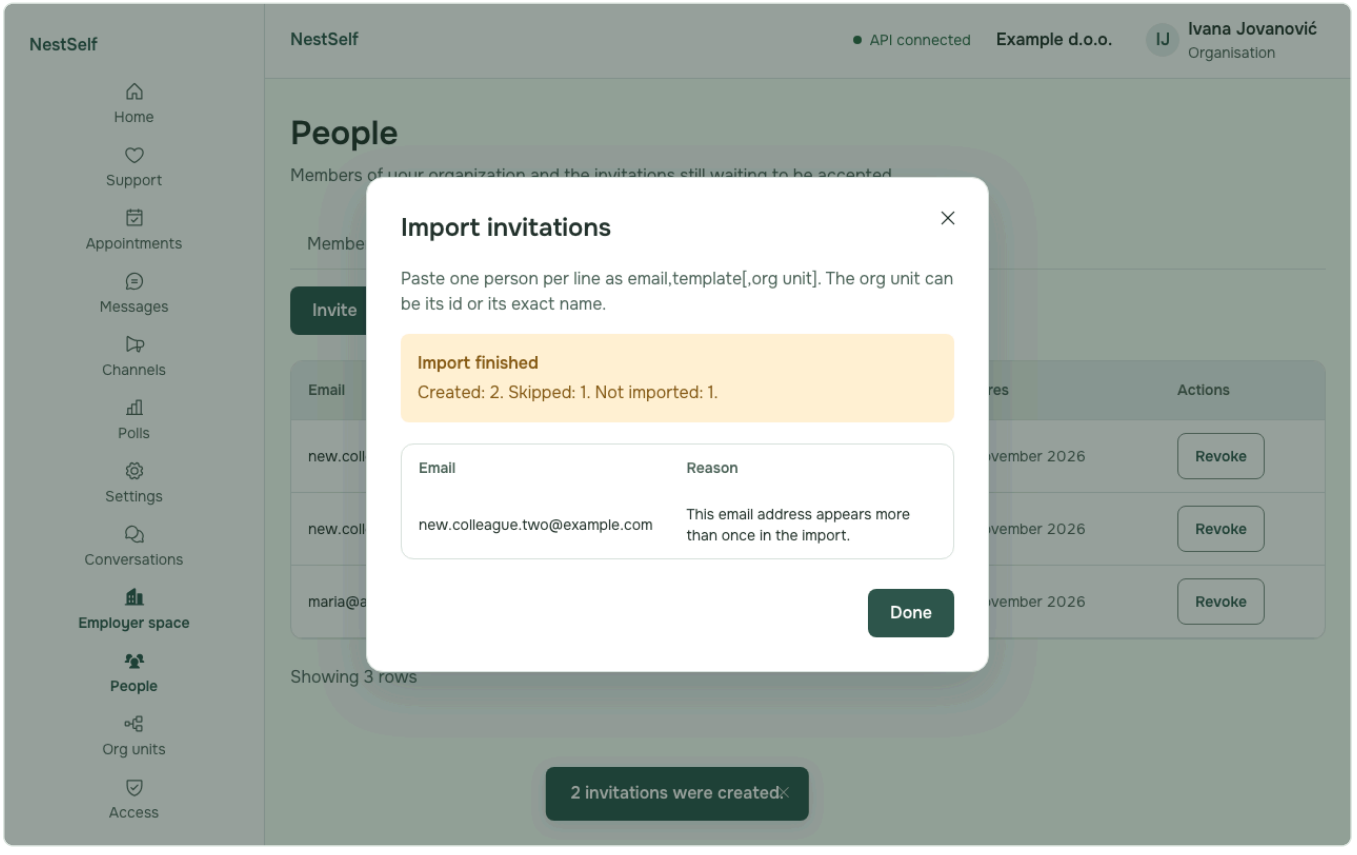
People and org units in the workspace. `/hr/people` (`features/people/`) has two tabs. *Members* lists the organization's members page by page with the API cursor ("Load more", never capped), filtered by org unit, status and a debounced name or email search, with the org unit and the line manager by name. Move (`people.move`) changes the org unit and the line manager and sends only what changed; End membership (`people.deactivate` , administration step-up) asks for a reason and is not offered on one's own row or on ended members. *Invitations* lists the pending invitations; Invite takes an email address, an invitable template (active `organization` and `employee` templates, the same rule as the server) and an optional org unit; Import takes pasted `email,template[,org unit]` lines (unit by id or exact name, up to 500 rows), shows the problems line by line and sends nothing until they are fixed, then shows the three counts of the answer (created, skipped, not imported) and a table of the rows not imported with their translated reasons (`people.import.reasons.*` through `importReasonKey` in `features/people/people-errors.ts` ; an unknown reason renders the fallback copy, never the API's words); Revoke ends a pending invitation. `/hr/org-units` (`features/org-units/` , `org_units.manage`) shows the tree indented by depth with Rename, Add child (department or team) and Move (to any unit that is not itself, below itself or its current parent; the company root does not move).



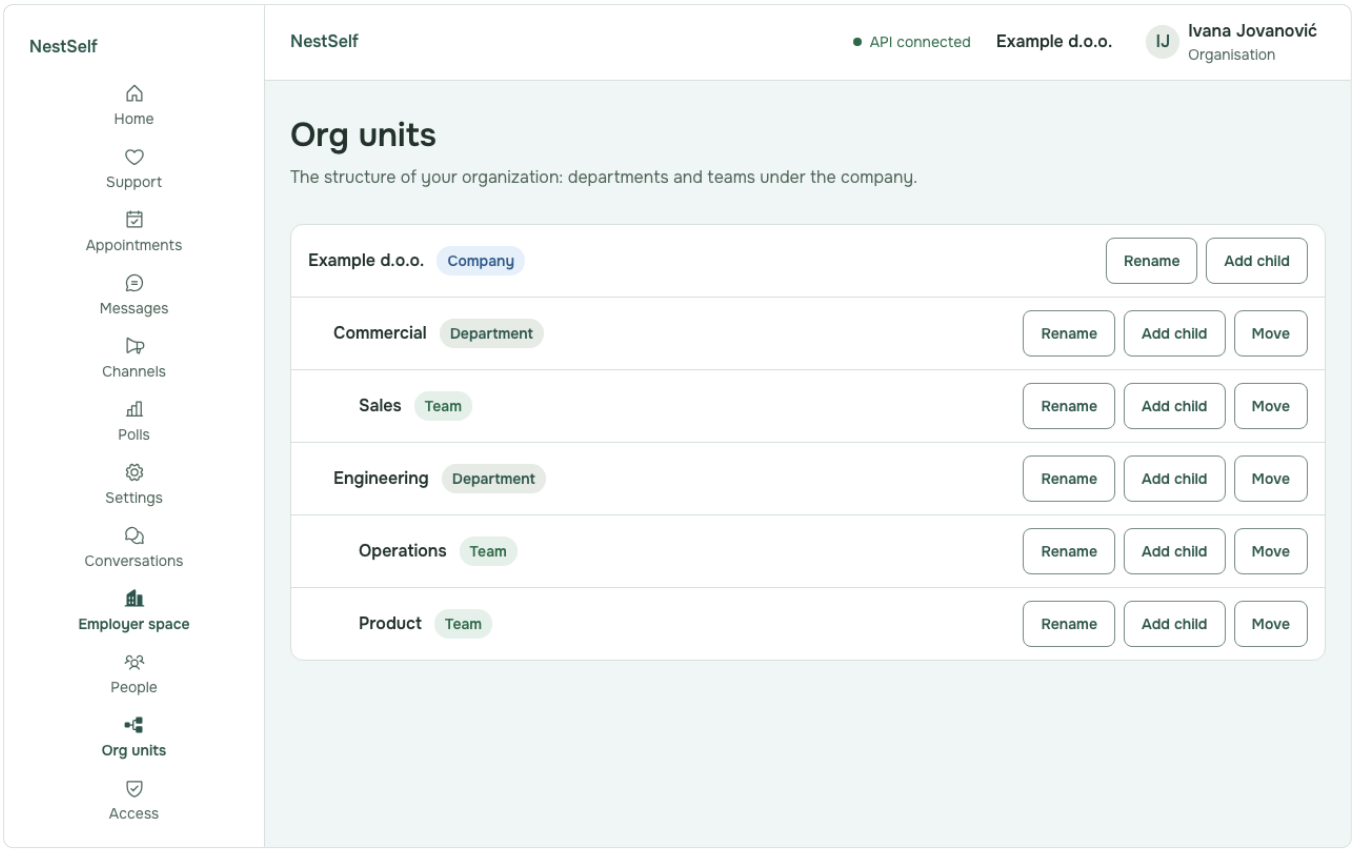
`/hr/people` , Members: filters, the member table with org unit, line manager and status, Move and End membership per row.



The Invitations tab with the two pending invitations the import below created in the disposable e2e database, and the one the SSO screenshots staged for Maria.



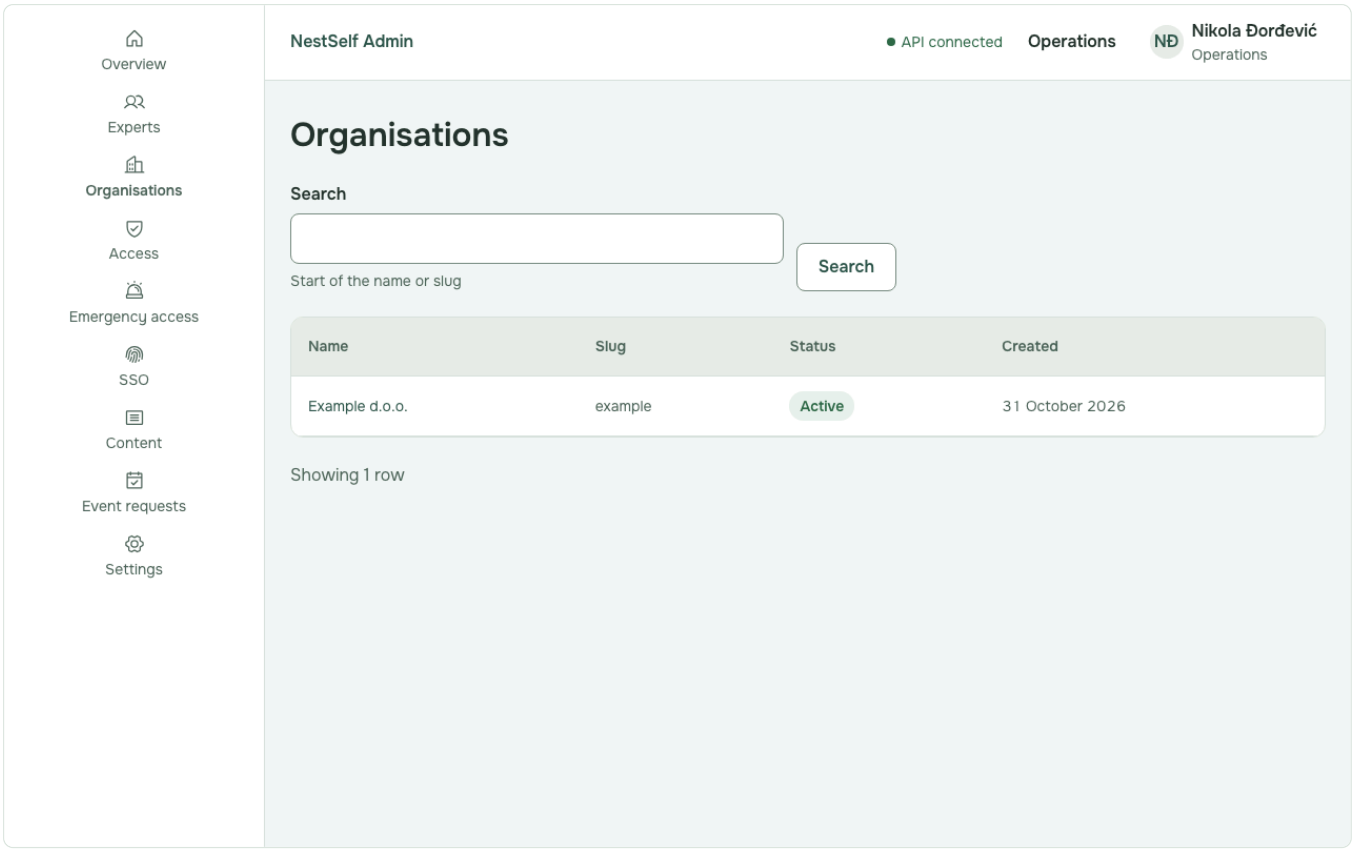
Import result, one row per bucket at least: two invitations created; Ana Nikolić's row skipped, because she is already an active member (a count, not a refusal); the second row for `new.colleague.two@example.com` not imported, because the address appears twice in the list. Only the not-imported row is in the table.



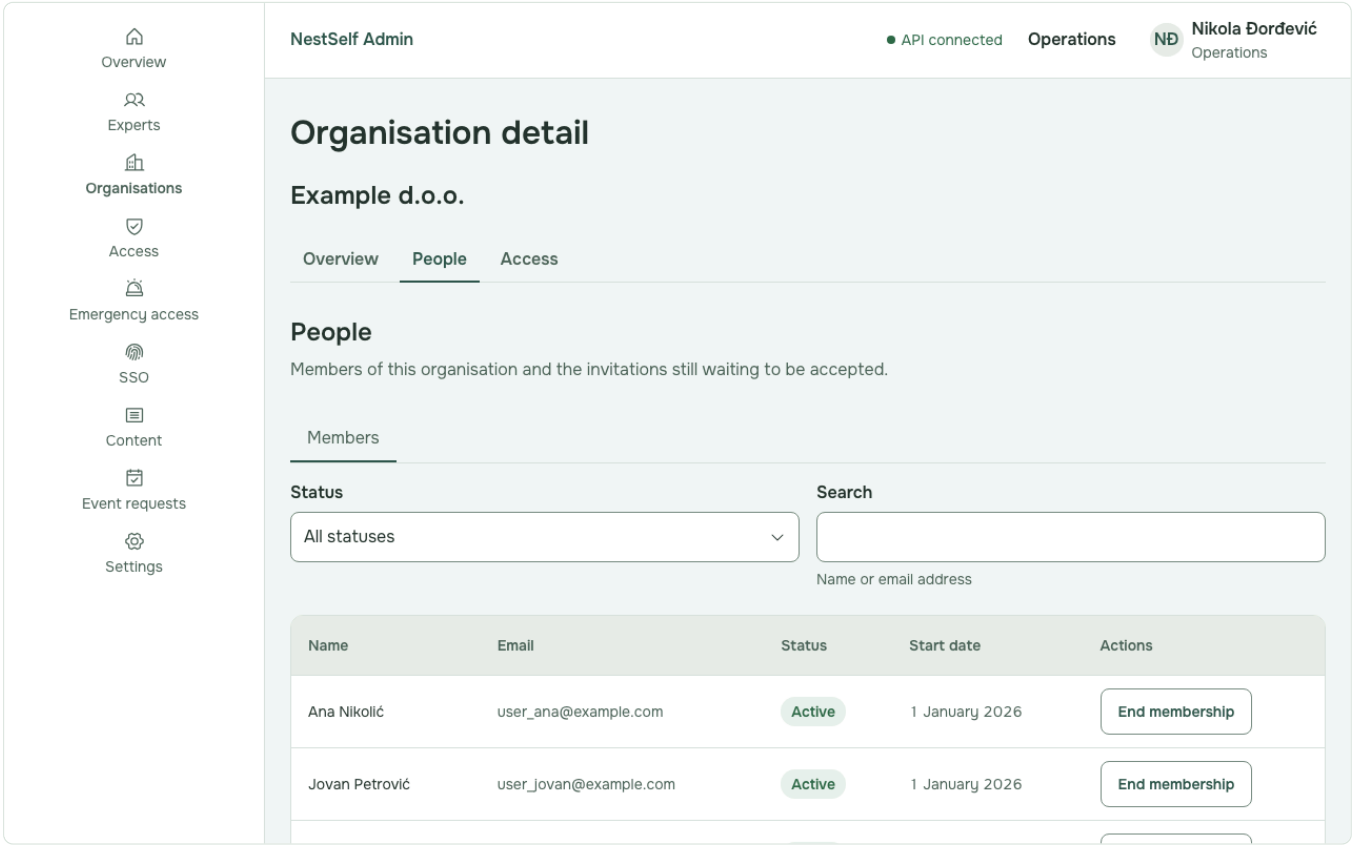
`/hr/org-units`: the sample tree under the company, Example d.o.o.

`PATCH /orgs/:orgId/members/:id` returns the full member item, the member's name and email included, since 2026-10-25. It used to answer with empty strings, which the client's `MemberItemSchema` rejects, so every successful Move showed "Something went wrong"; the Playwright journey `hr-people.spec.ts` found it.

Organizations and people in the admin app. `/organizations` (`features/organizations/`) lists organizations with a prefix search and "Load more"; a caller without the platform-scoped list (a `client_operations` user) sees a note that access is limited to assigned clients and opens them by link. The detail page has the tabs Overview (id, slug, status, the owner from the active `employer_owner` grant and Transfer ownership through the staff route, also when there is no owner yet), People and Access (section 11.1). People (`client.configure`) lists the members with a status filter and search and offers End membership with a reason (`SUSPENDED_BY_PLATFORM` among them); its Invitations tab (`people.onboard`, import with `people.import`) mirrors the workspace one at organization level, because the ops API has no org-unit routes. `platform_owner` holds no `people.onboard`, so a platform owner sees Members only.

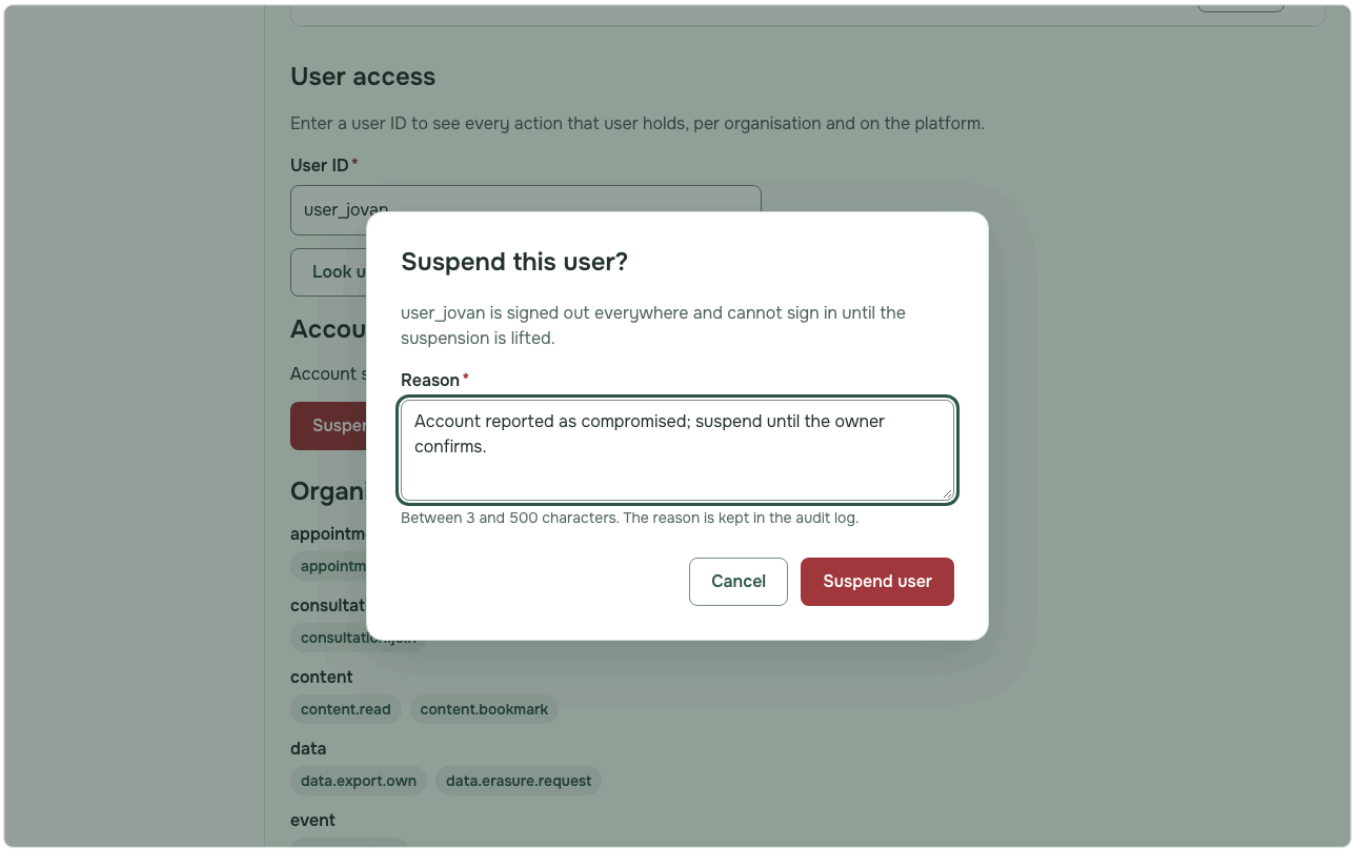


Admin `/organizations` for a platform owner: the sample organization with its slug, status and creation date.



An organization's People tab in the admin app: members with End membership.

Suspend and reinstate. Once the user-access lookup (`/access` , and the Access tab of an organization) returns, `features/users/user-account-actions.tsx` shows the account state and Suspend user (reason of 3 to 500 characters) or Reinstate user, both `people.deactivate` and through `withStepUp` . For one's own account Suspend is replaced by a note, and the API refuses it too with 409 `cannot_suspend_self` . Reinstatement now invalidates the cached principal, so a reinstated user can sign in at once.

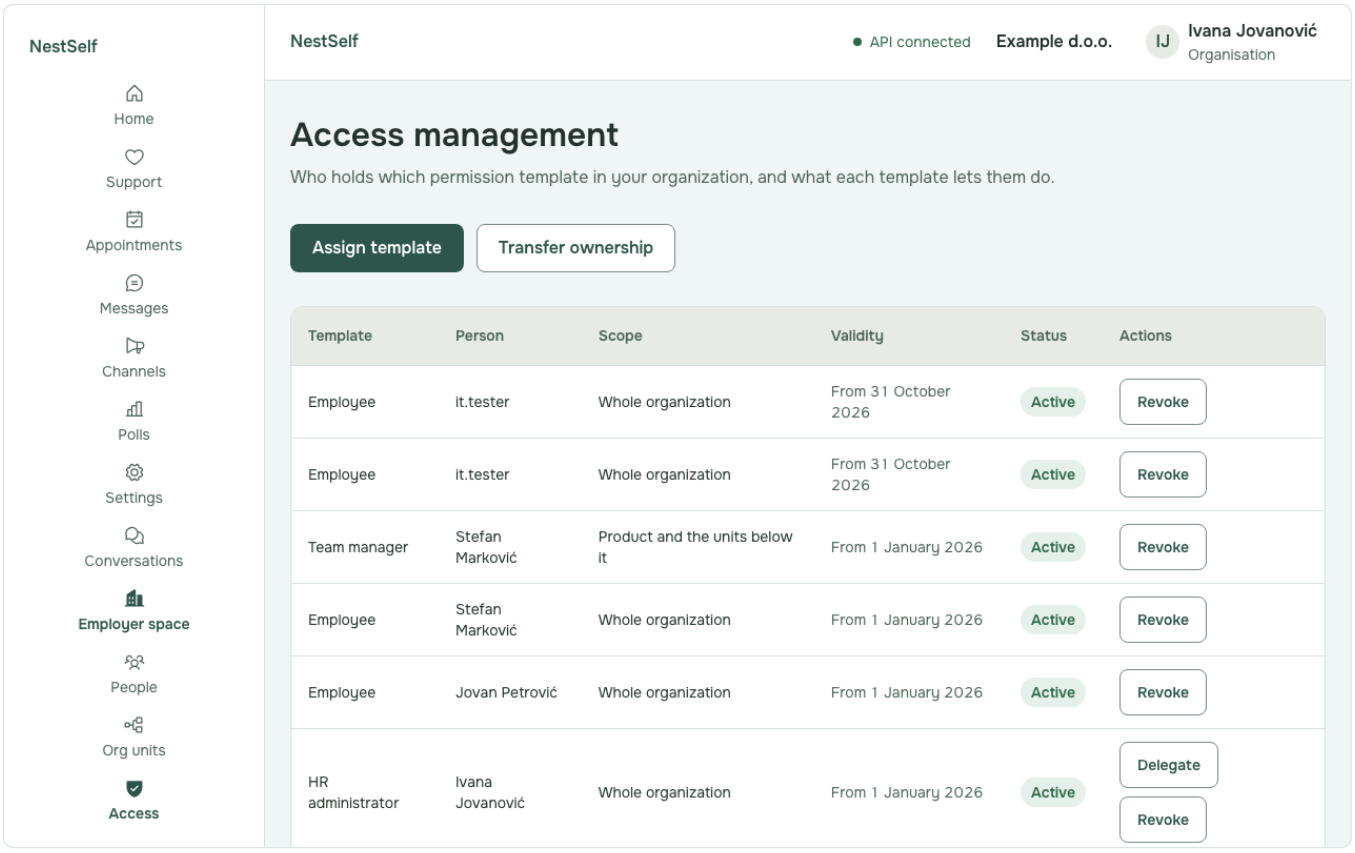


Suspend this user, opened from the lookup of `user_jovan` and not confirmed.

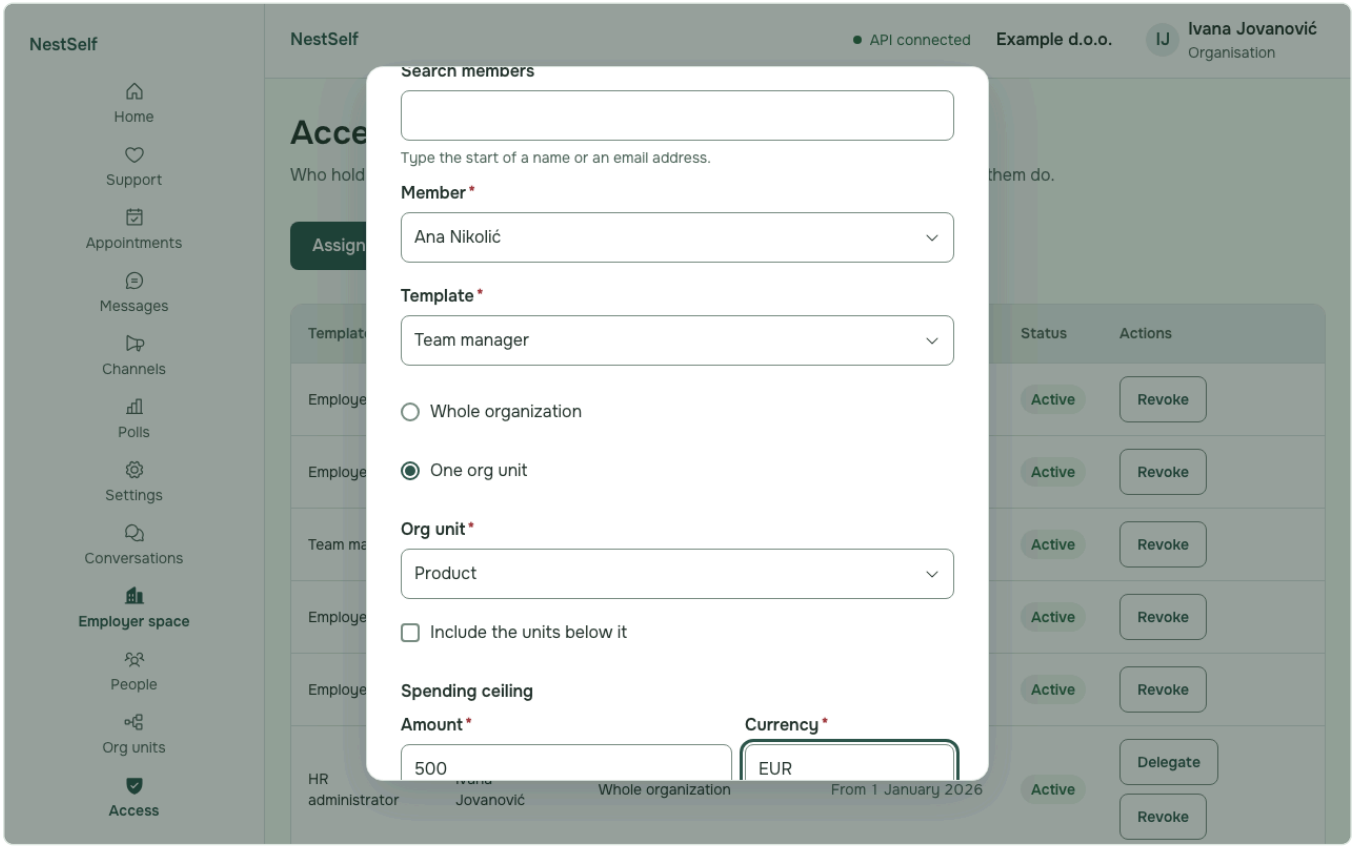
`people.deactivate` is held by three templates: `hr_admin` (organization scope, ends memberships of its own organization through `/orgs/:orgId/members/:id/end`), `platform_owner` (platform scope, the only template that reaches `/ops/users/:id/suspend`, whose resource has no organization) and `client_operations` (scope `assigned_clients`, through `/ops/orgs/:orgId/members/:id/end` for an organization the caller holds a CLIENT assignment to). The sample data has no platform owner; tests create that grant from the template.

11.1 Grant administration

Create (`POST /orgs/:orgId/grants`, `{ templateKey, subjectUserId, scopeType, scopeOrgUnitId?, includeDescendants?, conditions, validFrom?, validUntil? }`). The template must be active (400 `template_inactive`) and belong to the `organization` workspace (400 `validation`). The scope is `organization` or `org_unit`, with a unit of this organization (404 otherwise). The subject must hold an ACTIVE membership (409 `subject_not_member`). Every `requiredConditionTypes` entry of the template must be present (400 `missing_conditions`, `details.missing`). Actions are copied from the template, `validFrom` defaults to now and `validUntil` must come after it. A second active `employer_owner` grant is refused with 409 `owner_exists` (see "Organization ownership" below).



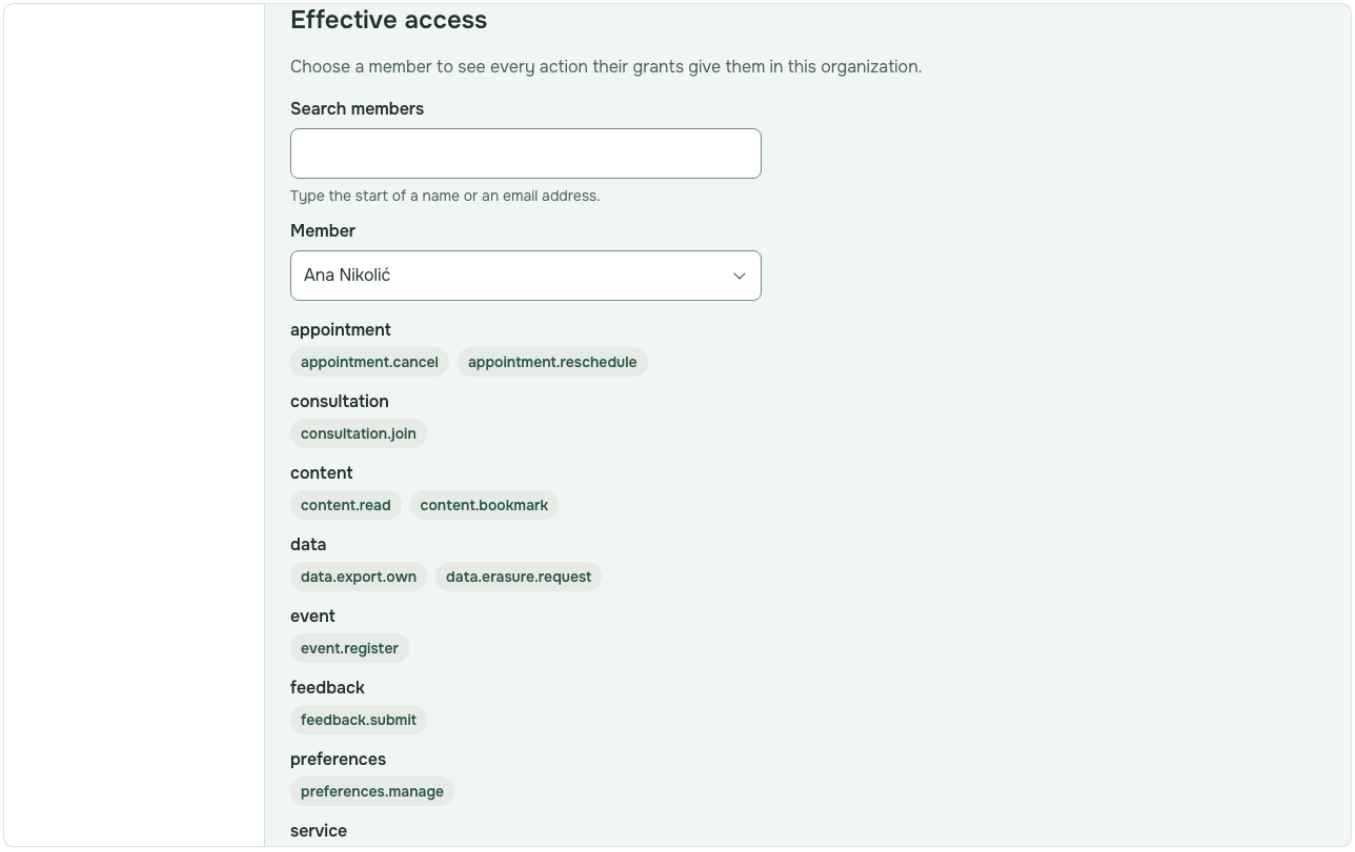
Workspace `/hr/access` for Ivana Jovanović, the sample HR administrator and employer owner: the organization's grants with Delegate and Revoke per row, and Assign template and Transfer ownership above the table. The two `it.tester` rows are the employee grants of the SSO screenshots' test sign-ins.



Assign: a `team_manager` grant for Ana Nikolić on the Product unit, with the spending ceiling the template requires. "Search members" above the member choice asks the API for the members whose name or email starts with the typed text.

Update (`PATCH`) changes conditions and the validity window only. A revoked grant answers 409 `conflict` , another organization's grant 404. When the patch changes conditions and the grant has active delegated children, each child is checked against the patched parent; if any falls outside, the update is refused with 409 `conflict` and `details.children` naming them, so the administrator adjusts or revokes those first. A shorter validity window is not checked against the children: the engine denies them at runtime once the parent has expired.

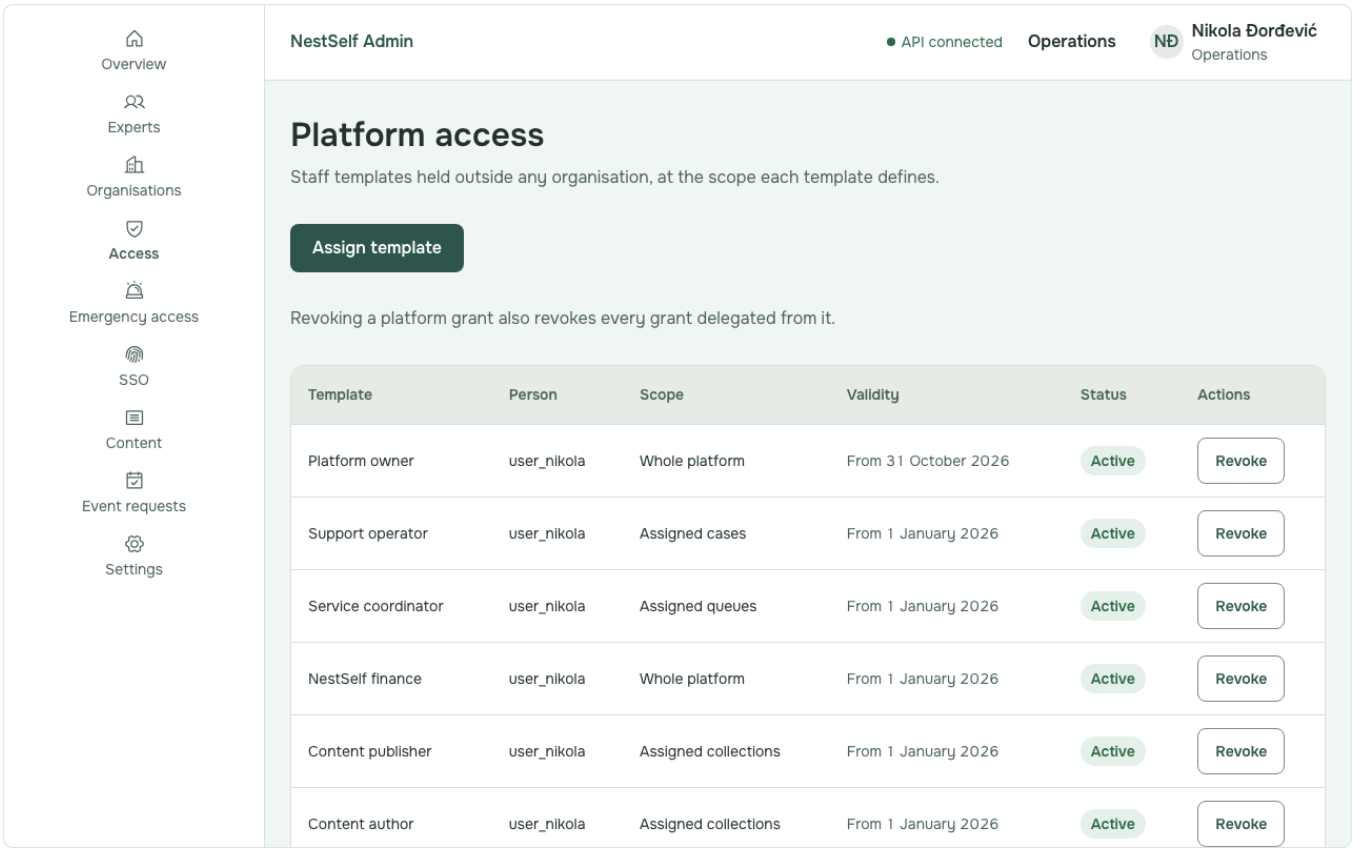
Revoke takes a reason (3 to 500 characters) and revokes the grant and every descendant through `revokeGrantTree`; an already revoked grant answers 409. **Preview** (`?userId=`) requires the subject to be an active member (404 otherwise), runs `effectiveAccess` for the subject in this organization and drops the confidential actions (AUDIT-R-02).



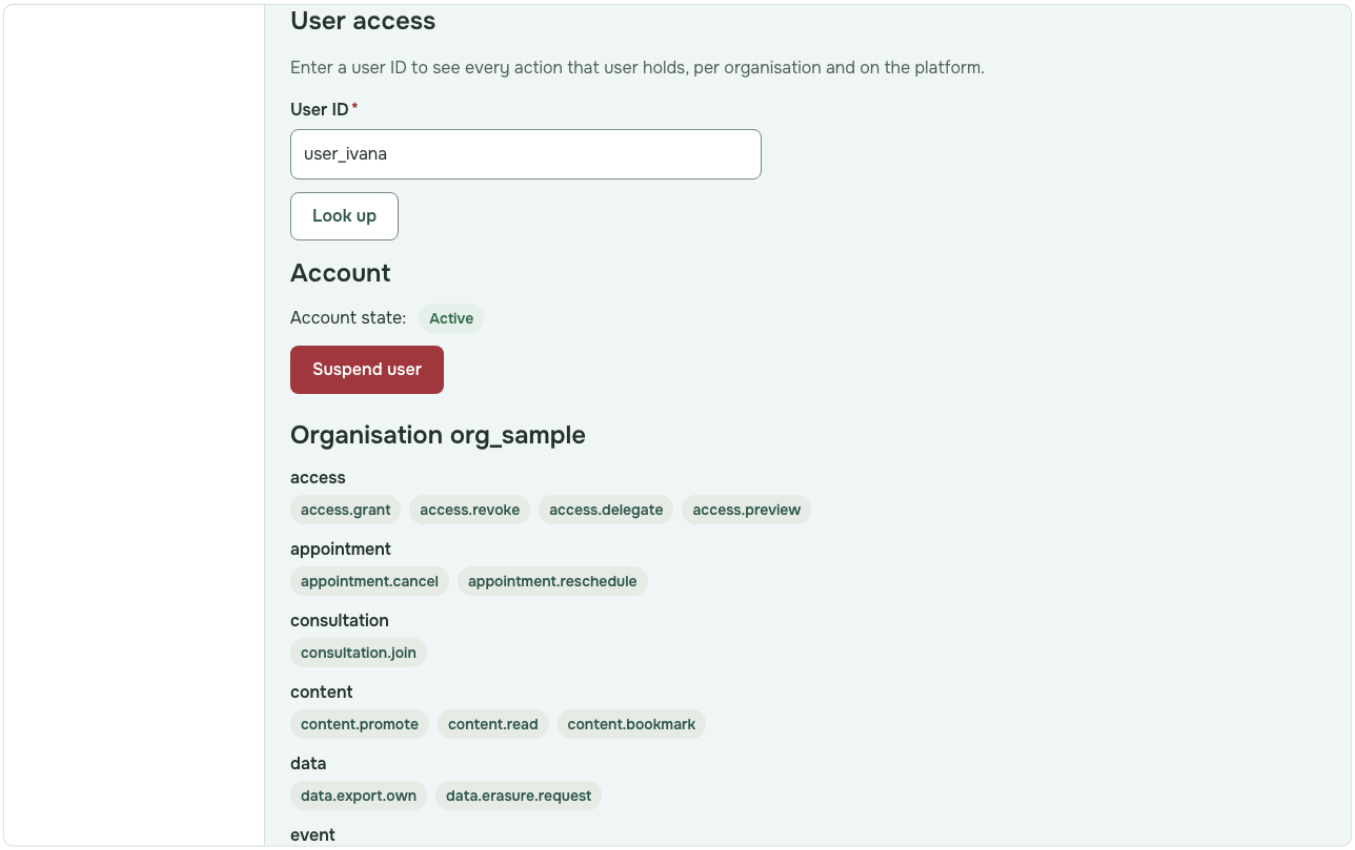
The effective-access panel of `/hr/access`: every action Ana Nikolić's grants give her in the organization, grouped by prefix, with the member search above the choice.

Every write records one audit event (`grant.created`, `grant.adjusted`, `grant.revoked`, `grant.delegation_revoked` per descendant, `grant.delegated`) with field names, counts and condition types, never values, and invalidates the subject and every delegatee.

Operations side. `POST /ops/orgs/:orgId/grants`, its list, preview and revoke use the same service. `POST /ops/grants` creates a platform grant (`scope_org_id` null) from an `operations` workspace template at the template's own scope type, inside `DbService.withTransaction` with its audit call. `POST /ops/grants/:grantId/revoke` (reason body, write throttle) revokes a platform grant and its descendants: 404 `not_found` for a missing or organization-scoped grant (those go through `/ops/orgs/:orgId/grants/:grantId/revoke`), 409 `conflict` when already revoked; otherwise `revokeGrantTree`, `grant.revoked` and one `grant.delegation_revoked` per descendant in one system transaction, then the subject is invalidated for the platform, their membership organizations and their CLIENT organizations (the same set as on create) and every delegatee below. This is how a member of staff is off-boarded from the operations workspace. `GET /ops/users/:userId/access` returns one context per active membership plus a platform context, each with actions (confidential ones dropped), grant ids and assignment ids. The ops side has no org-unit, update, delegation or approval-authority routes.



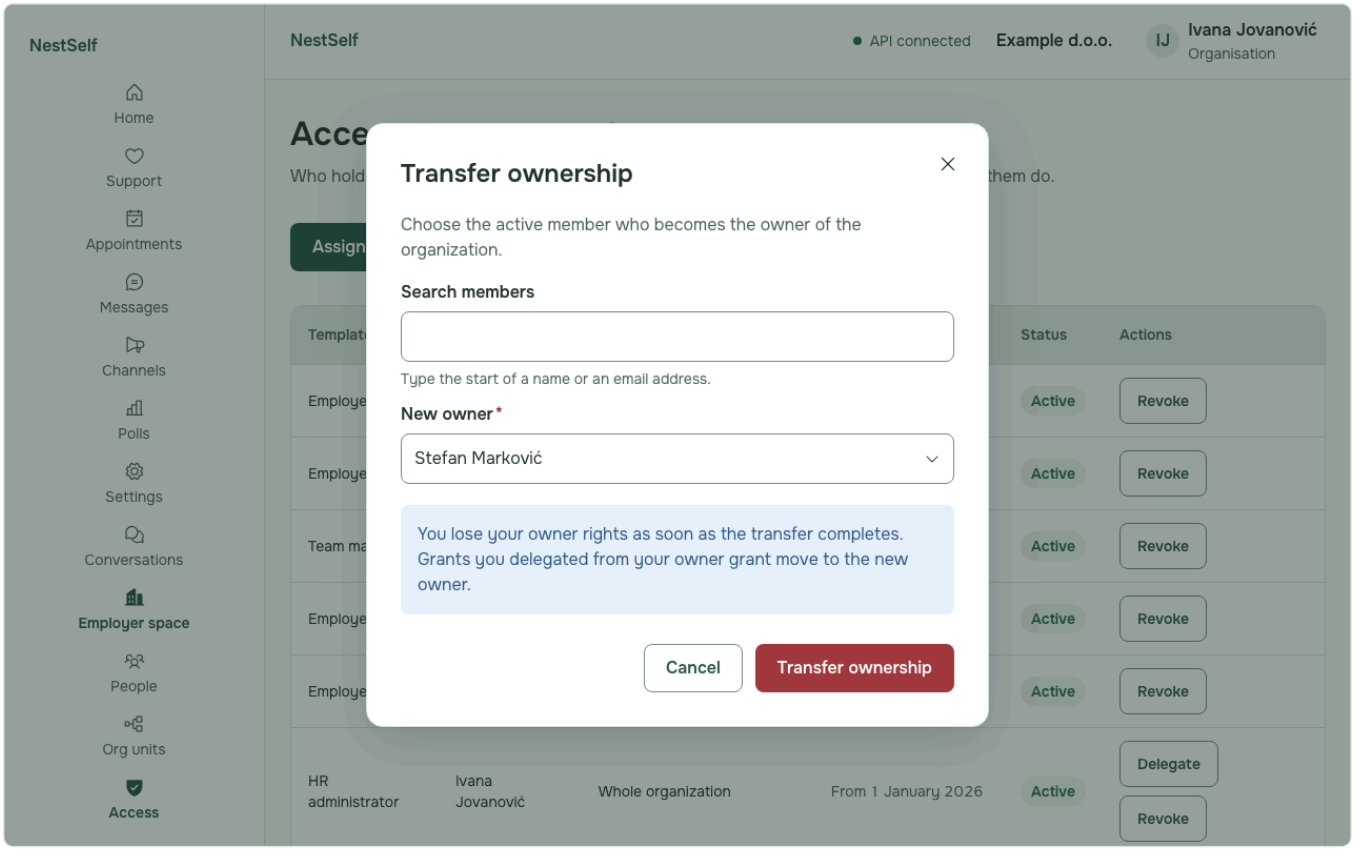
Admin `/access`: platform grants with revoke. The first row is the `platform_owner` grant the screenshot run gives Nikola Đorđević; the sample data has none (section 19).



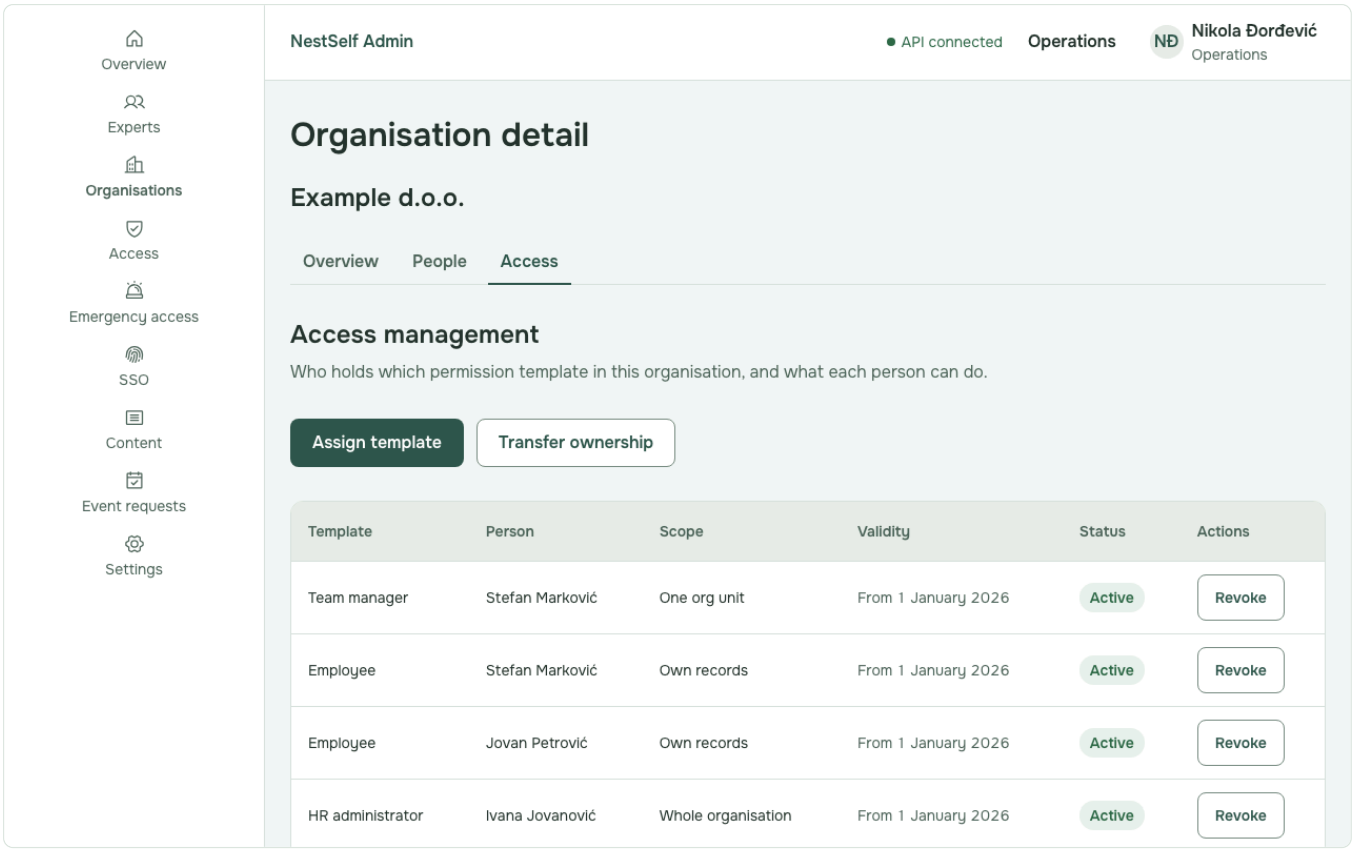
The user-access lookup (`GET /ops/users/:userId/access`) for `user_ivana`: one section per context.

Organization ownership (owner model, product owner decision of 2026-10-24; strategy 4.4 amendment). `employer_owner` is the single superadmin of an organization. It holds `access.grant`, `access.revoke`, `access.delegate`, `access.preview`, `policy.manage`, `service.configure` and `org.settings`, and its three `access.*` writes are in the `administration` step-up class. `hr_admin` holds none of the `access.*` actions; the owner gives an HR administrator grant and revoke rights by assigning or delegating them. NestSelf staff with `platform_owner` keep every right through the `/ops/orgs/:orgId/*` routes, so support can always change anything for an organization.

- **One owner per organization.** `GrantsService` keeps at most one active (unrevoked, not expired) `employer_owner` grant per `scopeOrgId`. `AuthorizationRepository.findActiveOwnerGrants` takes a transaction-scoped advisory lock on the organization's ownership and returns the active owner grants; create (both route families), a PATCH of an owner grant's window, the transfers, and invitation create, import and acceptance refuse a second owner with 409 `owner_exists` and `details.currentOwnerUserId`. A grant whose window starts later already counts, so a future-dated second owner is refused too, and a PATCH cannot revive an expired owner grant next to the current one.
- **Transfer by the owner**: `POST /orgs/:orgId/grants/transfer-ownership` with `{ newOwnerUserId }` (`access.grant`, so step-up applies). The caller must hold the started owner grant (403 `forbidden` otherwise), the new owner must be an ACTIVE member (409 `subject_not_member`) and must not be the current owner (409 `conflict`).
- **Transfer by staff**: `POST /ops/orgs/:orgId/grants/transfer-ownership` (`access.grant` through `operationsOrgResource`) has no current-owner requirement and creates the first owner when the organization has none.
- **One transaction** (`transferOwnership`, `withOrgScope`): revoke the old owner grant only (reason `ownership_transferred`), create the new one from the template, and re-parent the old grant's active children to the new grant (`AuthorizationRepository.reparentActiveChildren`). The bounds still hold because the new grant has the same template and actions; without the move the engine would deny the children (revoked parent) and the nightly sweep would end them. Audit: `grant.revoked` and `grant.created` with `details.transfer = true` (plus `byStaff = true` on the ops route), and one `grant.adjusted` per moved child with `details.reparented = true` and `fromGrantId`. The response is `OwnershipTransferResponse` (`revokedGrantId`, null when staff created the first owner, and `createdGrant`). The old and new owners, the moved children's subjects and the delegatees below both grants are invalidated afterwards.
- The seed makes the sample HR admin, Ivana Jovanović, the owner of `org_sample` (`grant_user_ivana_employer_owner`), which is what opens `/hr/access` in development and in `pnpm e2e:auth`.
- **An owner needs no HR template** (phase 4). The workspace route `/hr/access` is gated by `access.preview` alone, which `employer_owner` holds, and the member and org-unit lists the page reads accept `access.preview` as an alternative to `program.overview` (section 4), so an owner who is not also an HR administrator manages access with their own session; the rest of `/hr` stays behind `program.overview`. The journey `hr-access.spec.ts` proves it with Stefan Marković as a temporary owner (`e2e/README.md`).



Transfer ownership in the workspace, shown only to the current owner: the new owner is chosen from the active members (searchable), and the note says what happens to the owner's rights and delegations.



The admin organization Access tab: staff assign organization-wide templates, revoke and transfer ownership for the organization.

Delegation (`POST /orgs/:orgId/grants/delegate`, `{ parentGrantId, subjectUserId, actions, scopeType?, scopeOrgUnitId?, includeDescendants?, conditions }`). The checks run in this order, and each failure answers 403 `delegation_exceeds_boundary` with a `details.reason` unless noted:

1. the parent grant belongs to this organization (404 otherwise);
2. the parent's subject is the caller (`not_parent_subject`);
3. the scope is `organization` or `org_unit` inside the organization (400 or 404);
4. the delegatee is an active member (`subject_not_member`);
5. the new depth is at most 3 (`delegation_depth_exceeded`);
6. the parent grant bounds the child: actions are a subset, the scope sits within the parent's, a spending ceiling is not higher and categories are a subset (`actions_exceed_parent`, `scope_exceeds_parent`, `spending_ceiling_exceeds_parent`, `categories_exceed_parent`; `grant-rules.ts`);
7. `validateDelegation` over the parent chain loaded from the database, with the parent's `delegationEnvelope`, when present, as an extra restriction.

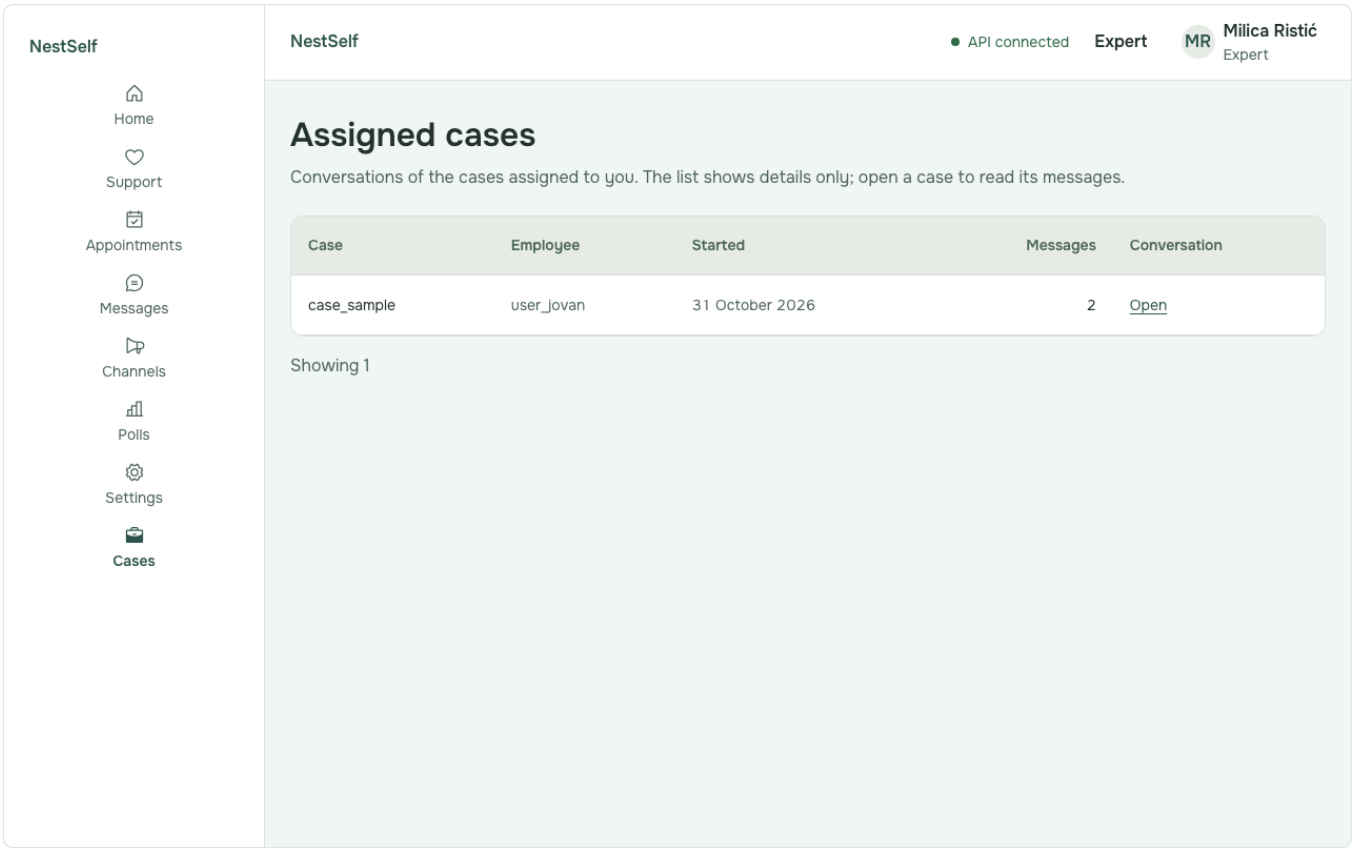
The delegated grant is stored with `templateKey: null`. Strategy AUTHZ-R-13 says a parent without an envelope may not be delegated from at all; here the parent grant itself is the bound in that case (AUTHZ-R-37), which is the reading the plan took. At runtime the delegatee's context carries the ancestor chain (section 9), so the delegated actions apply to them, and the revocation or expiry of any ancestor ends them at once, before a job runs.

Approval authorities. The approver and the alternate must both be active members (409 `subject_not_member`), the alternate must differ from the approver (400 `validation`) and the org unit must belong to the organization. A PATCH never changes `userId`, `orgId`, `orgUnitId` or `delegatedFromId`; an empty PATCH writes nothing. Each write audits `approval_authority.changed` with `details.changedFields`.

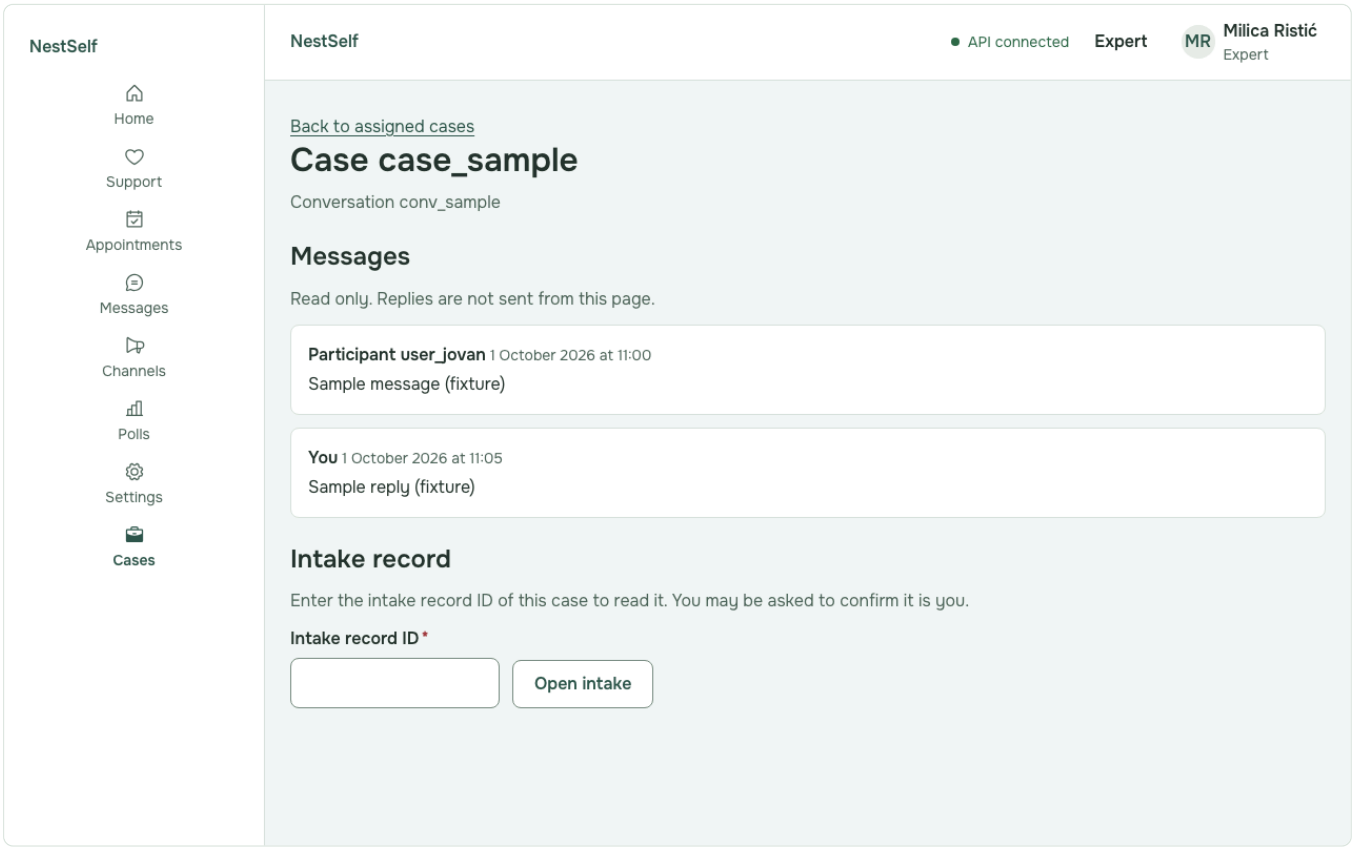
11.2 Confidential domain

`conversations`, `conversation_messages` and `intake_records` (`src/db/schema/professional.ts`, migration `0003`) are placeholder tables with the minimum columns, under forced RLS; since migration `0007` the first and the last reference `cases(id, org_id)` together with their own organization (section 10). No route writes them yet; the seed adds `case_sample`, then `conv_sample` and `intake_sample` for it (`SAMPLE_CASE`) with a CASE assignment for Milica. The service loads the row (404 when it is missing), calls `AuthorizationService.require()` with `{ type, id: caseId, orgId, ownerId: employeeUserId, isConfidential: true }` and `{ denyAs: 'not_found' }`, and only then reads the messages. A denial answers the same 404 `not_found` as a missing id, whatever the caller's role, so the answer no longer tells anyone whether a record exists (phase 4 design 4.4); the `access.denied` audit row is written as before, and a stale step-up stays 403 `step_up_required`. One residual remains: a refused read of an existing record writes that audit row and a missing id writes none, so the two can differ in timing. Two paths are allowed: the participant through `employee` (`own`) on `/professional/me/conversations/:id`, and the assigned expert through the `expert` template (`assigned_cases`) together with a CASE or SUPERVISION assignment whose `targetId` is the case id. The `expert` template gained `conversation.read.assigned` for that path on 2026-10-24. Organization, org-unit, platform and assigned-client grants are hard-denied (golden rows `CONF-DENY-01` to `06`). Bodies are never logged or audited.

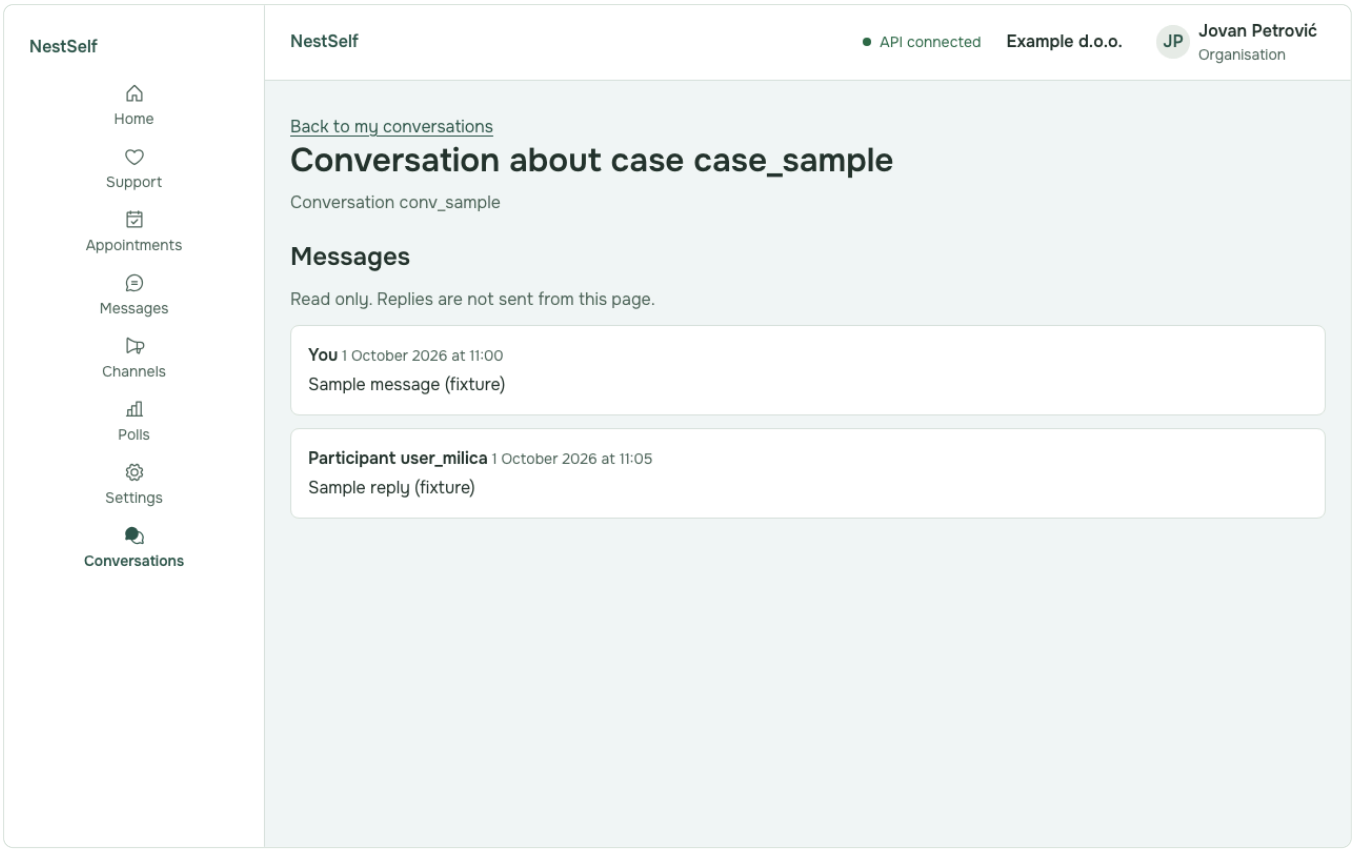
Expert area and participant view. In the workspace, `/cases` (`features/cases/`, route gate `principal.expert` or `conversation.read.assigned`) lists the conversations of the caller's assigned cases (case, employee id, start, message count) with "Load more", and switches an expert to the expert context once. `/cases/:conversationId` reads the conversation through `withStepUp(getConversation)` (the `restricted` class, 15 minutes) and shows the messages read-only, oldest first, with the viewer labelled "You"; a cancelled step-up shows "Nothing was opened" with Try again, and 403, 404 and 429 show copy. The intake section takes the intake record id, because neither the conversation nor the case points to its intake yet (section 21). Employees reach their own conversations under `/conversations` (`conversation.participate`, outside the expert context). Restricted reads are never cached (`gcTime: 0`, no refetch) and never logged. Participants appear as user ids, because the responses carry no names.



`/cases` for Milica Ristić, the sample expert: the conversation of `case_sample`.



The case conversation, read-only, with the intake lookup below. The bodies are the seed's fixture placeholders.



The same conversation as its participant, Jovan Petrović, sees it under `/conversations`.

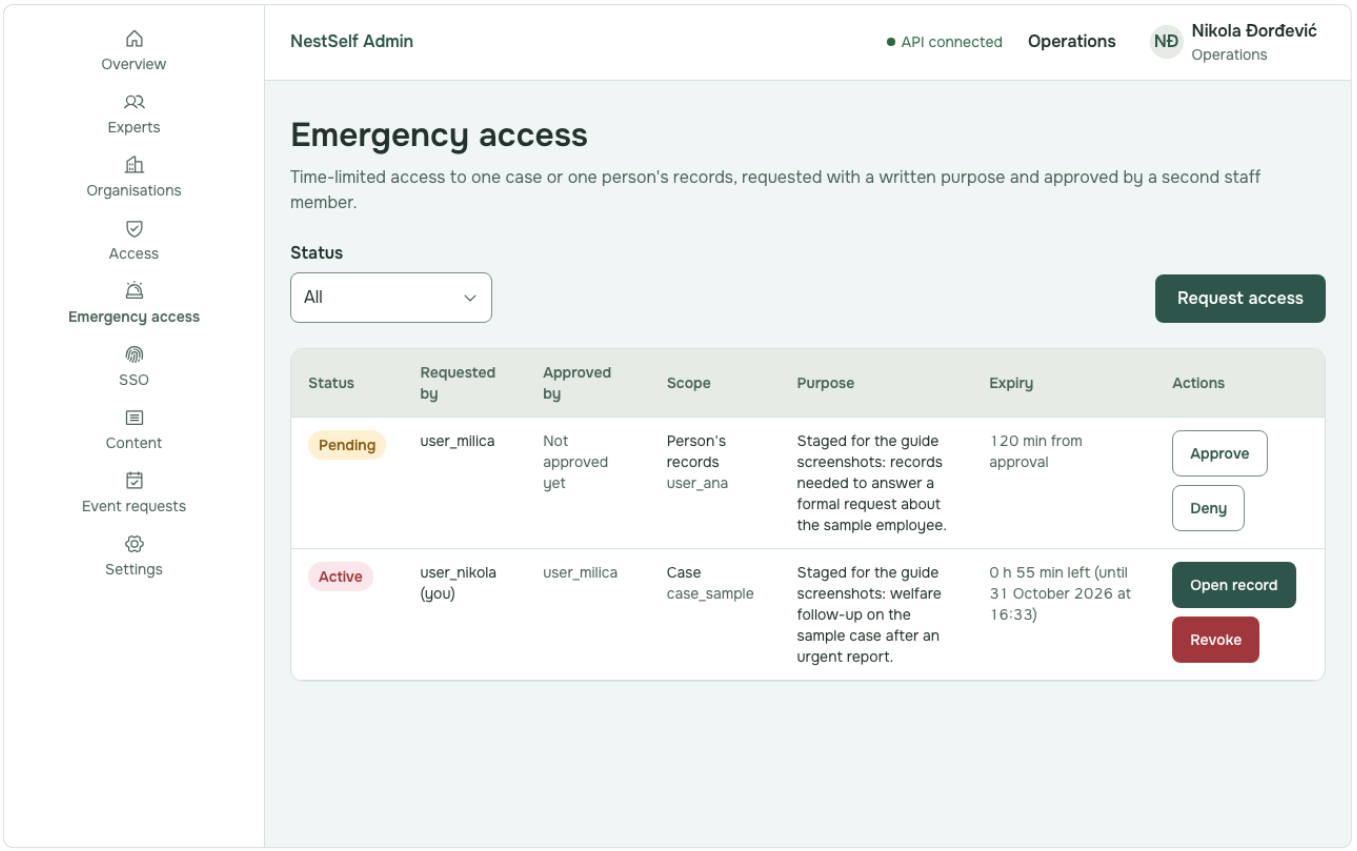
Cases are bound to their organization (phase 4, design 4.5). The CASE assignment match in `authorize()` compares `targetId` only, and `assignments.target_id` has no organization. Since migration `0007` a conversation or intake record can carry only a case of its own organization (`conversations_case_org_fk`, `intake_records_case_org_fk` over `(case_id, org_id)`), so a case id cannot lead from one organization to another's records; `ProfessionalRepository` and the seed insert the case first. This was the blocker before any conversation write path (it was listed in section 21 until phase 4).

11.3 Emergency access

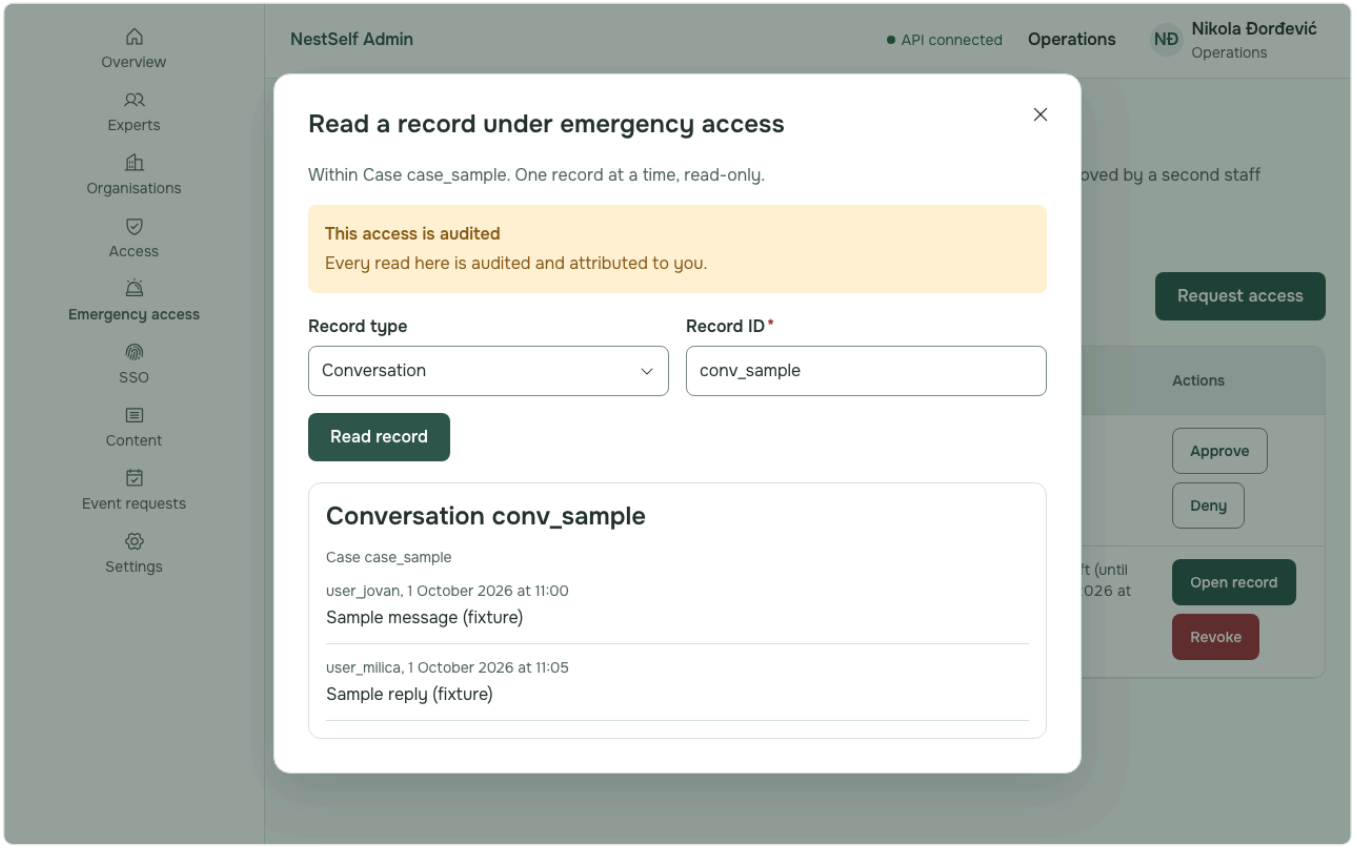
- **Request**: `POST /ops/emergency-access` with `{ purpose (10 to 1000 characters), scopeKind: 'CASE' | 'USER_RECORDS', scopeId, durationMinutes (15 to 240, default 240) }` creates a pending row (`approvedByUserId` null) and audits `emergency_access.requested` with the purpose truncated to 500 characters.
- **Approve**: a second staff member with `access.emergency.approve`. The requester gets 403 `self_approval_denied`, audited as `access.denied`; a row that is not pending answers 409. The window restarts at approval and ends `min(durationMinutes, 240)` minutes later. One pino `warn` line `{ action: 'emergency_access.approved', emergencyAccessId }` feeds the `EmergencyAccessActivations` metric filter of the observability stack.
- **Deny** deletes a pending row (204, `emergency_access.denied`); **revoke** closes an active one (`emergency_access.revoked`, 409 `emergency_not_active` otherwise).
- **Read**: `GET /ops/emergency-access/:id/records/:recordType/:recordId` (`conversation` or `intake`, `EmergencyService.readRecord`) needs an operations session, then checks in this order, so that neither the state of the access nor its scope lets a caller probe record ids: the access record must exist and be the caller's own (404 otherwise); it must be active, that is approved, not revoked and not expired (`isActive`; 403 `emergency_not_active`, answered the same for any record id, before any record is looked up); then the record is loaded (404 when it is missing) and must be covered by the access scope (404, like a missing one); then `require(principal, 'records.read.emergency', resource)`. The two refusals the engine never sees are audited as `access.denied` with `details.reason` `emergency_not_active` or `outside_emergency_scope` and identifiers only; the inactive-access row carries `orgId: null`, because no record was loaded, so a tenant-scoped audit query does not find it. As for the professional reads, a refusal outside the scope writes a row that a missing id does not. A read returns the same view as the professional route and audits `emergency_access.used`. There is no list of records (CONF-R-09).

`findActiveEmergencyAccess` ignores unapproved rows and the engine checks `expiresAt` on every read, so an expired window is denied before the sweeper (section 14) closes the row. Request and approve are in the `emergency` step-up class (5 minutes), the read in `restricted` (15 minutes). Every write runs in `withTransaction` with its audit call and invalidates the requester's principal.

The emergency screen. Admin `/emergency` (`features/emergency/`; nav item and list need `audit.read`, which only `platform_owner` holds) lists the records with status (pending, active, expired, revoked; filterable), requester ("(you)" for the caller), approver, scope, purpose and the window ("N min from approval" while pending, a countdown while active). Request access takes the purpose (10 to 1000 characters), the scope kind and id and a duration of 15, 60, 120 or 240 minutes. Approve and Deny appear only on someone else's pending request and only with `access.emergency.approve`; on one's own request the row says it waits for another staff member, and the API refuses self-approval anyway (403 `self_approval_denied`, translated). Revoke appears on active rows. Open record appears only on the caller's own active access: the dialog states "This access is audited" before and after a read, takes the record type and id, reads through `withStepUp` and shows the conversation or the intake read-only; the content is validated against the professional schemas, kept out of the query cache and dropped when the dialog closes. Request, decision and read failures show translated copy (`self_approval_denied`, `emergency_not_active`, `conflict`, `not_found`, `forbidden`, `validation`, 429, `unknown`).



Admin `/emergency` for Nikola Đorđević: a pending request by another staff member (Approve, Deny) and his own active access (Open record, Revoke). Both rows were staged in the e2e database for the screenshot; their purposes say so.



The audited record view after reading `conv_sample` under the active access.

12. Audit log

`AuditWriter.record()` (`src/audit/audit-writer.ts`) inserts into `audit_events` with the category derived from the action (`audit-events.ts`: `auth.*` and `user.suspended` are SECURITY, `grant.*` and `access.denied` AUTHORIZATION, administration ADMIN, content CONTENT_ACCESS), the retention horizon from

`retention_policies` (10-minute cache, defaults per tier), a keyed hash of the IP (HMAC with `IP_HASH_SALT`, falling back to the session encryption key, so it cannot be reversed from a table of all addresses) and the parsed user-agent family. Details are sanitised: forbidden keys (tokens, codes, passwords, secrets, the SSO verifier, state and nonces, the pending-bind `handle` and every `challengeToken`, message bodies, intake text, notes, voucher codes, raw email) are dropped and strings truncated (`FORBIDDEN_KEYS`). The table is range-partitioned by month and protected by triggers that reject UPDATE and DELETE; the application role also lacks those privileges. `pino-redaction.ts` redacts the same fields from HTTP logs, plus the `Location` of every response and the `Referer` of every request, because a redirect or the sign-in page's own address can carry a pending-bind handle or an OAuth state. Add a new action to `AUDIT_ACTIONS` before using it; the type will not compile otherwise. Inside a `withOrgScope` or `withTransaction` callback, `record(event, { tx })` inserts through that transaction: the row commits with the change it describes, and a failed insert propagates and rolls the change back (`test/grants.e2e.spec.ts`, Review Focus 5). Without `tx` (denials, sign-in events, audits after a commit) the row is written on the pool and `record()` never throws in production; tests set `strictMode`. The retention lookup always uses the pool.

13. WebSockets

`NestSelfIoAdapter` (`src/ws/nestself-io.adapter.ts`, installed in `main.ts`) serves Socket.IO under `/api/ws`. The handshake middleware (`ws-auth.service.ts`) checks the `Origin` against the allowed surfaces, then authenticates either the web session cookie (same code path as HTTP) or a mobile ticket from `POST /auth/ws-ticket`: 30-second TTL, hashed, consumed atomically with `GETDEL` (`ws_ticket:{hash}`). The gateway (`events.gateway.ts`) exposes `subscribe { room, action, resource }` and `unsubscribe`; each subscription runs `authorize()` (`subscription-authorizer.ts`) and denied rooms emit `subscribe:denied`. `ws-invalidation.service.ts` listens on the `invalidate` channel and, for connected sockets of that user, reloads the principal, disconnects suspended or deactivated users and leaves rooms whose grants are gone (`subscription:revoked`). `ws-connection-registry.ts` tracks sockets per user.

14. Background jobs

Two kinds of job live in `src/jobs/`.

Jobs on behalf of a principal. `JobAuthorization` (`job-authorization.ts`) wraps BullMQ processors. A job payload carries `{ principalId, context, action, resource, data }` and never tokens or grants (AUTHZ-R-54). At execution the wrapper reloads the principal, rebuilds the context and calls `authorize()`; an unknown, suspended or denied principal completes the job as `skipped` (not failed, so no retry) with an `access.denied` audit event, while allowed jobs run the handler and audit with `actorKind: 'JOB'`. Feature modules that enqueue work (exports, imports) wrap their processors with it.

System maintenance jobs. `MaintenanceJobsService` (`maintenance-jobs.service.ts`) holds idempotent jobs that run in system context (no tenant setting) and audit with `actorKind: 'JOB'`, `actorUserId: null`:

Job	Schedule (local)	What it does
`audit.ensure-partitions`	01:00	Calls `audit_events_ensure_partitions(3)` so the next three months' partitions exist (D-21)
`grants.reconcile`	02:15	Closes grants whose `valid_until` passed (`grant.expired`), revokes delegations whose parent is revoked (`grant.delegation_revoked`), revokes organization-scoped grants whose subject has no ACTIVE membership (`grant.revoked`, reason `membership_ended`), then invalidates each affected principal and every delegatee below a closed grant (T-2.10, D-17). Each grant is closed in its own transaction together with its audit row; a grant that fails is rolled back, logged by id, counted in `failed` and retried on the next run, and the run goes on with the next one
`mobile.prune-stale-families`	02:30	Scans `user_mobile_families:*`; a family whose `mobile_family:{jti}` key has expired (30 days unseen) is denylisted in `revoked_family:{jti}` for 90 days, removed from the set and audited as `auth.mobile_family.revoked` (AUTHZ-R-64)
`sessions.reconcile-ttl`	02:45	Scans `session:*`; a record whose `absoluteExpiresAt` has passed (its key outlived its TTL) is destroyed through `SessionService.destroyOwnedByHash`, which revokes the refresh token, removes the hash from `user_sessions:{userId}` and publishes the invalidation. Malformed records are counted and left alone; a record whose destruction fails is counted in `failed` and retried on the next run (SESS-R-16)
`sso.certificate-expiry`	03:00	For every `ACTIVE` SSO connection (`checkSsoCertificates`): a SAML connection with a `metadataUrl` has its metadata fetched through the `MetadataFetcher` (section 22.11) and the expiry it carries stored when it changed; a SAML connection registered from metadata XML without a known expiry is looked up with `describeProvider` and the expiry stored. A certificate that expires within `SSO_CERTIFICATE_WARNING_DAYS` (30), or already has, is audited as `sso_connection.changed` (details `{ reason: 'certificate_expiring', expiresAt, connectionId, daysLeft, job: 'sso.certificate-expiry' }`, actor `JOB`, the connection's organization) in its own transaction and logged as a warning, every night until the certificate is replaced. Once that row has committed, `ops_certificate_alert` is mailed to `OPS_ALERT_EMAIL` (English, section 23) at most once per connection in `SSO_CERT_ALERT_TTL` (7 days, `sso_cert_alert:{connectionId}`, taken before the send and released when it fails); without `OPS_ALERT_EMAIL` the run audits and logs only and warns once that alert mail is off. A refresh that fails (fetch, provider lookup or write) or fetched metadata without a signing certificate is counted in `failed`, and the connection is still checked against its stored expiry; a failed audit transaction or alert is counted in `failed` too, and everything failed is retried the next night. Logs and audit details carry the provider name, dates and ids, never the metadata URL or document. Report `{ scanned, refreshed, expiring, alerted, failed }` (SSO-R-14, section 22.11)
`emergency.sweep-expired`	every 15 minutes	Sets `revoked_at` on approved emergency access whose `expires_at` has passed, audits `emergency_access.expired` and invalidates the requester, one row per transaction with its audit row; a row that fails is counted in `failed` and retried on the next run while the sweep goes on. The engine already denies an expired window, so the sweep only closes the record

`JobSchedulerService` (`job-scheduler.service.ts`) registers these as BullMQ job schedulers on the `nestself-maintenance` queue (its own ioredis connection to `REDIS_URL`, cron in `JOBS_TIMEZONE`, default `Europe/Belgrade`, three attempts with exponential backoff) and runs a worker with concurrency 1. It is started only by the worker process: `src/worker/main.ts` boots the application context without an HTTP listener and calls `start()`; the ECS `worker` service runs `node dist/worker/main.js` from the same image as the API, locally `pnpm --filter @nestself/api worker:dev`. Without `REDIS_URL` the worker exits with an error. The worker boots the same `AppModule` as the API, so it also needs the API's identity provider settings and session encryption key: `sessions.reconcile-ttl` revokes refresh tokens through `SessionService`. `pnpm --filter @nestself/api job:run <name>` builds with `nest build` and executes one job immediately through `src/worker/run-job.ts` (`node dist/worker/run-job.js <name>`), printing its report, which is how to verify a job after a deploy or catch up after downtime. `KeyValueStore.scan()` exists for these jobs only; request paths never call it.

Local job tooling goes through the Nest compiler. Nest resolves constructor dependencies from decorator metadata, which `tsx` (esbuild) does not emit, so `worker:dev` is `nest start --entryFile worker/main --watch` and `job:run` is `nest build && node dist/worker/run-job.js`; the runner moved from `scripts/` to `src/worker/run-job.ts` so the build compiles it. Under `tsx` both failed with `Nest can't resolve dependencies of the AuthorizationService (?, ...)` until 2026-10-24. The other `tsx` scripts (`db:migrate`, `db:seed`, `db:backfill-grants`) do not boot Nest and are unaffected.

Unit tests (`maintenance-jobs.service.test.ts`) use mocked repositories, a mocked identity provider and the in-memory store; `test/jobs.e2e.spec.ts` runs the reconciliation and partition jobs against the real database and checks the audit trail and the application role's permission to create partitions; `test/emergency.e2e.spec.ts` runs the emergency sweep.

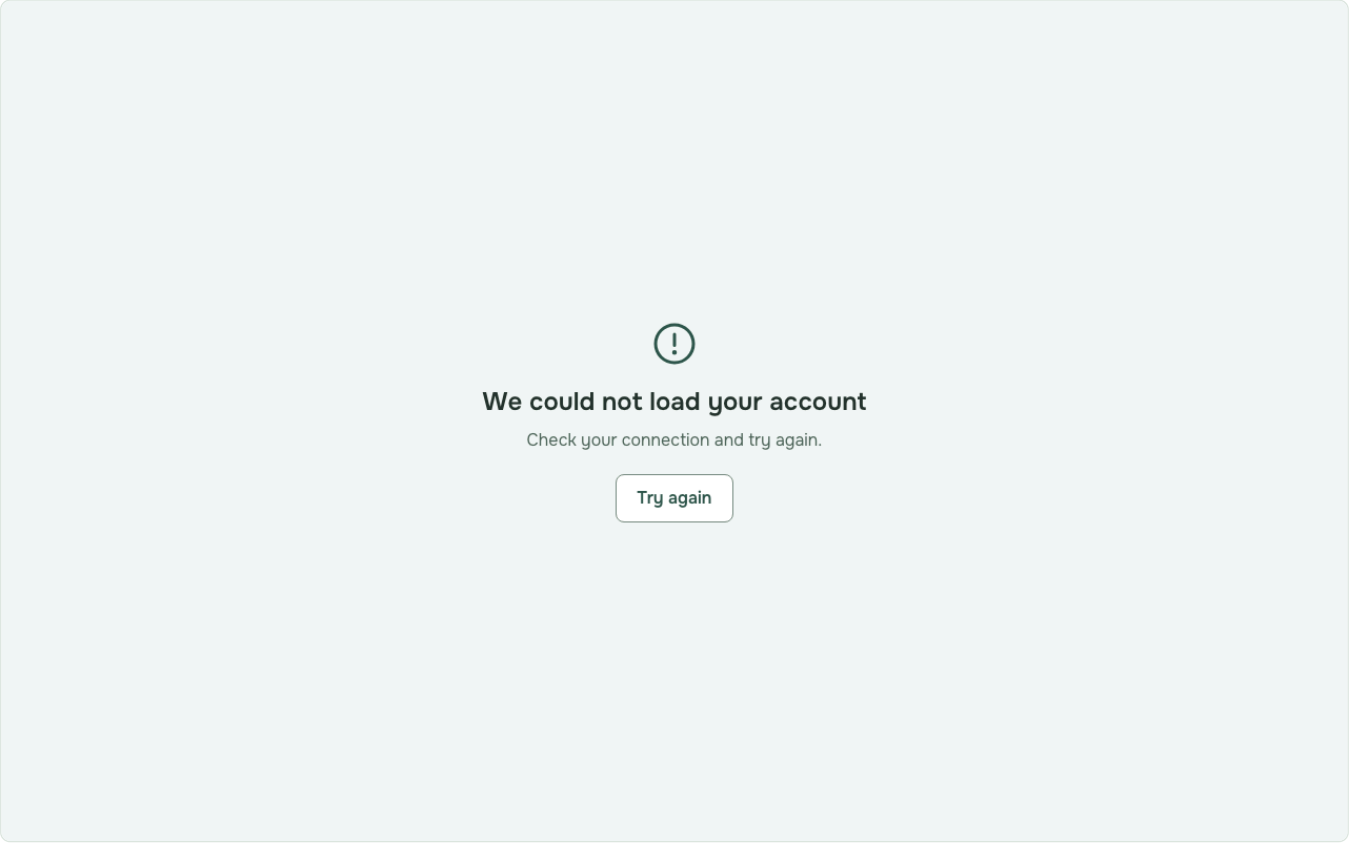
15. Web clients

`packages/api-client/src/client.ts` works in cookie mode (default): `credentials: 'include'` and the CSRF cookie copied into `X-CSRF-Token` on mutating requests. `auth.ts` exposes typed functions for every endpoint (`discover`, `emailOtpStart`, `emailOtpVerify`, `password`, `challenge`, `totpEnrol`, `me`, `switchContext`, `signOut`, `listSessions`, `stepUpStart`, `stepUpResend`, `stepUpVerify`, `ssoBindStart`, `ssoBindVerify`, `ssoBindResend`, `ssoBindToken`, `invitationAccept` ...) validated with the contracts schemas; `me` takes an optional `{ signal }`, and `invitationAccept` an optional locale.

`apps/workspace/src/auth/` and `apps/admin/src/auth/`: `AuthProvider` calls `GET /auth/me` on mount and window focus and exposes `status`, `principal`, `effectiveActions` (computed by the API, section 9) and `stepUp()` through `useAuth()`; a 401 flips to `unauthenticated`, and any other failure to `error` (below). `sample-auth-provider.tsx` serves sample mode with `SAMPLE_PRINCIPALS`. `screens/sign-in-page.tsx` drives the identifier-first flow with one screen per step (`identifier`, `otp`, `password`, `new-password`, `totp`, `totp-enrol`, and `screens/sso-bind-screen.tsx` for the confirm-email step of a pending SSO binding, section 22.4); `map-auth-error.ts` maps error codes to localized messages (`packages/i18n/locales/*/auth.json`), with `error.unknown` for codes it does not know. Routes under `routes/_authenticated/` redirect to `/sign-in` without a principal and gate sections with `can()` (for example `_hr.tsx` requires `program.overview`); `routes/_public/invite.$token.tsx` handles invitation links.

Route guards wait for the session. The guards (`_authenticated` in both apps; `_hr`, `_manager` and `hr/access` in the workspace) await `resolveRouteAuth(context.auth)` from `auth/route-auth.ts` before deciding. On a full page load the router context starts without a principal; in api mode it carries `ensure()`, which resolves once the first `/auth/me` has settled, and the provider reuses that same request. A deep link to a signed-in page therefore opens the page instead of bouncing to `/sign-in`; a 401 still redirects, with `search.return` set to the path.

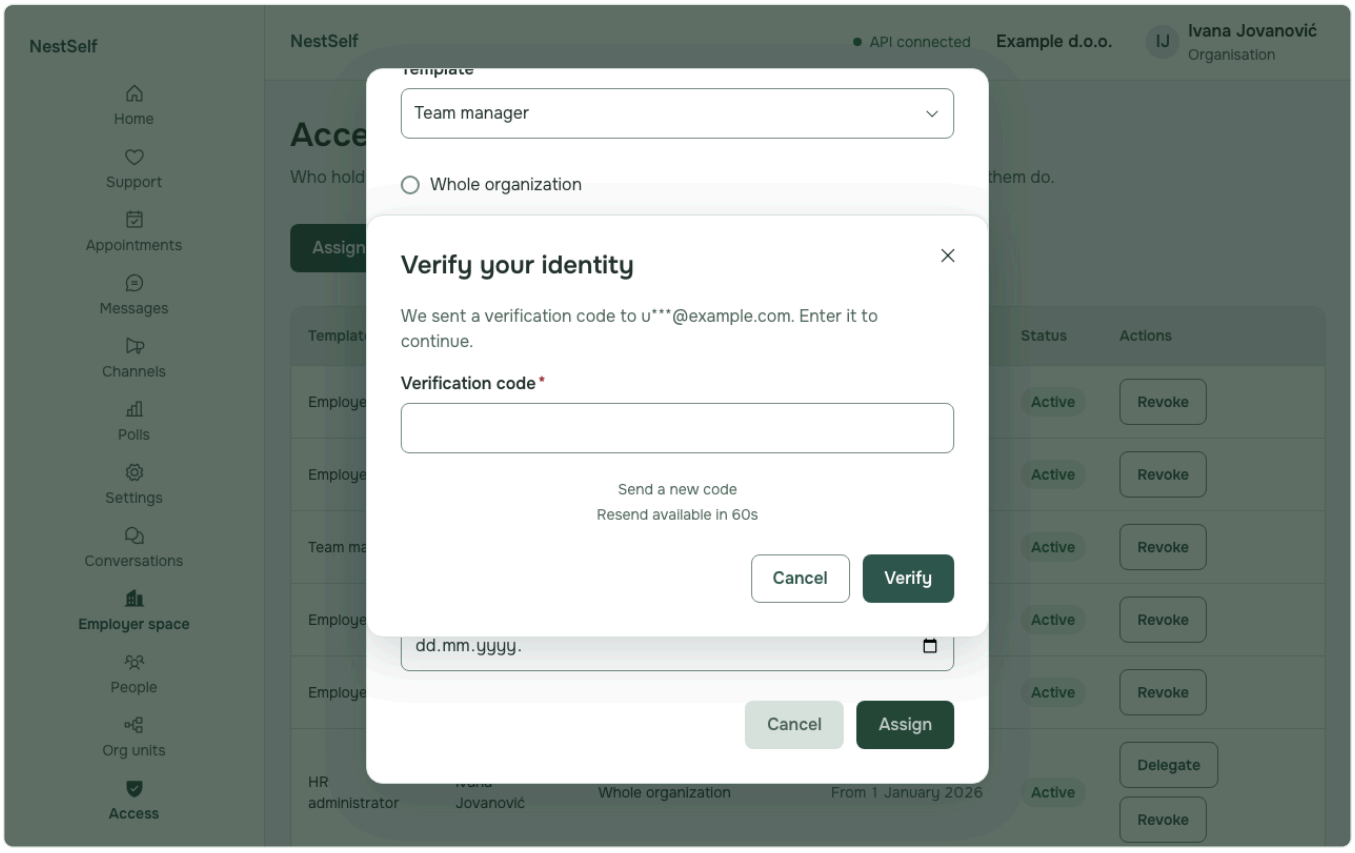
A session that cannot be loaded is not a sign-out (phase 4, both apps, byte-identical). `initialMe` sends the first `GET /auth/me` with `AbortSignal.timeout(ME_TIMEOUT_MS)` (10 s, `auth/route-auth.ts`), and the provider's later refreshes give up after the same time. `initialSnapshot` turns a 401 into the signed-out snapshot and every other failure (an error answer, no answer in time, no connection) into `SessionUnavailableError`, because the user may well be signed in. The provider then sets `status: 'error'` and keeps the principal it already had; the router sync in `routes/__root.tsx` treats `error` as no result (`settled` stays false), so the guards ask again instead of redirecting. The `_authenticated` route's `errorComponent`, `AuthenticatedErrorComponent` (`auth/session-unavailable.tsx`), renders a `StatusBlock` with "We could not load your account" and Try again (`common:state.sessionUnavailable`), which asks the provider for the session; the page then opens, or a 401 sends the user to `/sign-in`. Any other route error keeps the router's own error screen. The journey `workspace-sign-in.spec.ts` answers `/auth/me` with a 500 to prove it.



`/settings` while `GET /auth/me` answers 500 (stubbed in the browser for the screenshot): the session is unknown, so the page offers Try again instead of the sign-in page. The API's own message never reaches the screen.

Step-up in the browser. `stepUp()` opens `StepUpDialog` (`auth/step-up-dialog.tsx`, byte-identical in both apps), rendered once inside the provider; concurrent callers share one dialog and one promise. The dialog calls `stepUpStart()` when it opens and names the masked address from the answer

(`auth:stepUp.descriptionMasked`), takes the code in `OtpInput` with the email-code length bounds and calls `stepUpVerify()` . A wrong code keeps the challenge, since the API allows three tries; `invalid_challenge` , `expired_code` and `too_many_attempts` drop it, and "Send a new code" then starts a new one at once. "Send a new code" otherwise calls `stepUpResend()` and is disabled for 60 seconds after each code, with a countdown (`auth:otp.resendCountdown`), counted from `details.retryAfter` when the API refuses a resend as too soon. One request runs at a time (start, resend and verify never overlap), Cancel is disabled while a verify is in flight, and an answer that arrives after the dialog was closed changes nothing. Failures render in `FormErrorSummary` and on the field: `invalid_code` , an expired or spent challenge, 429 (the lockout copy, with the minutes from `retryAfter` when given), `step_up_unavailable` ("Verification is not available for your account. Ask your administrator."), `unavailable` and `network` with their own copy, `error.unknown` for anything else. `withStepUp(stepUp, fn)` (`auth/with-step-up.ts`) runs a mutation and, on 403 `step_up_required` , awaits `stepUp()` and retries once. When the user cancels it throws `StepUpCancelledError` ; callers treat that as "declined" (the access dialogs close without an error) and `handleApiError` ignores it. The sample providers resolve `stepUp()` to `true` .



The step-up dialog over the assign dialog: it names the masked address the code went to (`u***@example.com` for Ivana) and counts down to the next resend. For the screenshot the browser answered the first create with the API's own 403 `step_up_required` body, as the server does once the session is older than 30 minutes; the dialog then started a real challenge against the stub API.

Access screens. `createOrgAdminApi(client, { kind: 'org' | 'ops' })` in `@nestself/api-client` serves both apps; every mutation runs through `withStepUp` , and API codes map to translated copy under `access.errors.*` with `unknown` as the fallback.

- Workspace `/hr/access` (`features/access/` , route `routes/_authenticated/hr/access.tsx`): the nav item and the route need `access.preview` only; the route sits outside the `_hr` layout since phase 4, so an employer owner without an HR template reaches it (section 11.1). The screen switches to the organization context of the user's single active membership and shows the organization's grants. Every member choice (assign, delegate, transfer ownership, the effective-access preview) is a `MemberPicker` (`member-picker.tsx`): a "Search members" field whose trimmed text, debounced for 300 ms, goes to `GET /orgs/:orgId/members?search=` (the API matches the start of the name or the email and escapes the wildcards), above a `Select` of what came back; without a search the choices are the first 100 active members (`MEMBER_PAGE_SIZE` in `use-org-admin.ts`). The chosen member stays among the choices when a later search does not return them, the delegator or current owner is left out, Enter in the search field does not submit the dialog, and a failed search shows a translated alert. Assign (`access.grant`) offers organization templates at organization or org-unit scope with the required conditions; revoke (`access.revoke`) asks for a reason; delegate (`access.delegate`) hands a subset of one's own grant to another member; the effective-access panel previews any member. Transfer ownership appears only for the holder of the active owner grant and maps `owner_exists` , `subject_not_member` and `forbidden` to translated copy; after a transfer the owner-only controls disappear.
- Admin `/organizations/:orgId/access` (a tab next to Overview) and `/access` (platform grants): both need `access.preview` , switch to the operations context and use the ops API. The organization page assigns at organization scope only, because the ops API has no org-unit route; org-unit grants are HR's job in the workspace. It also transfers ownership through the staff route, which works when the organization has no owner yet. The platform page assigns `operations` templates at their own scope, revokes platform grants through `/ops/grants/:grantId/revoke` (the shared revoke dialog; a note says delegated grants go too) and shows user ids rather than names in api mode. Both web apps' query clients fail 4xx answers at once (`src/query-retry.ts`) and retry only 5xx, network and other errors, up to three times. Both pages offer the user-access lookup (`/ops/users/:userId/access`). The admin app's member choices (the organization Access tab, the owner transfer on the Overview) still read the first 100 active members and have no search (`MEMBER_PAGE_SIZE` in `features/access/use-ops-admin.ts` ; section 21).
- In sample mode the screens read `SAMPLE_GRANTS` and disable every mutation with a note.
- The grant table recomputes each row's status on every render, so a grant created a moment ago shows Active, not Scheduled.

The dialog, the helpers, `route-auth.ts` , `session-unavailable.tsx` , `sso-bind-screen.tsx` and the access screens exist once per app, because a shared package would need an install.

Popups inside dialogs. In `@nestself/ui`, Select and Menu popups opened inside a Dialog, AlertDialog or Sheet layer above it: the dialog's portal element sets `--z-popup` to one above `--z-dialog` (`src/lib/layers.ts`). Before 2026-10-24 the popup sat below the dialog and a pointer could not choose an option.

16. Mobile client

`apps/mobile/src/auth/`: `cognito-public-client.ts` talks to Cognito directly (`InitiateAuth` with `USER_AUTH` for email codes, refresh, revoke), `sso.ts` prepares the PKCE authorization-code flow through `expo-auth-session`, `token-store.ts` keeps tokens in `expo-secure-store` (device-only, unlocked access), `jwt.ts` decodes claims, `auth-state.ts` is the reducer and `auth-provider.tsx` orchestrates sign-in, refresh, sign-out and the sample mode. API calls use the bearer mode of the shared client; WebSocket connections fetch a ticket first. Mobile settings come from `EXPO_PUBLIC_COGNITO_*`. The email-code start sends the device locale as `ClientMetadata.locale`. There is no step-up screen yet: a `step_up_required` answer has no UI on mobile, although the API side accepts bearer callers (section 9).

Federated sign-in (phase 4). After the hosted-UI exchange, `completeSsoSignIn` in `auth-provider.tsx` runs `finishSsoSignIn` (`sso.ts`): it presents the ID token to `POST /auth/sso/bind-token` (section 8) before anything else, and only a bound sign-in stores its tokens and loads `/auth/me`. A refused binding, or one that cannot reach the API, keeps no tokens, clears any left over and rethrows, so the app is not signed in and the sign-in screen shows why: `signInErrorKey` (`sign-in-error.ts`) maps `sso_denied` to `auth:error.ssoDenied`, `sso_email_unverified` to `auth:error.ssoEmailUnverifiedMobile` ("Sign in once on the web to confirm your email address, then sign in here again."), `rate_limited` to `auth:error.ssoRateLimited`, `unavailable` and network failures to their own copy, and anything else to `error.unknown`. `finishSsoSignIn` and `signInErrorKey` have unit tests; no screen-level test proves the alert on the sign-in screen, because the repository has no React Native renderer and installs are barred while this work runs in parallel.

17. Infrastructure and the Cognito rollout

`infra/cdk/lib/auth-stack.ts` defines the user pool `nestself-<stage>` (Essentials plan, email OTP and password first factors, optional TOTP, no self sign-up), three app clients (confidential web with secret, public mobile with PKCE, reserved machine-to-machine), the `CustomMessage` and `PreSignUp` Lambda triggers (`lib/triggers/`), the web client secret and the session encryption key in Secrets Manager, WAF and SES for staging and production, and for those two stages the custom domain of the hosted UI, `auth.<stage domain>` (section 17.2). Outputs: `UserPoolId`, `WebClientId`, `MobileClientId`, `CognitoDomain`, `CognitoIssuer`, secret ARNs.

The CDK app contains only this stack unless `-c platform=true` is passed, which keeps the rollout to Cognito-only until the web applications ship (`infra/cdk/README.md`, "Rollout order"). To connect the API to a real pool:

```
pnpm --filter @nestself/cdk exec cdk bootstrap aws://<ACCOUNT_ID>/eu-central-1
pnpm --filter @nestself/cdk exec cdk deploy NestSelf-dev-Auth -c stage=dev -c account=<ACCOUNT_ID>
node infra/cdk/scripts/env-from-outputs.mjs dev >> apps/api/.env     # COGNITO_* and the secret
```

The step-by-step account setup, credentials, IAM permissions and every environment variable are in `CDK-setup.md` at the repository root. Then set `AUTH_MODE=cognito`, `REDIS_URL` and `SESSION_ENCRYPTION_KEY` (from the stack secret) in `apps/api/.env`. Users must exist in the pool: create them through an invitation (the accept flow calls `AdminCreateUser`) or with the AWS console for a first test; the API links them by `sub` on first sign-in only when an `external_identities` row exists, so onboarding goes through invitations. `env-from-outputs.mjs <stage> --mobile` prints the `EXPO_PUBLIC_*` values for the app (`EXPO_PUBLIC_COGNITO_CLIENT_ID` is the name `apps/mobile/src/auth/config.ts` reads).

The API process needs AWS credentials for the Cognito admin calls (`AdminInitiateAuth`, `AdminRespondToAuthChallenge`, `AdminCreateUser`, `AdminDeleteUser`, `AdminDisableUser`, `AdminEnableUser`, `AdminGetUser`, `AdminSetUserPassword`, `AdminUserGlobalSignOut`, `GetTokensFromRefreshToken`, `RevokeToken`, `ListUsers`, `DescribeUserPoolClient`; for enterprise SSO also `UpdateUserPoolClient` and `Create` / `Update` / `Delete` / `Describe` / `ListIdentityProviders`, section 22.6) and, when `SES_FROM_EMAIL` is set, `ses:SendEmail`. In AWS these come from the ECS task role (`infra/cdk/lib/constructs/ecs-task-roles.ts`); locally from the default credential chain (`AWS_PROFILE`). The compute stack's `worker` service runs `node dist/worker/main.js` (section 14) with the worker task role.

17.1 The dev pool as deployed

First deployed 2026-10-24 into a dedicated account. Current coordinates:

Account / region	`634972095709` / `eu-central-1`
Stacks	`CDKToolkit`, `NestSelf-dev-Auth`
User pool	`eu-central-1_g6OGYONQ0`, hosted domain `nestself-dev`
App clients	`nestself-web-dev`, `nestself-mobile-dev`, `nestself-m2m-dev`
Secrets	`nestself-dev/cognito/web-client-secret`, `nestself-dev/api/session-encryption-key`
Triggers	`nestself-dev-cognito-custom-message`, `nestself-dev-cognito-pre-sign-up`

Three things about this stack are easy to get wrong.

The username is the email address. `auth-stack.ts` sets `signInAliases: { email: true }` without `username: true`, which CDK compiles to `UsernameAttributes: ['email']`, not `AliasAttributes: ['email']`. `AdminCreateUser` therefore **requires** an email as `--username` and rejects a plain handle with `Username should be an email`. Cognito then generates the internal username itself, and it is the same UUID as `sub`, so a manually linked `external_identities` row gets that one value in both `subject` and `cognito_username`:

```
aws cognito-idp admin-create-user --user-pool-id eu-central-1_g6OGYONQ0   --username <address> --user-attributes Name=email,Value=<address>
Name=email_verified,Value=true   --message-action SUPPRESS
```

`--message-action SUPPRESS` keeps AWS from sending anything at creation time; the first mail is the sign-in code. The `PreSignUp` trigger refuses self sign-up but does not block `AdminCreateUser`.

`USER_ID_FOR_SRP` is absent in the EMAIL_OTP challenge. `AdminInitiateAuth` with `PREFERRED_CHALLENGE: EMAIL_OTP` returns only `CODE_DELIVERY_DELIVERY_MEDIUM` and `CODE_DELIVERY_DESTINATION` in `ChallengeParameters`. `startEmailOtp` reads `ChallengeParameters['USER_ID_FOR_SRP'] ?? input.email`, so the fallback always wins and the email is the username in both the initiate and the respond call. That keeps `SECRET_HASH` consistent across the two calls, which is what the confidential web client requires.

The email code is eight digits, not six. See section 6. Anything that validates or caps the length must use `EMAIL_OTP_MIN_LENGTH` / `EMAIL_OTP_MAX_LENGTH`; a six-digit `maxLength` silently truncates and produces `CodeMismatchException`.

`AWS_PROFILE` in `apps/api/.env` needs the ConfigModule merge. `envSchema` is a plain `z.object`, which strips undeclared keys, and ConfigModule's `validate:` branch assigns only what the function returns back onto `process.env` (its `validationSchema:` branch merges them back, `validate:` does not). Before 2026-10-24 that silently dropped `AWS_PROFILE`, the SDK fell back to the machine's default profile, and the Cognito calls failed with a cross-account `AccessDenied` that still named the correct pool id. `validateEnv` now returns `{ ...config, ...appEnv }`, which also preserves `AWS_ACCESS_KEY_ID`, `AWS_SESSION_TOKEN`, `HTTPS_PROXY` and `NODE_EXTRA_CA_CERTS`.

Tearing the pool down takes three calls in order, because the stack enables deletion protection and Cognito refuses to drop a pool that still has a domain:

```
aws cognito-idp update-user-pool --user-pool-id <id> --deletion-protection INACTIVE
aws cognito-idp delete-user-pool-domain --user-pool-id <id> --domain <prefix>
aws cognito-idp delete-user-pool --user-pool-id <id>
```

17.2 Staging and production: the auth domain, the API environment and the load balancer

Phase 4 prepared staging and production in CDK; nothing was deployed (design 1, "a deployment" is an anti-goal). The values that belong to a deployment are CDK context, and a stage that lacks one does not synthesize: the error names the key. `CDK-setup.md` section 8 is the step-by-step, `infra/cdk/README.md` the stack reference.

- **Cognito custom domain.** `StageConfig.authDomainName` (`infra/cdk/lib/config.ts`: `auth.staging.nestself.rs`, `auth.nestself.rs`; undefined in dev, which keeps the prefix domain `nestself-dev`) replaces the old `enableCustomDomain` flag. With it, `auth-stack.ts` adds the user pool domain with the ACM certificate from context `authCertificateArn` (issued by hand in **us-east-1**, because Cognito serves the domain through CloudFront), an A and an AAAA alias through `UserPoolDomainTarget` in the hosted zone named by context `hostedZoneId`, and the `CognitoDomain` output `https://auth.<stage domain>`, which `env-from-outputs.mjs` and the compute stack read as `COGNITO_DOMAIN`. A missing value fails the synth with "auth custom domain needs context authCertificateArn and hostedZoneId; see CDK-setup.md". `infra/cdk/cdk.json` sets `@aws-cdk/aws-route53-targets:userPoolDomainNameMethodWithoutCustomResource`, so the alias reads the CloudFront distribution attribute instead of adding a Lambda-backed custom resource. Cognito refuses a custom domain whose parent name does not resolve to an A record (`CDK-setup.md` 8.3).
- **The domain is final before the first employer.** The SAML ACS URL (`<COGNITO_DOMAIN>/saml2/idpresponse`) and the OIDC redirect URI an employer registers at its IdP derive from it, so changing the domain later means every employer reconfigures its provider (`CDK-setup.md` 8.7, section 22.13).
- **API environment per stage** (`sharedEnv` of `compute-stack.ts`, the api, worker and migrate tasks): `NODE_ENV=production` on every stage, staging included (no Swagger, JSON logs, no `devToken`), `TRUST_PROXY_HOPS` from `trustProxyHops` (2: CloudFront, then the load balancer), `SES_FROM_EMAIL` from the stage's `sesFromEmail` (`no-reply@staging.nestself.rs`, `no-reply@nestself.rs`) and `OPS_ALERT_EMAIL` from context `opsAlertEmail` when it is passed (validated by the API's environment schema at start, not at synth). The marketing container runs with `NODE_ENV=production` on every stage too.
- **The load balancer admits CloudFront only.** With `trustProxyHops` above 0 the ALB security group admits TCP 80 from the CloudFront origin-facing managed prefix list (context `cloudfrontOriginPrefixListId`, required then) and nothing else; the listener is created with `open: false`, because its default would add `0.0.0.0/0` next to that rule, and the cdk-nag acknowledgment of an open ALB (`AwsSolutions-EC23`) is gone. Only port 80: CloudFront reaches the origin over HTTP and the managed list counts as 55 of the default 60 rules per security group. This is what makes `TRUST_PROXY_HOPS=2` safe: the client address the throttles count (section 4) is read two entries from the right of `X-Forwarded-For`, which a caller could forge by reaching the ALB directly.
- **Residual risk and checks.** The prefix list admits every CloudFront distribution, not only ours: any AWS account can point a distribution at the ALB and reach the API without passing our edge WAF, and whether such a distribution's `X-Forwarded-For` would carry the client address at the depth the API trusts is not confirmed. A secret origin header that our distributions add and a listener rule checks would close both; it is a follow-up (section 21). Nothing routes the apex to the ALB yet (the edge stack has no marketing distribution), so the marketing site has no public path until that distribution exists (section 21). After the first real deploy the owner confirms that the ALB does not answer without CloudFront (`CDK-setup.md` 8.6 step 2). The dev auth stack is unchanged by phase 4 (its prefix-domain branch is untouched); the owner confirms with `cdk diff NestSelf-dev-Auth -c stage=dev -c account=$AWS_ACCOUNT_DEV`, expecting no differences, before the next dev deploy.
- **CI.** The `cdk-nag-gate` job synthesizes staging offline and passes fixture values for `authCertificateArn`, `hostedZoneId` and `cloudfrontOriginPrefixListId` (`.github/workflows/ci.yml`); the real values come from `CDK-setup.md` section 8.

18. Configuration reference

Validated in `apps/api/src/config/env.ts`; documented in `docs/environment.md`.

Variable	Purpose
`NODE_ENV`	`development`, `production` or `test`. Every AWS stage runs `production` since phase 4, staging included (`compute-stack.ts`): no Swagger, JSON logs, no `devToken` in invitation responses. `test` always selects the logging mailer (section 23)
`AUTH_MODE`	`stub` (default; refused in production) or `cognito`
`DATABASE_URL`, `DATABASE_POOL_MAX`	PostgreSQL connection for the API (a `nestself_app` member in Docker and AWS)
`REDIS_URL`	Required in cognito mode; optional in stub mode (in-memory store)
`JWT_SECRET`, `JWT_EXPIRES_IN`	Stub-mode bearer tokens only; removed after cutover

Variable	Purpose
`COGNITO_REGION`, `COGNITO_USER_POOL_ID`, `COGNITO_WEB_CLIENT_ID`, `COGNITO_WEB_CLIENT_SECRET`, `COGNITO_MOBILE_CLIENT_ID`, `COGNITO_DOMAIN`, `COGNITO_ISSUER`	Pool coordinates; `env-from-outputs.mjs` prints them
`SESSION_ENCRYPTION_KEY`, `SESSION_ENCRYPTION_KEY_PREVIOUS`	32-byte hex keys for token encryption at rest; derived from `JWT_SECRET` in stub mode
`IP_HASH_SALT`	Key for the audit and session IP hashes; defaults to the session encryption key
`SESSION_COOKIE_NAME`, `CSRF_COOKIE_NAME`, `SESSION_COOKIE_INSECURE_DEV`	Cookie names and the localhost switch (it also drops `Secure` and the `__Secure-` prefix from the SSO browser-binding cookie, `nestself_sso`)
`SESSION_LIFETIME_OVERRIDES`	JSON overrides per `population:context`
`MARKETING_URL`, `WORKSPACE_URL`, `ADMIN_URL`, `API_PUBLIC_URL`	Allowed origins for CORS, CSRF and WebSocket handshakes; invitation links. `WORKSPACE_URL` and `ADMIN_URL` are also where the SSO callback sends the browser. **`WORKSPACE_URL`, `ADMIN_URL` and `API_PUBLIC_URL` are required with `AUTH_MODE=cognito`** (the API refuses to start without them); in stub mode they stay optional and fall back to `http://localhost:5173`, `:5174` and `:4000`
`API_PUBLIC_URL` (SSO)	Public origin of the API for mobile and direct callers. **Browser SSO URLs do not use it**: the API origin for a surface is that surface's own origin (`apiOriginForSurface` in `src/auth/sso-start-url.ts`), because every SPA proxies `/api` to the API (CloudFront `/api/*` behaviours in AWS, the Vite proxy locally) and the host-only session cookie must be set on the host whose `/api/auth/me` reads it. So discovery's `ssoStartUrl`, invitation acceptance, the `redirect_uri` sent to the IdP (`<WORKSPACE_URL or ADMIN_URL origin>/api/auth/sso/callback`), the test link (a workspace flow) and the fake authorize hop all sit on `WORKSPACE_URL` or `ADMIN_URL`; `API_PUBLIC_URL` is the fallback only when that surface URL is unset (stub mode). It is still required in cognito mode (above), and its callback stays in the registered set the fake authorize hop accepts
`COGNITO_MOBILE_CLIENT_ID`, `COGNITO_DOMAIN` (SSO)	Required in cognito mode: provider registration updates the mobile client too, and the mobile client id is the audience `POST /auth/sso/bind-token` requires of an ID token (section 8); the token endpoint and the ACS URL in the admin detail panel come from the domain (with `COGNITO_USER_POOL_ID` for the entity ID), `https://auth.<stage domain>` on staging and production (section 17.2)
`TRUST_PROXY_HOPS`	How many proxies stand in front of the API: `main.ts` sets Express's `trust proxy` to it when it is above 0, so `req.ip` (the address every throttle counts, section 4) is taken from `X-Forwarded-For` that many entries from the right. 0 on a laptop (default); 2 in AWS (CloudFront, then the load balancer), set by the compute stack from `trustProxyHops`. Safe only while the load balancer admits CloudFront alone (section 17.2)
`SES_FROM_EMAIL`	The verified SES sender. Set, and with `NODE_ENV` other than `test`, every mail goes out through SES (`SesMailer`, section 23): invitations, the codes NestSelf issues, the ops alert. Unset, the logging mailer sends nothing. The compute stack sets the stage's sender (`sesFromEmail`)
`OPS_ALERT_EMAIL`	The operations mailbox for alerts, today the SSO certificate-expiry notice of the nightly job (section 14). Unset, the job audits and logs only. In AWS it comes from `-c opsAlertEmail` (section 17.2)
`JOBS_TIMEZONE`	Worker process: IANA timezone of the nightly job schedules (default `Europe/Belgrade`)
`FAKE_IDP_FIXED_OTP`	Stub mode: an always-accepted six-digit code; while `AUTH_MODE` is not `cognito` it also fixes the codes NestSelf issues (federated step-up, the confirm-email step), so Playwright types the same code everywhere
`FAKE_DNS_TXT_AUTOVERIFY`	Stub mode, `true` / `false`: the fake DNS resolver answers every pending SSO domain with its expected TXT value (the Playwright stack sets it); ignored with `AUTH_MODE=cognito`, which always queries the authoritative name server
`TEST_DATABASE_URL`	Integration tests; suites create `<name>_schema`, `<name>_auth`, ... databases
`AWS_PROFILE`, `AWS_ACCESS_KEY_ID`, `AWS_SESSION_TOKEN`, `HTTPS_PROXY`, `NODE_EXTRA_CA_CERTS`	Not declared in `envSchema`, but passed through to `process.env` for the AWS SDK and the HTTP stack; see the merge in `validateEnv` and section 17.1

Anything not declared in `envSchema` is still forwarded to `process.env`, because `validateEnv` returns `{ ...config, ...appEnv }`. Removing that spread re-breaks every library that reads `process.env` directly instead of through `ConfigService`.

Redis keys and TTLs are listed in `apps/api/src/redis/redis-keys.ts`. Phase 4 added `otp:{purpose}:{sha256(token)}` and `otp_resend:{purpose}:{hash}` (section 6), `sso_pending_bind:{sha256(handle)}` and `sso_bind_start:{sha256(handle)}` (22.4), `bearer_mismatch_audited:{connectionId}:{sha256(sub)}` (section 8), `sso_cert_alert:{connectionId}` (section 14), `throttle:step_up:{userId}` (section 9) and, with `REDIS_URL`, the throttler's counters `throttle:{throttler name}:{key}` (the key the throttler derives from the route and the tracker) and their `:block` keys (section 4).

19. Testing

- Unit tests sit next to the code (`*.test.ts`, `*.spec.ts`) and mock repositories and services. The authorization engine has unit, property and golden-table tests; `step-up-freshness.test.ts`, `grant-rules.test.ts` and `effective-access.test.ts` cover the phase 2 rules.
- Integration suites in `apps/api/test/` boot the real application against a disposable PostgreSQL (`TEST_DATABASE_URL`, one database per suite via `test/db/test-database.ts`), use the fake identity provider and the in-memory store (the test environment ignores `.env` and `REDIS_URL`). Without `TEST_DATABASE_URL` they skip; locally use `TEST_DATABASE_URL=postgresql://nestself:nestself_dev@localhost:54329/nestself_test`.
- **Per-worktree test database names.** Each suite's database is `<base>_<prefix>_<suffix>`: `<base>` is the database in `TEST_DATABASE_URL`, `<suffix>` names the suite, and `<prefix>` is `TEST_DATABASE_PREFIX` or, when that is unset, the first eight hex characters of the SHA-1 of the repository root path (`apps/api/test/db/test-database-name.ts`). Git worktrees of the repository, where several agents often work at once, therefore get separate databases on one server and never drop each other's. Names over PostgreSQL's 63-byte limit shorten the base; old databases are not removed automatically.

Suite	Covers
`auth.e2e.spec.ts`	Sign-in, CSRF, context switch, profile, sessions, lockout, the three-attempt email code, step-up start, verify and the resend cooldown, `effectiveActions` in `/auth/me`
`ops-clients.e2e.spec.ts`	`/ops/orgs/:orgId/members` and `/invitations` through the CLIENT assignment
`grants.e2e.spec.ts`	Organization grant administration, delegation bounds and depth, delegatee access and its revocation, approval authorities, the write throttle; the API runs as the `nestself_app` role, and an audit insert refused inside a grant creation answers 500 and leaves no grant (Review Focus 5)
`ops-grants.e2e.spec.ts`	Platform grants, ops organization grants, user access
`professional.e2e.spec.ts`	Confidential reads by participant and assigned expert, denials answered 404 like a missing id and their audit, step-up on a stale session
`emergency.e2e.spec.ts`	Request, self-approval refusal, approval, record read and its order (own access, active, record, scope), expiry before the sweep, sweep, deny, revoke
`members.e2e.spec.ts`, `invitations.e2e.spec.ts`, `org-admin.e2e.spec.ts`, `ws.e2e.spec.ts`	Their areas; members also proves delegatee invalidation on membership end and the member and org-unit lists for `access.preview` (an employer owner without HR rights, a plain employee refused); invitations also proves the import buckets (one per row, only the created rows mailed) and the locale of the accept's code email
`health.e2e.spec.ts`, `jobs.e2e.spec.ts`	Health with database and Redis; the maintenance jobs, among them the certificate job reading a metadata-URL connection from its metadata and mailing the ops alert once in seven days (the sweepers' per-row failures are unit-tested in `maintenance-jobs.service.test.ts`)
`sso.e2e.spec.ts`	SSO connection lifecycle, domains and their verification, provider-name rules, `idpInitiatedAllowed`, the OIDC secret and the verification token kept out of responses and audit rows, the claim switch, `http:` and non-public metadata URLs refused, `SUSPENDED → TESTING` on a provider change, the certificate expiry fetched at test, non-staff 403 (section 22.12)
`sso-auth.e2e.spec.ts`	Home-realm discovery and its timing, `/auth/sso/start` and `returnPath`, every callback path, the verified-domain rule, login CSRF with two agents, single-use test links, the binding connection, federated step-up, the pending binding end to end, `bind-token` and the federated bearer, the throttles and the redirect on 429
`db/schema.spec.ts`	Eleven migrations, partitions, append-only triggers, RLS on all eighteen tenant tables (`sso_domains` since `0006`, `cases` since `0007`), the status check, the composite case keys, `0007` and `0008` over existing rows, the seed's stale-grant cleanup
`db/memberships-repository.spec.ts`	The member search against the real `ILIKE`: a wildcard in the term matches only itself, a plain term a case-insensitive prefix of the name or the email
`db/sso-repository.spec.ts`	`SsoRepository` as the application role: uniqueness, active-for-domain, the job listings, tenant-scoped writes refused across organizations
`db/authorization-repository.spec.ts`	Grant repository methods, delegation walks and emergency access queries under RLS
`db/backfill-grants.spec.ts`	The legacy role backfill and the `0005` guard

- Browser journeys live in `e2e/auth/` and run with `pnpm e2e:auth` (`playwright.auth.config.ts`, project `auth-desktop`): fourteen tests. `workspace-sign-in.spec.ts` (email-code sign-in, sign-out through the account menu, guard redirect, wrong code, and a `/auth/me` answered with a 500 that shows "We could not load your account" until Try again opens the page), `admin-sign-in.spec.ts` (password and authenticator), `hr-access.spec.ts` (Ivana's grant table, the member search by name and by email prefix, preview, delegation, assign and revoke; and an employer owner without HR rights: Ivana hands the ownership to Stefan Marković, who opens `/hr/access` with the owner template alone, then hands it back), `account.spec.ts` (Ivana's display name by deep link, change and header, sessions list with this browser marked, sign out everywhere ending a second browser too), `hr-people.spec.ts` (invite and revoke a fictional address, move Ana Nikolić to Sales and back, add and rename an org unit, open and cancel End membership), `admin-ops.spec.ts` (organizations list, the sample organization's People tab, an emergency request by Nikola with Approve and Deny hidden on his own row, approval by Milica in a second browser, the audited record view reading `conv_sample`, the user lookup offering Suspend for `user_jovan`, cancelled) `expert-cases.spec.ts` (Milica opens `case_sample` from `/cases` and reads its messages without a step-up) and `sso-sign-in.spec.ts` (section 22.12: Nikola sets up `org-acme-saml` for `acme.example` through the API, an invited tester completes the single-use test sign-in through the fake identity provider, Nikola activates, an invited employee signs in from the workspace sign-in screen, and a federated HR administrator ends a membership after a step-up with the code NestSelf mails; a second test runs an OIDC connection whose provider does not confirm the address, where the staff test and an employee both finish with the confirm-email step). A half-failed run of the owner journey leaves Stefan as the owner until the next reseed, which every run does. `admin-ops.spec.ts` stages `platform_owner` grants for Nikola and Milica (`grant_user_nikola_platform_owner_e2e`, `grant_user_milica_platform_owner_e2e`) because the seed has no platform owner and emergency approval needs a second one; the fake identity provider lives in the API process, so a new staff account cannot be created from the browser test. `e2e/auth/support.ts` holds the sign-in helpers, the per-user authenticator secrets in `e2e/.auth-state/` and the staging helper, which also drops the user's cached principal from Redis (Redis outlives the API process). They need the API in stub mode with `FAKE_IDP_FIXED_OTP=123456` and `FAKE_DNS_TXT_AUTOVERIFY=true` and the web apps in `api` data mode; the config starts all three. It also sets fictional `COGNITO_USER_POOL_ID` and `COGNITO_DOMAIN` values for the stub API, which stub mode reads only for the SSO service-provider values, so the admin SSO detail never shows the dev pool from `apps/api/.env`. See `e2e/README.md`, which also explains why a cognito-mode API on port 4000 must be stopped first.
- **The e2e database.** The stub API never uses the developer database in `apps/api/.env`. It runs against `nestself_e2e_<prefix>` on the `TEST_DATABASE_URL` server (same prefix rule as above), or exactly `E2E_DATABASE_URL`, which must name a `nestself_e2e_*` or `nestself_test_*` database. `e2e/global-setup.ts` drops it (`WITH (FORCE)`), recreates it, migrates it and runs the sample seed on every run through the API's own scripts (`scripts/reset-database.ts`, `migrate.ts`, `seed.ts`) with `DATABASE_URL` overridden; the API web server command first runs the same file with `--ensure`, because Playwright starts web servers before global setup and the API pings its database on boot. `reset-database.ts` refuses any other database name. Without `TEST_DATABASE_URL` or `E2E_DATABASE_URL` the run stops with a message before a server starts.
- **Guide screenshots.** `pnpm docs:auth:screenshots` runs the `screenshots` project of the same config (`e2e/screenshots/*.shot.ts`, 1280×800, light, device scale 1, reduced motion) against the same stack and writes the PNGs in `docs/auth/screenshots/`; `pnpm docs:auth:pdf` rebuilds both PDFs (`docs/auth/pdf/build.mjs` resolves relative images against the Markdown file and fails on a missing one). One capture is taller: `admin-sso-create-connection.png` is 1280×1500, because the whole OIDC form of the create dialog does not fit in 800 pixels (`sso.shot.ts` sets that viewport for the one capture). The script cannot take a file (an appended path lands among the values of its trailing `--project`), so run one file with `pnpm exec playwright test -c

`playwright.auth.config.ts --project=screenshots e2e/screenshots/<file>`. Five files: `admin.shot.ts`, `operations.shot.ts`, `sso.shot.ts`, `workspace-screens.shot.ts` and `workspace.shot.ts` (they run in name order, so `admin.shot.ts` enrols Nikola first). Nothing is created through the screens except the import in `workspace-screens.shot.ts`, which creates two invitations for fictional `new.colleague.*@example.com` addresses in the disposable database (its other two rows are skipped and not imported, so the result shows all three buckets); every other dialog is cancelled or only read. Before any capture every shot file removes the real developer account `user_dragan` from the disposable database (section 21), and `shot()` refuses a capture that shows it. No capture shows a token, code, nonce or secret: `maskText` (`e2e/screenshots/shots.ts`) replaces the authenticator key on the enrolment screen (whose QR code is blurred too), the pending domain's verification token in the SSO detail and the test link's single-use nonce before the capture, and the captions say so; the masked addresses of the step-up dialog and the confirm-email step are what the screens show. What is staged, and the captions say so: a `platform_owner` grant for Nikola (`admin.shot.ts`, `operations.shot.ts` and `sso.shot.ts`), because the sample data has no platform owner and the admin pages need it; two emergency access rows in `operations.shot.ts` whose purposes start with "Staged for the guide screenshots" (one pending request by Milica, one approved access of Nikola's for `case_sample`), because an approval needs a second platform owner; in `workspace.shot.ts` one create answered in the browser with the API's `step_up_required` body to open the step-up dialog; in `workspace-screens.shot.ts` a `/auth/me` answered with a 500 in the browser for the session-unavailable state; and in `sso.shot.ts` invitations for `it.tester@acme.example`, `it.tester@acmeid.example` and `maria@acmeid.example`, the SAML connection `org-acme-saml` (from the test-only metadata fixture, through a real test sign-in against the fake identity provider), its domains `acme.example` (verified) and `eu.acme.example` (pending), the OIDC connection `org-acmeid-oidc` (a draft for the list, then tested from the page and activated), Maria's sign-in with `email_verified=false`, which stops at the confirm-email step, and a discovery answer whose start URL the browser replaced so the redirect line stays on screen. Every certificate date in stub mode is the fixture's, 29 November 2027 (`SIGNING_CERT_B64` in `apps/api/src/identity/saml-metadata.fixture.ts`; the fake metadata fetcher serves the same document for any metadata URL). The next reset removes everything. The enrolment screenshot needs a fresh API process, so stop any API on port 4000 first.

- **Visual baselines.** The non-gated suite (`pnpm e2e`, `playwright.config.ts`, `e2e/__screenshots__/`) runs the apps in sample mode. Every authenticated page has the account menu and the context picker in its header since the UI-completeness plan. On 2026-10-25 the workspace and admin entries were checked: below 480px the header now hides the API status, so it fits 320px (it overflowed by 67px) and the mobile baselines did not move; only the four settings baselines (new account, sessions and authenticator sections) were updated and the missing sign-in baselines added. The same run also fixed sample-mode deep links with `?role=` (`/hr?role=hr_admin` was redirected home, because the guards of the first navigation saw no principal; `sampleRouteSnapshot()` in `auth/sample-auth-provider.tsx`). A workspace with `VITE_DATA_SOURCE=api` in `.env.local` must run with `VITE_DATA_SOURCE=sample` set in the shell for this suite (Vite lets the shell override `.env.local`), and a reused workspace server in api mode must be stopped first.
- `pnpm --filter @nestself/api test` runs everything; `pnpm test` runs the whole repository. CI (`.github/workflows/ci.yml`) provides Postgres and Redis service containers.

20. Recipes

Add a protected route. Put the handler in the right controller, pick the action from `ACTIONS`, and decorate with `@Authorize(ACTIONS['x.y'], orgResourceFromContext('thing'))` (or `operationsResource`, or `operationsOrgResource` under `/ops/orgs/:orgId`); pass a list when any of several actions may open the route (section 4). Step-up follows from the action's class; nothing to add. Access-administration writes also carry `@Throttle(WRITE_THROTTLE)`. When the decision needs row data (owner, case, confidentiality), mark the route `@Authenticated()`, load the row in the service and call `AuthorizationService.require()` before returning anything. Read the body with `@Body(new ZodValidationPipe(Schema))`; schemas live in `packages/contracts/src/org-admin.ts`. Do writes inside `dbService.withOrgScope(orgId, tx => repo.method(..., tx))`, audit through `AuditWriter`, and invalidate caches that your change affects (`PrincipalLoader.invalidateUser`, or `org_tree:{orgId}` for tree changes). The default-deny test tells you if you forgot the decorator; add an e2e case to the matching suite.

Add an action or template. Extend `ACTIONS` in `packages/contracts/src/actions.ts` (and `CONFIDENTIAL_ACTIONS` or `STEP_UP_CLASS_BY_ACTION` where relevant), add it to the template in `templates.ts`, add golden-table tuples in `apps/api/src/authz/golden-table.json`, and re-run the seed so sample grants pick it up.

Add a table or column. Edit `apps/api/src/db/schema/*.ts`, run `db:generate`, review the SQL in `apps/api/drizzle/`, add a `pgPolicy` with `tenantPredicate()` if the table has an organization column, run `db:migrate`, and extend the matching repository. Partitions, triggers and roles go in a `--custom` migration.

Add an audit action. Append to `AUDIT_ACTIONS` in `apps/api/src/audit/audit-events.ts` and map its category in `resolveCategory`; the spec file enumerates them.

Revoke someone's access programmatically. Write the audit event first, then `AccessRevocationService.revokeAllAccess(userId, { sessionsForOrgId?, orgIds? })`.

Call a sensitive action from a web screen. Wrap the mutation in `withStepUp(useAuth().stepUp, () => api.call(...))`, map its `ApiError` codes to translated copy, and treat `StepUpCancelledError` as the user declining (close or leave the form without an error).

Debug a denial. Call `explain(principal, action, resource, ctx)` from `src/authz/explain.ts` in a test, or read the `access.denied` audit row: `details.reason` carries the engine's denial reason.

Send an email. Add the template to `MailTemplate` and `EMAIL_TEMPLATES` (`src/email/`) in four locales, `sr-Cyrl` transliterated from `sr-Latn` (section 23), inject `Mailer`, and call `send` after the transaction that justifies the mail has committed: collect inside `withOrgScope`, send after it returns, as the invitation import does. Log the template, never the address, a code or a link.

Fetch a URL that a tenant or staff typed. Check it with `assertPublicHttpsUrl` and fetch it through a fetcher built like `HttpsMetadataFetcher` (`src/sso/metadata-fetcher.ts`: `https:` only, public unicast addresses only, a timeout, a size cap, no redirects), with a fake for stub mode and tests; log the error's reason, never the URL or the body (section 22.11).

21. Known gaps and next steps

Phase 4 (`docs/superpowers/plans/2026-10-30-auth-phase-4-pilot-readiness.md`) is done and tested locally; `docs/tasks/auth/phase-2-readiness.md` has the status and commits of every task of phases 2 to 4. The first table lists what phase 4 closed, from the previous version of this section and from section 22.14, with the task and commits that closed it. The second lists everything still open, each with its reason:

- **owner decision**: waits for the product owner;
- **pilot**: needs the pilot organisation, its identity provider or real traffic;
- **AWS**: needs a deployment, the AWS accounts or the live pool;

- **out of scope**: deliberately not built in the auth phases so far, or a small item nobody has scheduled.

Closed in phase 4

Item (where it was listed)	Closed by	Now
`conversations.case_id` and `intake_records.case_id` were free text: the blocker before any conversation write path (21)	Task 1, `f78d658..031061f`	`cases(id, org_id)` with composite foreign keys, migration `0007` (sections 10, 11.2)
`organizations.status` was free text (21, smaller items)	Task 1, `f78d658..031061f`	`organizations_status_check`, migration `0010` (section 10)
Import row reasons were English sentences the apps matched by prefix, and a refused duplicate or member was counted in `skipped` and in `errors` (21)	Task 1 (codes), Task 12 `6c4d12d..70794e4` (buckets), Task 14 `30e5e01..e4828c1` (copy for every reason)	One bucket per row with coded reasons (section 11)
Invitation mails were sent inside the import transaction, and the SES mailer was private to invitations	Task 2, `031061f..588f1f2`	The shared `Mailer`, mails after the commit (sections 11, 23)
Audit rows went through the writer's own connection, so a change could commit without its row; the emergency sweeper stopped at the first error (21, 22.14)	Task 4, `91d13b0..8f80b00`	`record(event, { tx })` in every transactional write; the sweepers close one row per transaction and go on (sections 12, 14)
SSO start and callback shared 10 a minute per IP, which one office would exhaust; a refused hop showed raw JSON (22.14)	Task 5, `8f80b00..268c494`	60 per minute per IP and connection for start, 120 per IP for the callback, Redis counters, a redirect on 429, `TRUST_PROXY_HOPS` (sections 4, 22.3)
Step-up answered a bare `forbidden` without a native identity, and federated users could not step up at all (SSO-R-17); step-up and invitation-accept codes carried no locale; wrong step-up codes had no per-user lock; the dialog dropped the challenge after one wrong code and had no resend cooldown or in-flight guard (21, 22.14)	Task 6, `268c494..84e9e93`	A NestSelf email code for federated users, `step_up_unavailable`, `StepUpThrottle`, resend, the locale, the dialog fixes (sections 6, 9, 15)
The external identity did not record the connection that bound it (22.14); the session took the pool username from a claim real ID tokens lack	Task 7, `84e9e93..93e0aff` (the column came with Task 1)	`external_identities.sso_connection_id`, checked by path 1; `cognito:username` (sections 10, 22.4)
The email code after `sso_email_unverified` did not bind, and that redirect carried the address (22.14)	Task 8, `93e0aff..f44048a`	The pending binding with an opaque handle and the confirm-email step (section 22.4)
Federated users were not resolved on mobile bearer lookups (22.14)	Task 9, `f44048a..4f833b1`	`POST /auth/sso/bind-token` and the federated bearer path (sections 8, 16)
The certificate expiry stayed unknown for metadata-URL providers, no alert mail went out, and provider changes on a `SUSPENDED` connection kept `last_tested_at` (22.11, 22.14)	Task 10, `4f833b1..c2b8fd5`	The guarded metadata fetch, `ops_certificate_alert`, `SUSPENDED → TESTING` (sections 14, 22.2, 22.11)
The admin SSO screen listed only the first 25 organisations, offered no protocol change or reset to draft, and kept the OIDC secret in mutation state (22.14)	Task 11, `c2b8fd5..6c4d12d`	Organisation search, protocol change in draft, Reset to draft, the secret cleared after every request, the test link shown as a link (section 22.8). The old note that the fallback link spent a second nonce was wrong: it reused the first one; the real problems were that the opened tab could not be confirmed and that following both links denied the second
`@Authorize` took one action, so the access page's member and org-unit lists needed `program.overview`; the member search treated `%` and `_` as wildcards	Task 12, `6c4d12d..70794e4`	Any-of `@Authorize`, `likePrefix` (sections 4, 10)
Confidential reads told a missing id (404) from a denied one (403), and the emergency read looked at the record before the access (21)	Task 13, `70794e4..30e5e01`	404 for every denial; the emergency read order (sections 11.2, 11.3)
The Redis health indicator detected memory mode with `instanceof`, and its 3-second timeout was untested (21)	Task 13, `30e5e01`	`KeyValueStore.kind`; a fake-timer test of the limit
`/hr/access` sat under `_hr`, so an employer owner without `hr_admin` could not reach it; the workspace member pickers stopped at the first 100 members; a stalled or failing `/auth/me` looked like being signed out (21)	Task 14, `30e5e01..e4828c1` and `8141da3`	The `access.preview` route gate, the member search, the session-unavailable state (sections 11.1, 15)
Staging and production had no Cognito domain, and AWS never set `SES_FROM_EMAIL` or `TRUST_PROXY_HOPS` (22.14)	Task 15, `e4828c1..c5a9c4c`	`authDomainName` with its context values, the API environment, the CloudFront-only load balancer (section 17.2); deploying it is an AWS item below
"Ending one's own membership is still refused only by the UI; the API follow-up is a 409 `self_end`" (21)	Already done before phase 4 (`ce4f9e4`, 2026-10-25): the note was stale	The API answers 409 `cannot_end_self` (`src/members/members.service.ts`)

Still open

Item	Reason	Notes
Authenticator (or passkey) step-up for the `emergency` class	owner decision	Every class steps up with an email code (section 9); the strategy asks for an authenticator, which with Cognito means a password-plus-authenticator step-up or passkeys
The `presenter` template (GAP-H02)	owner decision	Default: `content_publisher` keeps replay review, presenters use `expert` (`phase-2-readiness.md` section 4)

Item	Reason	Notes
Retention periods (L-02, L-03) and with them the retention sweeper	owner decision	The defaults of `audit-writer.ts` (1 to 10 years) stay meanwhile
Who may approve emergency access, and whether a second approver is required	owner decision	Default: `platform_owner`, one approver
The staging date	owner decision	Gates every AWS item below
Whether staff with `access.emergency.request` or `access.emergency.approve` but without `audit.read` (`security_admin`) should see `/emergency`, and with which list action	owner decision	The list route needs `audit.read`, which only `platform_owner` holds
IdP-initiated SSO	owner decision	Stays off (`idpInitiatedAllowed` is refused with 422) until a risk-acceptance process exists; the `saml_assertion:{id}` replay cache is not wired (SSO-R-12)
SCIM provisioning	owner decision	Accounts come from invitations or JIT (section 22.4)
The compliance track: privacy notices, processing agreement, breach procedure, data-subject-rights endpoints (T-C.4)	owner decision	Must land before the first real employee signs in
A real person in the sample data	owner decision	`SAMPLE_PEOPLE` carries the developer account `user_dragan`, added by the owner on 2026-10-24 to link the real dev pool and flagged `sample: true`. Guide screenshots show fictional people only: every shot file calls `removeDeveloperAccount()` (`e2e/auth/support.ts`) before any capture and `shot()` fails if the account is on screen (member lists are ordered by random ids, so filtering per screen would not be reliable)
The live IdP test with the pilot organisation (T-3.4)	pilot	Section 22.13: what the employer provides, what NestSelf prepares, and what only the live pool shows, including whether Cognito carries the IdP's own authentication time (the one condition for reconsidering an IdP round trip at step-up, section 9)
Whether the pilot's OIDC provider emits `email_verified`	pilot	Entra ID does not by default. Keep `oidcEmailVerifiedClaimRequired` on (each employee confirms the address once with a code) or turn it off after the client confirms in writing that its `email` claim is the directory's verified address (section 22.4)
A sweep of unbound Cognito pool users	pilot	A pending binding that is abandoned or expires leaves its federated pool user in Cognito with no identity; it can do nothing and the next sign-in with the same subject reuses it. The manual clean-up is in section 22.15; a job can follow once the pilot shows how often it happens
Retry and batching of invitation mails	pilot	An import of up to 500 rows sends that many mails at once, each without retry; a failed mail is logged and the invitation stands (section 11)
SSO throttle budgets against real traffic	pilot	60 starts per minute per IP and connection and 120 callbacks per IP (section 4); check them against the pilot's offices and egress addresses
Member choices in the admin app read the first 100 active members, with no search	pilot	The organization Access tab and the owner transfer on the Overview (`features/access/use-ops-admin.ts`); a client with more members, as the pilot organisation will be, needs the workspace's `MemberPicker` there
Deploy staging and production with the auth domain and the context values	AWS	`CDK-setup.md` section 8, with SES production access and the verified sender (section 9 there). The domain must exist before the first employer configures its IdP (section 17.2)
Confirm per stage that the load balancer refuses direct traffic	AWS	After the first real deploy, by the owner: `CDK-setup.md` 8.6 step 2 (section 17.2)
`cdk diff NestSelf-dev-Auth -c stage=dev -c account=$AWS_ACCOUNT_DEV` shows no change	AWS	Run by the owner before the next dev deploy; phase 4 left the dev template unchanged (section 17.2)
A secret origin header checked by an ALB listener rule	AWS	The CloudFront prefix list admits every distribution, so another account's distribution could reach the API around our edge WAF (section 17.2)
A marketing CloudFront distribution before the marketing launch	AWS	Nothing routes the apex to the load balancer, so the marketing site has no public path yet (section 17.2)
T-2.8: security baseline, SCPs, the production account	AWS	Waits for staging
T-2.12: the weekly Cognito user export	AWS	Needs the S3 archive bucket of staging
The audit export to S3 with Object Lock (AUDIT-R-06), then partition detachment	AWS	A method on `MaintenanceJobsService` plus a schedule entry, like the other jobs
Whether Cognito answers the third `CodeMismatchException` within one `Session` with `NotAuthorizedException`	AWS	The fake keeps the session alive for all three tries; if Cognito ends it sooner, the API treats that as any other provider error and ends the challenge, which is safe but gives fewer retries than the screens suggest (section 6)
A step-up screen and administration screens on mobile	out of scope	Web only by the decision of 2026-10-25; the API accepts bearer step-up (section 9)
A confirm-email screen on mobile	out of scope	A mobile-first user whose OIDC provider omits `email_verified` confirms the address once on the web (section 8)

Item	Reason	Notes
Screens for approval authorities and for changing a grant's conditions or validity (`PATCH /orgs/:orgId/grants/:id`)	out of scope	The API routes exist
T-2.9 and T-2.11: file delivery and export authorization	out of scope	Nothing to authorize yet
Retiring the community feature	out of scope	`posts`, `polls`, `poll_options`, `poll_votes`, `legacy_teams` and `memberships.team_id` are tenant-isolated but deprecated; the contracts keep `OrgRole` and `PlatformRole` for sample mode
An explicit request timeout for the SES client	out of scope	The client uses the SDK defaults, so a resend keeps its code challenge out of the store for the whole provider call, SDK retries included (section 6)
Access-administration writes are limited per route and client IP	out of scope	10 a minute per route and IP, not 10 per session across routes as AUTHZ-R-14 asks; a provider-originated 429 shows other copy than the lock message
A bearer principal without a token family is asked to step up but cannot	out of scope	The step-up routes answer 401 `session_required` (section 9): a bearer whose access token has no `origin_jti` has no token family to record a step-up on
Grants and delegation	out of scope	Delegatee invalidation runs after the commit, so a failure leaves caches stale for up to the grants TTL (about a minute). A PATCH that shortens a parent's validity does not re-check its children, and one that changes both conditions and validity checks the children against the stored validity. Repeated identical delegations are accepted and self-delegation is not refused. `grants.service.ts` should split platform grants and subject access into their own services. `POST /ops/grants` does not check the subject's population. An approval-authority PATCH cannot clear `validUntil` or `maxAmountMinor`
Invitations	out of scope	The two route families return slightly different 400 bodies; the invitation mail is written in `sr-Latn`, because an invitation carries no locale yet (`invitation-mailer.ts`)
Ownership transfer	out of scope	A new owner who already held a grant delegated from the old owner grant gets it re-parented under their own owner grant (redundant, harmless). The API e2e cases for a 403 transfer probably stop at the guard; the service's own owner check is proven by unit tests. Golden rows for the owner copy the template's actions by hand
Confidential and emergency reads	out of scope	Accepted residual of the 404 ruling: a refused read of an existing record writes an audit row that a missing id does not, so the two can differ in timing; the inactive-access `access.denied` row has `orgId: null` (sections 11.2, 11.3). Pending emergency requests never lapse, the record list is ordered by `grantedAt`, which approval overwrites, and `listMessages` is unbounded
Backfill	out of scope	Grant ids for a user's first organization still lack the organization; the HR admin chosen as `grantedByUserId` depends on a creation-time tie-break; an expert without approved credentials gets a grant without category conditions; the `0005` guard ignores `revoked_at` and validity; a legacy `MANAGER` without an org unit gets an organization-wide `team_manager` grant (reported in `widened`)
Web polish	out of scope	Spending ceilings assume two-decimal currencies; the user-access lookup cannot be re-run with the same id; the access dialogs pin revoke errors to the reason field and can focus a disabled confirm; a searchable combobox primitive in `@nestself/ui` (the pickers put a search field in front of `Select`)
Names and links the responses do not carry	out of scope	Conversations do not name their intake (the expert types the intake id); conversation participants, emergency requesters and approvers and the platform grant table show user ids; `/auth/me` exposes no CLIENT assignments, so a `client_operations` user reaches an assigned client only by link
Smaller UI items	out of scope	Content read in the emergency record view stays on screen if the window ends while the dialog is open; picker failures are toasts only; the sample-mode context switch does not change the navigation; the Playwright stub API reads `REDIS_URL` from `apps/api/.env`, so it shares Redis with a developer's API (sessions of earlier runs appear in the sessions screenshot)
Visual snapshot baselines outside the workspace and admin entries	out of scope	The marketing pages fail axe's `document-title` rule and differ from their screenshots and 320px reflow baselines, and the suite's `testDir: './e2e'` also collects `e2e/auth/*.spec.ts` (`testIgnore` for `auth/**` would fix that)
No Playwright suite runs in CI	out of scope	The journeys, the guide screenshots and the motion suite run locally (design 6.7)
Tests	out of scope	Several API e2e suites share one session across cases or depend on case order; `ageSession` and `extractSessionId` are duplicated across specs
A `custom:locale` attribute wins over the screen's language	out of scope	The CustomMessage trigger prefers the pool user's attribute to `ClientMetadata.locale` (section 6)
Mobile authentication strength	out of scope	A native mobile bearer reports `email_otp` without a second factor, the only first factor the mobile client allows (section 8); revisit with passkeys
A throttled admin SSO start lands on the workspace sign-in	out of scope	The redirect cannot know the surface of a refused request (`AppThrottlerGuard`)

Item	Reason	Notes
WebSocket messages share one throttle bucket per handler	out of scope	Pre-existing, not verified
The metadata fetch's accepted residuals	out of scope	DNS rebinding between the address check and the connect is not caught (pinning the address would need a custom connector), and the nightly certificate sweep covers `ACTIVE` connections only: a `TESTING` or `SUSPENDED` connection's expiry is refreshed when staff test it or change its metadata (section 22.11)
The email code length is AWS's, not ours	out of scope	Below

The email code length is AWS's, not ours. Cognito's managed `EMAIL_OTP` challenge always sends eight digits: there is no pool setting, no CDK property and no CLI parameter for it (verified 2026-10-24 against `describe-user-pool`, `update-user-pool` and `UserPoolProps`). Shortening it to four or six means leaving the managed flow for `CUSTOM_AUTH` with Define, Create and Verify Auth Challenge triggers, where the code, its delivery through SES, its expiry and its attempt counter all become ours. That is a deliberate trade: shorter codes are friendlier to type and materially weaker (four digits is 10,000 combinations), and the throttle would have to tighten to compensate. Not started. The codes NestSelf issues itself (section 6) have the same eight digits on purpose, so every code field takes one length.

22. Enterprise SSO

Auth phase 3 (plan `docs/superpowers/plans/2026-10-25-auth-phase-3-sso.md`, branch `feat/auth-sso`) lets an employer's own SAML 2.0 or OIDC identity provider sign its employees in. Staff register, verify, test and activate the connection in the admin app; employees type their work address on the usual sign-in screen and are sent to their employer's sign-in. Everything runs locally against a fake identity provider; the live test against a real employer IdP (T-3.4) waits for the pilot organisation. Phase 4 closed the follow-ups that would have bitten on the first day with a real employer (section 21 names the tasks): per-IP-and-connection throttles with a redirect on 429, the binding connection on every federated identity, the pending binding and confirm-email step after an OIDC provider that does not confirm the address, federated step-up, federated users on mobile, the certificate expiry of metadata-URL providers with an alert mail, `SUSPENDED → TESTING` on a provider change, and the admin screen's gaps.

The flow is **SP-initiated only**: the sign-in always starts at NestSelf. `idpInitiatedAllowed` stays `false` (create and PATCH answer 422 `validation` for `true`) until there is a risk-acceptance process, so the `saml_assertion:{id}` replay cache of the strategy is not wired (SSO-R-12).

22.1 Data model

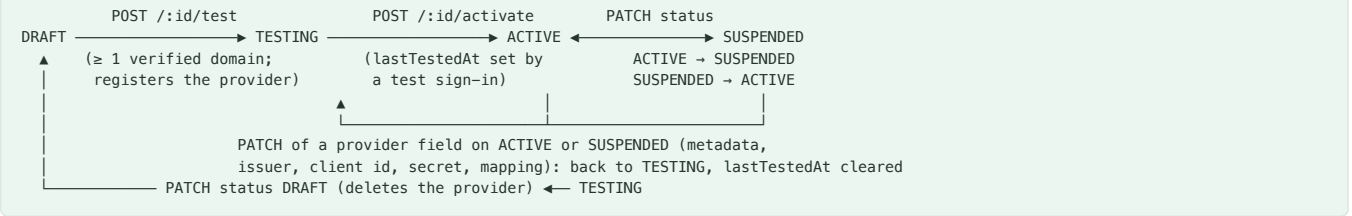
Two tables in `apps/api/src/db/schema/invitations.ts`, both under forced row-level security:

- `sso_connections`: `org_id`, `protocol` (`SAML` | `OIDC`), `cognito_provider_name` (unique, `org-<slug>-saml` or `org-<slug>-oidc`, at most 20 characters; `buildProviderName` in `src/sso/provider-name.ts`; the slug is reserved for one organization, see 22.2), `status`, SAML `metadata_url` / `metadata_xml`, OIDC `oidc_issuer` / `oidc_client_id`, `attribute_mapping`, `idp_initiated_allowed`, `jit_provisioning`, `oidc_email_verified_claim_required` (default `true`, migration `0009`; 22.4), `default_org_unit_id`, `certificate_expires_at`, `last_tested_at`. Policy `sso_connections_tenant_isolation` on `org_id`. **The OIDC client secret is never stored**: it is a write-only field of the create and PATCH bodies, handed straight to Cognito, absent from every response, and dropped from audit details and logs (`oidcClientSecret` is in the audit writer's forbidden keys and in the pino redaction paths).
- `sso_domains`: `connection_id`, `domain` (lowercase, unique across all connections), `verification_token`, `verified_at`. Migration `0006_sso_domains_rls.sql` gives it the policy `sso_domains_tenant_isolation`, which reaches the tenant through `EXISTS (select 1 from sso_connections ... org_id = current setting)`.

A federated identity records the connection it was bound through: `external_identities.sso_connection_id` (`src/db/schema/identity.ts`, migration `0008`, `on delete restrict`), which path 1 checks (22.4).

`SsoRepository` (`src/db/repositories/sso.repository.ts`) does every read and write. Writes run in the connection's tenant scope (`withOrgScope(conn.orgId, ...)`) together with their audit rows. Reads that must span organizations run in the system context on purpose: provider-name uniqueness, the per-organization slug reservation (`findConnectionsByProviderNames`) and domain uniqueness, the staff list, `findActiveConnectionForDomain` (discovery happens before any organization is known), and `listActiveConnections` for the nightly job.

22.2 Lifecycle



What each route does (`src/sso/sso-connections.service.ts`):

Route	Allowed from	Effect
`POST /ops/sso-connections`		Creates a `DRAFT` row. 422 `validation` for an over-long provider name (`details.field: 'slug'`), an OIDC secret on a SAML connection, `idpInitiatedAllowed: true` or a default org unit outside the organization; 404 for an unknown organization; 409 `conflict` for a provider name the same organization already uses, and 409 `conflict` with `details: { reason: 'provider_name_taken', field: 'slug' }` when a connection of **another organization** holds `org-<slug>-saml` or `org-<slug>-oidc` (the slug reservation, below). An OIDC create that carries the secret registers the provider at once, because NestSelf cannot hold the secret until the test ("staged registration"; discovery ignores `DRAFT` either way). `metadataUrl` and `oidcIssuer` must be public `https:` URLs (below the table). `oidcEmailVerifiedClaimRequired` is stored as sent (default `true`) and its value audited. Audit `sso_connection.created`
`POST /:id/domains`	any status	422 `domain_blocklisted` for a public email domain, 409 `conflict` when the domain is attached to any connection; returns the TXT record name and value. Audit `sso_domain.added`
`POST /:id/domains/:domainId/verify`	any status	DNS lookup (22.5); sets `verified_at` or answers 422 `domain_unverified`; an already verified domain is returned unchanged. Audit `sso_domain.verified`
`POST /:id/test`	`DRAFT`, `TESTING`	Needs a verified domain (409 `sso_connection_state`) and complete metadata (422 before any provider call); checks the URLs again (a host may resolve elsewhere than when it was stored), creates or updates the Cognito provider, re-reads the signing-certificate expiry from the metadata (fetched from `metadataUrl` through the `MetadataFetcher`, else read from `metadataXml`; a failed fetch keeps the stored date, 22.11), sets `TESTING` and clears `last_tested_at`; returns `{ startUrl }`, a **single-use test link** (22.4). Audit `sso_connection.changed { from, to }` (when the status changed) and `sso_connection.tested`
`POST /:id/activate`	`TESTING`	Needs `last_tested_at` and a verified domain, else 409 `sso_connection_state`. Audit `sso_connection.changed` and `sso_connection.activated`
`PATCH /:id`		`status`: `ACTIVE → SUSPENDED`, `SUSPENDED → ACTIVE`, `TESTING → DRAFT` (deletes the provider and clears `last_tested_at`); any other transition 409 `sso_connection_state`. Protocol changes only in `DRAFT` (422 otherwise); the rename to the other protocol's name answers 409 `provider_name_taken` (`field: 'protocol'`) when another organization holds either name of the slug. Metadata, OIDC fields, secret or mapping changes update the registered provider; while `TESTING` they clear `last_tested_at`, and on an `ACTIVE` or `SUSPENDED` connection they also move it back to `TESTING` (from `ACTIVE`: audit `sso_connection.deactivated { from: 'ACTIVE' }` plus `sso_connection.changed { from: 'ACTIVE', to: 'TESTING' }`; from `SUSPENDED`: `sso_connection.changed { from: 'SUSPENDED', to: 'TESTING' }`): discovery stops routing to it, and it is activated again only after a test sign-in of the new configuration (SSO-R-01; before phase 4 a `SUSPENDED` connection kept `last_tested_at`, so a reactivation needed no test). A provider change together with a status change on an `ACTIVE` or `SUSPENDED` connection is refused with 422 `validation` (`field: 'status'`). On a `DRAFT` a PATCH keeps the status and stages the provider: a secret registers it, a provider field updates a staged one, a switch to SAML drops it (`syncStagedProvider`); only `TESTING → DRAFT` deletes the provider. A change of `metadataUrl` or `metadataXml` re-reads the certificate expiry as the test does. `oidcEmailVerifiedClaimRequired` is NestSelf's own rule, not a provider setting: a PATCH changes it without a re-test, and the audit row records its new value. Audit `sso_connection.updated { fields }`, and for a status change `sso_connection.changed { from, to }` plus `activated` / `deactivated`
`GET /ops/sso-connections[?orgId=]`		The list with domains and `spMetadata`

Status changes are conditional (`updateConnection(..., { fromStatus })`): a concurrent change answers 409.

Public `https:` URLs only. `metadataUrl` and `oidcIssuer` are URLs that something fetches (the API the metadata, Cognito the OIDC discovery document). The request schema accepts only `https://` (`httpsUrl` in `packages/contracts/src/ops-admin.ts`; anything else is 400 `validation`), and create, PATCH and test run `assertPublicHttpsUrl` (`src/sso/metadata-fetcher.ts`) with the metadata fetcher's own resolver (22.11), answering 422 `validation` with `details: { field, reason }`: `reason: 'scheme'` for a URL that does not parse or is not `https:` (the schema stops these first; a URL stored before phase 4 meets it at the next test), `'host'` for credentials in the URL or a host that does not exist (`ENOTFOUND`, `ENODATA`, `ENOTIMP` or no address), `'private_address'` for a host that is, or resolves to any, non-public address. A resolver that does not answer (`EAI_AGAIN`, `ETIMEOUT`, `ETIMEDOUT`, or nothing within 5 seconds) is 503 `unavailable`: the URL is neither accepted nor refused, try again. The admin app shows "The address must be a public https URL." at the field.

A slug belongs to one organization. Cognito keeps the federated pool users it created (`<providerName>_<NameID>`, each with its `sub`) after their provider is deleted. If another organization could later register the same provider name, its IdP asserting the same NameID would get the first organization's pool user and `sub`, and resolution path 1 (22.4) would sign it in as that organization's user. A DRAFT protocol switch frees a name in Cognito (`org-acme-saml` → `org-acme-oidc`), so the reservation covers both names of the slug: create and a protocol change look up `org-<slug>-saml` and `org-<slug>-oidc` across every organization (system context, `findConnectionsByProviderNames`) and refuse with 409 `provider_name_taken` when a row of another organization holds either. The reservation lives on the connection rows; there is no route that deletes a connection. Identity provider refusals (bad metadata, Cognito `InvalidParameterException`) answer 422 `validation`; outages 503 `unavailable`.

22.3 Routes

Method	Path	Decorator	Throttle
GET	`/ops/sso-connections`	`@Authorize(sso.manage)`, `operationsResource('sso_connection')`	default
POST	`/ops/sso-connections`	same	10/min
PATCH	`/ops/sso-connections/:id`	`@Authorize(sso.manage)`, `operationsResource('sso_connection', id)`	10/min
POST	`/ops/sso-connections/:id/domains`	same	10/min
POST	`/ops/sso-connections/:id/domains/:domainId/verify`	same	10/min
POST	`/ops/sso-connections/:id/test`	same	10/min

Method	Path	Decorator	Throttle
POST	`/ops/sso-connections/:id/activate`	same	10/min
GET	`/auth/sso/start`	`@Public`, `@RedirectOnThrottle()`	60/min per IP and connection (`SSO_START_THROTTLE`)
GET	`/auth/sso/callback`	`@Public`, `@RedirectOnThrottle()`	120/min per IP (`SSO_CALLBACK_THROTTLE`)
POST	`/auth/sso/bind/start`	`@Public` (pending binding, 22.4)	10/min per IP (`SSO_BIND_THROTTLE`); one code per pending binding a minute
POST	`/auth/sso/bind/verify`	`@Public` (sets the session cookies)	10/min per IP
POST	`/auth/sso/bind/resend`	`@Public`	10/min per IP; one code a minute per challenge
POST	`/auth/sso/bind-token`	`@Public` (the mobile app's binding, section 8; 204)	10/min per IP
GET	`/auth/sso/_fake/authorize`	`@Public`, **stub mode only** (22.7)	default

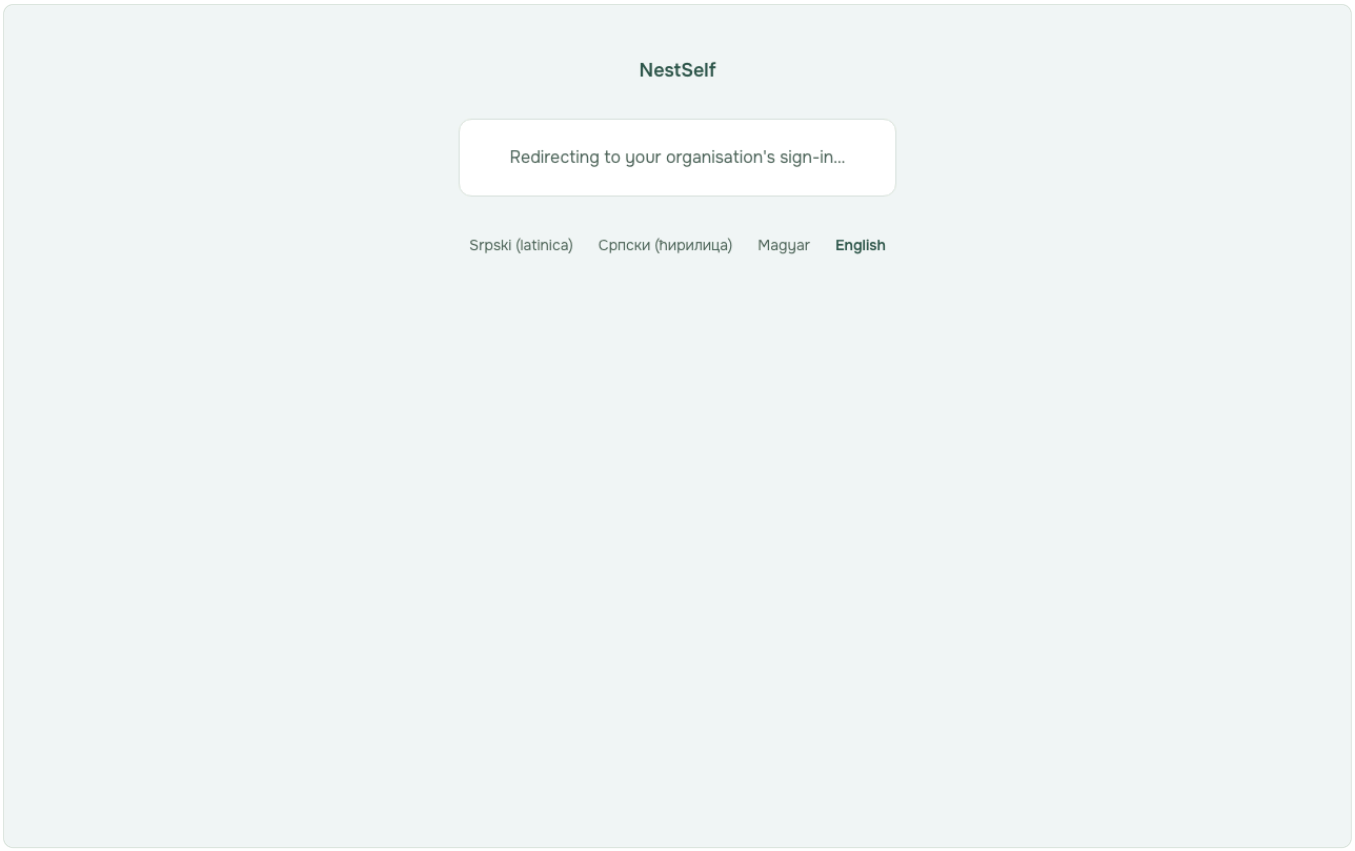
`sso.manage` is held by `platform_owner` only and has the `administration` step-up class, so every `/ops/sso-connections` route needs a sign-in or step-up from the last 30 minutes.

A refused `start` or `callback` is not the JSON 429: `AppThrottlerGuard` (`src/common/app-throttler.guard.ts`) throws `ThrottledRedirect` and the exception filter answers it with a bodyless 302 to the workspace `/sign-in?reason=sso_rate_limited` (`Cache-Control: no-store`, `Retry-After: 60`), which the sign-in page shows as "wait a minute" (`auth:error.ssoRateLimited`). The target is the workspace whatever the surface, because the callback's surface is known only from its PKCE record, which a refused request never reads. With `REDIS_URL` set the counters are in Redis (`KeyValueThrottlerStorage`), so every API task counts against the same budget. The start throttle counts per IP and `connection` value, so employees of two organizations behind one office or VPN address do not share a budget; rotating made-up connection names gains nothing, because an unknown connection writes no audit row and no PKCE record (22.4).

Which origin the browser routes use. `start`, `callback` and the fake `authorize` are reached through the surface's own origin, not `API_PUBLIC_URL`: the workspace origin for `surface=workspace`, the admin origin for `surface=admin` (`apiOriginForSurface`, `src/auth/sso-start-url.ts`). Every SPA proxies `/api` to the API (CloudFront `/api/*` to the ALB on both distributions, `infra/cdk/lib/edge-stack.ts`; the Vite dev proxy to `:4000`), and the session cookie `__Host-nestself_session` (`Path=/`, no `Domain`) binds to the exact host that sets it. With every URL built from `API_PUBLIC_URL` (the workspace origin in staging and production) an admin flow set its cookie on `app.<stage>` and redirected to `admin.<stage>`, whose `/api/auth/me` never saw it; locally it worked only because localhost cookies ignore ports. A flow also has to start on the origin its callback runs on, because the browser-binding cookie set at start is host-only. The registered web-client callbacks (`webCallbackUrls` in `infra/cdk/lib/config.ts`: both SPA origins, plus `:4000` in dev) are exactly the ones the API now sends; `API_PUBLIC_URL` remains for mobile and direct callers.

22.4 Home-realm discovery, start and callback

Discovery (`DiscoverService`, `POST /auth/discover`): the address is trimmed and lowercased and its domain taken; a public email domain (`PUBLIC_EMAIL_DOMAINS` in `src/auth/constants.ts`: the 13 domains of strategy 3.4.3: `gmail.com`, `yahoo.com`, `hotmail.com`, `outlook.com`, `live.com`, `icloud.com`, `aol.com`, `protonmail.com`, `mail.com`, `yandex.com`, `zoho.com`, `gmx.com`, `fastmail.com`; exact match after trimming a trailing dot, so subdomains do not match) never routes to SSO; a verified domain of an `ACTIVE` connection answers `{ method: 'sso', ssoStartUrl, connection }`, where `ssoStartUrl` is `<surface origin>/api/auth/sso/start?connection=<provider>&surface=<workspace|admin>` (the surface comes from the request's `Origin`; the origin is `WORKSPACE_URL` or `ADMIN_URL`, 22.3) and `connection` is the provider name (mobile passes it as `identity_provider`). Everything else gets the native method. Every answer, SSO or not, has the same shape and status and waits for the same `neutralDelay` (SSO-R-04); a failing connection lookup answers natively. The web sign-in screens show "Redirecting to your organisation's sign-in..." and follow the URL.



Workspace sign-in after discovery answered `sso` for `employee@acme.example`. For the screenshot the start URL in the real discovery answer was replaced with `javascript:void(0)`, so the browser stayed on this screen instead of leaving for the identity provider.

Start (`GET /auth/sso/start?connection&surface[&returnPath][&test]` , `SsoCallbackService.start`): the query is validated by `SsoStartQuerySchema` ; `returnPath` must be a relative path (no `//` , no backslash, no control characters, API-R-06), otherwise 400. The connection must be `ACTIVE` , or `TESTING` with a test nonce issued for it; anything else is a neutral 302 to `/sign-in?reason=sso_denied` . An existing connection that is not usable (`DRAFT` , `SUSPENDED` , `TESTING` without a valid nonce) writes an `auth.sign_in.failed` audit row (reason `sso_connection`) for its organization. An unknown connection name writes none, only a debug log line: the row would name no tenant, and the start throttle counts per connection value, so one client rotating made-up names could write rows without bound. Then a 32-byte PKCE verifier, a 32-byte state and a 32-byte browser nonce are generated. The PKCE record `{ verifier, surface, returnPath, connection, testNonce, browserNonceHash }` is stored at `pkce:{state}` for `PKCE_TTL` (600 s); the browser gets the nonce as a cookie and a 302 to the identity provider's authorize URL (`buildAuthorizeUrl` : Cognito's `/oauth2/authorize` with `identity_provider` , `code_challenge_method=S256` and the redirect URI `<surface origin>/api/auth/sso/callback` ; the code exchange repeats the same URI, derived from the record's `surface` , because Cognito compares them).

Browser binding. The nonce cookie closes login CSRF: an attacker who starts a flow and hands the victim the callback link cannot complete it in the victim's browser. The cookie is `__Secure-nestself_sso` (`nestself_sso` with `SESSION_COOKIE_INSECURE_DEV=true`), HttpOnly, `SameSite=Lax` , `Path=/api/auth/sso/callback` , `Max-Age` = `PKCE_TTL` . It is `__Secure-` rather than `__Host-` because a `__Host-` cookie must have `Path=/` , and the narrow callback path is worth more here; `__Secure-` keeps the Secure guarantee. Only its SHA-256 is stored in the PKCE record. Every callback response clears it.

Callback (`GET /auth/sso/callback` , always a 302, `Cache-Control: no-store`):

1. `GETDEL pkce:{state}` : a replayed or never-issued state finds nothing and goes to the workspace's `/sign-in?reason=sso_denied` (audit reason `sso_state`). Nothing else is recorded and no session is created.
2. A test flow's nonce (`sso_test:{nonce}`) is spent here, whatever the outcome.
3. The cookie's hash is compared timing-safe with the record (reason `sso_browser`); an IdP `error` or missing code (`sso_idp_error`), a failed code exchange (`sso_exchange`) and a code issued for another provider than the flow's (`sso_provider_mismatch`) are all neutral `sso_denied` redirects.
4. Identity resolution (D-04), below.
5. On success the session is created as for any sign-in, with `auth.method = 'sso'` and `secondFactor = true` : the second factor is delegated to the employer's IdP, which the client confirms enforces MFA when it onboards (3.4.9, runbook 10.1). The browser goes to `${WORKSPACE_URL|ADMIN_URL}${returnPath ?? '/'}` .

The four resolution paths (`SsoCallbackService.callback` ; paths 2 to 4 and the checks around them live in `FederatedBindingService` , `src/auth/federated-binding.service.ts` , which the confirm-email step and the mobile `bind-token` use too):

1. **Existing identity.** An `external_identities` row with provider `cognito:<providerName>` and the token's `sub` signs in, whatever the connection's status (a connection suspended mid-flow, or reset to `DRAFT` , still lets known users finish), provided `existingIdentityUsable` passes: the connection that owns the provider name still exists, the identity was bound through that very connection (`ssoConnectionId` equals its id; a null binding matches none), and the identity's user has an `ACTIVE` membership in that connection's organization (an `INVITED` or `ENDED` one does not vouch for the sign-in). This is defence in depth for the slug reservation (22.2): a provider name that ever moved to another organization would otherwise hand it the first organization's pool users. Otherwise the browser gets the neutral `sso_denied` , `auth.sign_in.failed` is audited with reason `sso_provider_mismatch` , and the pool user is **not** deleted (it belongs to a real user elsewhere).
2. **Pending invitation.** For a new binding the connection must be `ACTIVE` (or `TESTING` on a valid test flow) and the email verified (below). A pending invitation of **the connection's organization** for the same address (trimmed, lowercased, compared timing-safe with SHA-256 digests, SSO-R-09) creates or reuses the user by email, links the external identity with `ssoConnectionId` set to the connection, and activates the membership, in one transaction in the connection's tenant

scope with its audit rows. Audit `user.created` (new user), `identity.linked`, `membership.activated`, then `invitation.accepted`. An invitation of another organization never binds.

3. **JIT.** Only on an `ACTIVE` connection (never while `TESTING`) with `jit_provisioning` on: user (reused by email), external identity (bound to the connection) and an ACTIVE membership in `default_org_unit_id` with the `employee` grant. Audit `user.created` (new user), `identity.created`, `identity.linked`, `membership.activated` (details carry `via: 'sso_jit'` and the connection id). The grant is recorded with the new user as `grantedByUserId`, because there is no inviter.
4. **Denied.** Nothing to bind to: the orphan federated user Cognito created is deleted (`deleteUser`, SSO-R-10), `identity.denied` is audited (reason `no_invitation` or `connection_inactive`) and the browser lands on `/sign-in?reason=sso_denied`, where the screen says "Your organisation has not enabled your account yet. Ask your HR administrator."

A binding transaction that fails is logged by the error's name and SQLSTATE only (`bindingFailureReason`): the driver's message carries the statement's parameters, the address among them.

New bindings need a verified domain of the issuing connection. The email is treated as verified only when its domain is a verified domain of that same connection; for OIDC the token must also carry `email_verified === true`, unless staff turned the connection's `oidcEmailVerifiedClaimRequired` off; for SAML the domain inference is enough. This is stricter than SSO-R-08 on purpose (phase 3, Task 4 review): otherwise a tenant-controlled IdP could assert someone else's address, at another employer or a personal one, and, through the reuse of users by email, take that account over. An employer whose employees use addresses on other domains must verify those domains first. What follows when the rule fails depends on which half failed:

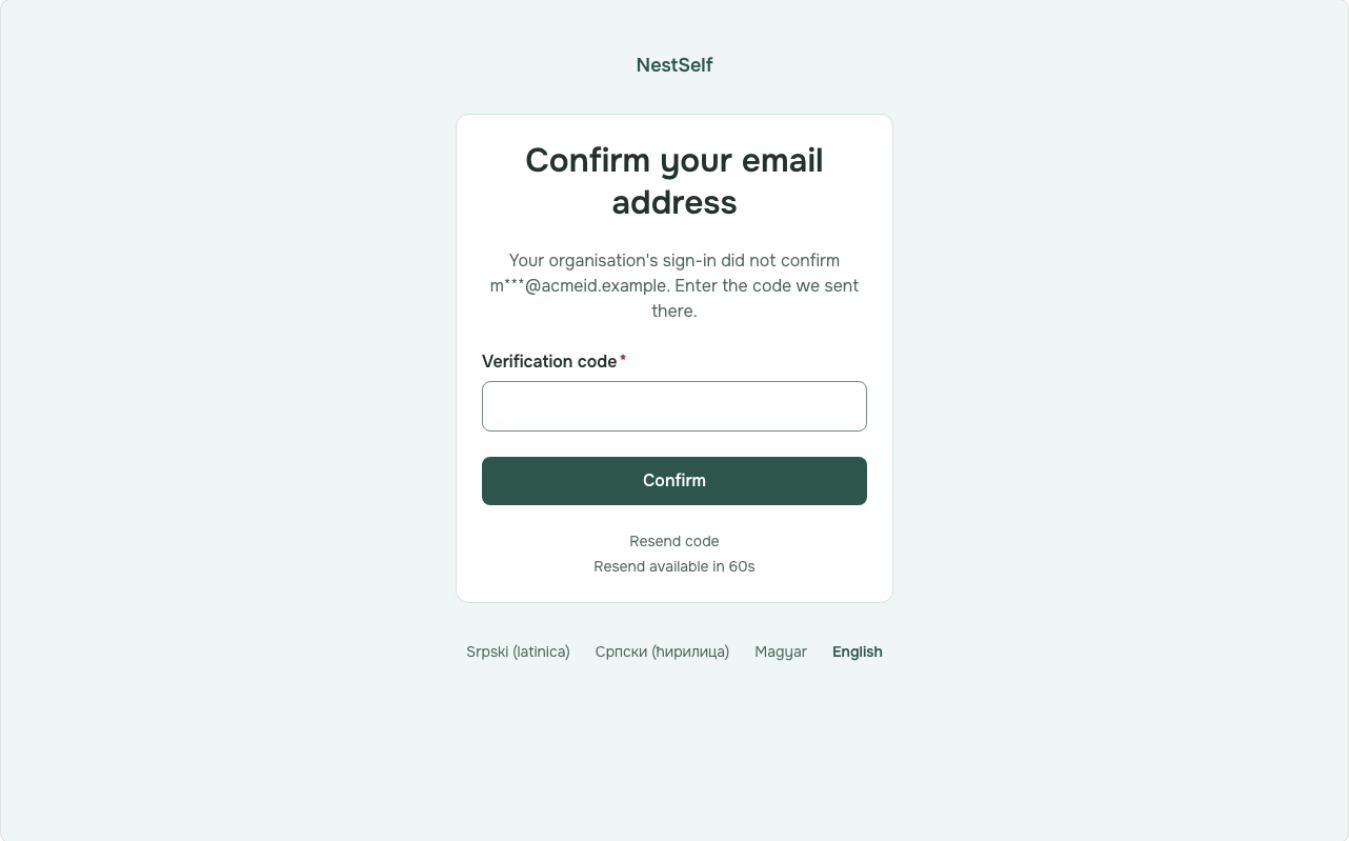
- **Off-domain address** (SAML or OIDC, whatever `email_verified` says): the orphan is deleted, `identity.denied` with reason `email_off_domain`, and the neutral `/sign-in?reason=sso_denied`. The connection vouches for nothing about that address, so the screen neither names it nor claims the IdP failed to confirm it.
- **On-domain OIDC address without `email_verified === true`** while the connection requires the claim: the sign-in waits for the user to prove the mailbox (the pending binding, below). With the switch off the address counts as verified, as for SAML, and paths 2 to 4 run at once. Staff turn it off only for an IdP that never emits the claim (Entra ID does not by default) and only after the client confirms in writing that its `email` claim is the directory's verified address (runbook 10.1); the create and PATCH audit rows record the value.

Existing identities are not affected by the rule.

The pending binding and the confirm-email step (SSO-R-08, strategy 3.4.5 step 4; phase 4 design 5.6). For the on-domain OIDC address above, the callback keeps the pool user and stores a pending binding (`PendingBindStore`, `src/auth/pending-bind-store.ts`): the connection, the provider name, the `sub` and pool username, the address, the surface and return path, whether the flow came through a valid staff test link (`testSignIn`) and the token set of the code exchange, its three tokens encrypted with the session `TokenCipher`. The record lives at `sso_pending_bind:{sha256(handle)}` for `SSO_PENDING_BIND_TTL` (900 s); the handle, 32 random bytes in base64url, is the only thing the browser carries. The callback audits `identity.verification_started` (`emailVerifiedClaim` says whether the provider said no or said nothing) and redirects to `/sign-in?reason=sso_email_unverified&bind=<handle>`. **The redirect never carries the address**; the access log redacts the `Location` of the redirect and the `Referer` of the page's later requests, which carry the handle (section 12).

The sign-in route accepts `bind` (32 to 128 URL-safe characters; a malformed one is dropped) and the page renders `SsoBindScreen` (`auth/screens/sso-bind-screen.tsx`, both apps):

- On mount it calls `POST /auth/sso/bind/start { handle, locale }` once (a ref keeps React's StrictMode from mailing twice). `SsoBindService.start` (`src/auth/sso-bind.service.ts`) mails a NestSelf code (purpose `sso_bind`, template `sso_bind_code`, section 6) to the asserted address and answers `{ challengeToken, emailMasked }`; the screen names the masked address. The pending binding stays: a code lives 5 minutes and the handle 15, so a user whose code expired asks for a new one from the same screen without signing in at the IdP again. One code per pending binding a minute (`sso_bind_start:{sha256(handle)}`, 60 s; 429 `rate_limited` with `retryAfter`, released when the mail is refused). An unknown or expired handle is 400 `invalid_challenge`.
- Confirm calls `POST /auth/sso/bind/verify { challengeToken, code }`. The address shares the email-code sign-in lock (`SignInThrottle`, `throttle:otp:{email}`: five failures in 15 minutes): a locked address gets no code and has none checked (429 `too_many_attempts`), every wrong code counts against it, and a sign-in through the code clears it, so guessing here is no cheaper than in sign-in. After the code, the pending binding is taken once (`GETDEL`). An IdP account that has an identity by now (another tab, a retry, a direct sign-in after the claim switch went off) signs in through path 1 exactly as the callback runs it, and its pool user is never deleted. Otherwise the connection is reloaded and must still vouch for the address: `ACTIVE` (or `TESTING` when `testSignIn` is set), the same provider name, the domain still verified (Review Focus 2). If not, the orphan is deleted, `identity.denied { reason: 'connection_inactive' }` is audited and the answer is 403 `sso_denied`. Then paths 2 to 4 bind exactly as in the callback, in the connection's tenant transaction, with `via: 'sso_email_code'` on the rows; nothing to bind deletes the orphan and answers 403 `sso_denied`. The session is created from the stored token set, whose `expiresIn` is reduced by the record's age, the cookies are set and the answer is `{ returnPath }`, to which the page continues.
- "Resend code" calls `POST /auth/sso/bind/resend`, at most once a minute per challenge, with a countdown; a challenge the server no longer has (expired, three wrong codes) is dropped and the button asks `bind/start` for a new code from the same handle. One request runs at a time. Failures render translated copy in `FormErrorSummary`; `sso_denied` and an expired handle end the step with a link back to `/sign-in`.
- A reload within a minute of the first code shows the countdown instead of mailing a new code (the start slot is taken), and the code from the first mail cannot be used after the reload, because its challenge token lived only in the page. After the countdown the screen asks for a new code.
- A `reason=sso_email_unverified` link without a handle (an old link) still explains (`auth:error.ssoEmailUnverified`) and continues with discovery.



The confirm-email step after an OIDC provider signed in `maria@acmeid.example` without confirming the address: the page names it masked and waits for the code NestSelf mailed. The URL carries only the opaque `bind` handle. In the screenshot run no code is entered, so nothing is bound.

A pending binding that is abandoned or expires leaves its federated pool user in Cognito: that user can do nothing (no identity, no bearer resolution, no native flow), and the next sign-in with the same subject reuses it and opens a new pending binding. There is no sweep yet (section 21; manual clean-up in 22.15).

The session username is the pool username from the ID token's `cognito:username` claim, then `username`, then `sub` (`IdentityResolutionService`, `src/auth/identity-resolution.service.ts`). A real Cognito ID token has no `username` claim (only the access token does), so before phase 4 a federated session kept the `sub`, which is not a federated user's pool username; the fake token factory now issues Cognito-shaped ID tokens, so every fake sign-in takes the real branch.

Two related guards outside the callback: `POST /invitations/accept` for an address on a verified domain of an `ACTIVE` connection of the inviting organization writes nothing and returns `{ method: 'sso', ssoStartUrl }`, so the binding happens at the callback; and inviting (or importing) an address whose domain belongs to another organization's `ACTIVE` connection is refused with 409 `conflict`, `details.reason: 'sso_domain_claimed'` (an import lists the row with reason `sso_domain_claimed`, section 11).

Test sign-ins. `POST /ops/sso-connections/:id/test` stores `sso_test:{nonce}` with `{ connectionId }` for `SSO_TEST_TTL` (900 s) and returns a start URL with `test=<nonce>` (32 to 128 URL-safe characters; a literal `test=1` is a 400). The test link is a workspace flow: the staff member opens it in a new tab and, after the employer's sign-in, lands on the workspace home as the test account (the admin app itself is not involved after the click). `start` on a `TESTING` connection accepts only a nonce issued for that connection; the callback consumes it; `last_tested_at` is set only when that flow's sign-in completed (binding or existing identity). A second use of the same link is denied, and every new test needs a new link. An OIDC provider that sends no `email_verified` (Entra ID by default) sends the staff test through the confirm-email step too: the pending binding records `testSignIn`, so `bind/verify` may bind on the `TESTING` connection (never JIT) and then sets `last_tested_at`, as the callback's test path does; otherwise no such connection could ever be activated.

22.5 DNS TXT verification

`POST /:id/domains` returns the record to publish: name `_nestself-verify.<domain>`, value `nestself-domain-verification=<token>`, where the token is `randomBytes(16)` in hex. `AuthoritativeDnsTxtResolver` (`src/sso/dns-txt-resolver.ts`, used with `AUTH_MODE=cognito`) walks up from the record name to the closest zone that has NS records (never the TLD), takes the first name server (sorted), resolves it to an address (IPv4, IPv6 fallback) and asks that server directly with its own `Resolver({ tries: 1 })`, so no caching resolver answers for the zone. TXT chunks are joined, every value is trimmed and compared with `timingSafeEqual`, the whole lookup has a 5-second limit, and any error counts as "not found". In stub mode the resolver is `FakeDnsTxtResolver` (tests set records through `app.get(DNS_TXT_RESOLVER)`; `FAKE_DNS_TXT_AUTOVERIFY=true` makes it answer every pending domain with its expected value, which the Playwright stack uses. The verification token never appears in audit details.

22.6 Cognito operations

`src/identity/cognito-sso-ops.ts`, called through `IdentityProviderService`:

- **Provider CRUD.** `CreateIdentityProvider`, `UpdateIdentityProvider`, `DeleteIdentityProvider`, `DescribeIdentityProvider`, `ListIdentityProviders` (paginated). SAML details are `MetadataURL` or `MetadataFile`; OIDC details are `client_id`, `client_secret`, `oidc_issuer`, `authorize_scopes`, `attributes_request_method`. Default attribute mappings (the SAML email claim; OIDC `email` and `email_verified`) sit under the caller's mapping. An OIDC update without a new secret keeps the stored one. `describeProvider` reports the earliest `notAfter` of the SAML signing certificates parsed from the metadata (`saml-metadata.ts`, `node:crypto` `X509Certificate`; encryption certificates ignored).

- **App clients.** Cognito refuses `identity_provider=<name>` unless the client lists the provider, so registering adds it to `SupportedIdentityProviders` of both the web and the mobile client, and deleting removes it first: `DescribeUserPoolClient`, then `UpdateUserPoolClient` with every writable field copied back (an update resets omitted fields), `COGNITO` always first, no call when nothing changes. The read-modify-write has no lock; two concurrent registrations could drop an entry, and the next `updateProvider` (every test) repairs it.
- **Token exchange.** `POST ${COGNITO_DOMAIN}/oauth2/token` with Basic client authentication, `code`, `redirect_uri` and `code_verifier`, 10-second timeout; `invalid_grant` maps to `invalid_code`, other failures to the retryable `unavailable`.
- **ID-token verification.** `aws-jwt-verify` (already a dependency) checks the RS256 signature against the pool JWKS (cached per `kid`), `iss`, `aud` (the web client) and `token_use = id`. JWKS outages are `unavailable`, every other verification failure `not_authorized`. Claims are normalised (`email_verified` as boolean, `identities` as an array, the pool username from `cognito:username`, which a real ID token carries instead of `username`; `toIdTokenClaims`). The mobile `bind-token` route verifies the mobile client's ID tokens the same way (section 8).

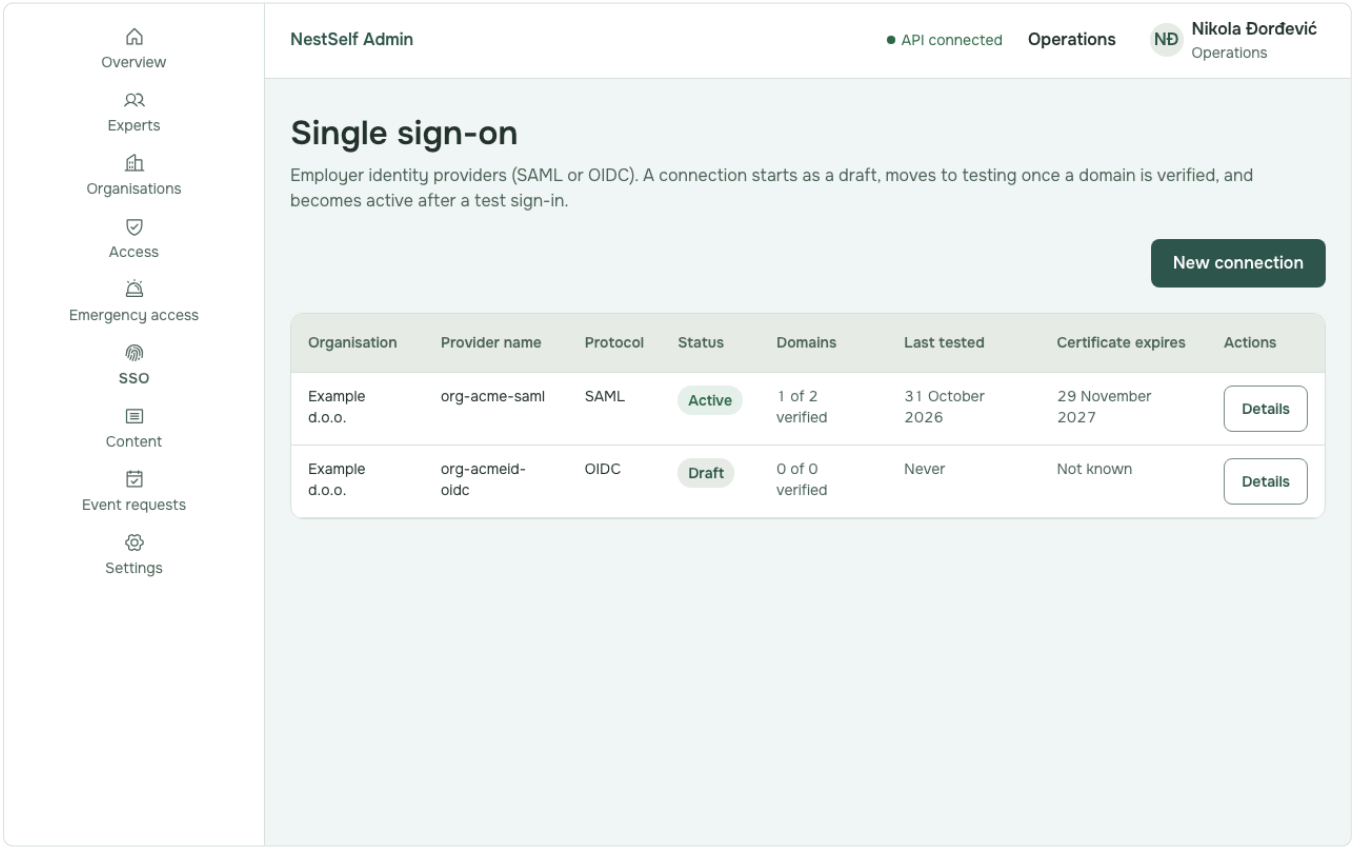
The API task role carries the provider actions and `UpdateUserPoolClient` (`infra/cdk/lib/constructs/ecs-task-roles.ts`; the policy JSON for a local least-privilege identity is in `CDK-setup.md` section 5.2).

22.7 The fake identity provider in stub mode

`FakeIdentityProvider` keeps providers in memory (`fake-sso-store.ts`; secrets are not kept, SAML metadata is parsed for the certificate expiry). `buildAuthorizeUrl` points at `/api/auth/sso/_fake/authorize` on the origin of the redirect URI, i.e. the surface's origin (`API_PUBLIC_URL` only when the redirect URI does not parse), which stands in for "Cognito, then the employer's IdP, then Cognito": it issues a single-use, 10-minute authorization code bound to the S256 challenge and the redirect URI and redirects to the callback. `as=<email>` picks the federated user who "signs in at the employer"; an address that was not seeded is created on the fly (verified; `email_verified=false` makes it unverified, and a user seeded with `emailVerified: null` gets no claim at all, as from Entra ID). The exchange checks the verifier, materialises the federated pool user (`<providerName>_<subject>`, kept apart from native users so email codes never reach it) and issues an ID token with `identities`, shaped like Cognito's: `cognito:username` and no `username` claim (that one is on the access token), so every fake sign-in takes the branch the real pool takes (`fake-token-factory.ts`). The route is registered only with `AUTH_MODE=stub` (`FakeSsoModule` through `ConditionalModule`; a cognito-mode API logs "Skipping the registration of FakeSsoModule"), accepts exactly the registered callbacks (`registeredSsoCallbackUrls`: the workspace, admin and `API_PUBLIC_URL` origins plus `/api/auth/sso/callback`) and refuses any other `redirect_uri` with 400, and 404s unless the identity provider is the fake.

22.8 The admin SSO page

Admin `/sso` (`apps/admin/src/features/sso/`, nav item "SSO" shown with `sso.manage`; the screen shows the permission state without it and switches to the operations context). Every mutation goes through `withStepUp`, refreshes the list afterwards (also after a failure) and maps API codes to `admin:sso.errors.*` in four locales with an `unknown` fallback; the API's `message` is never shown. A 422 `validation` naming `metadataUrl` or `oidcIssuer` with reason `scheme`, `host` or `private_address` shows "The address must be a public https URL." at that field, on create, edit and test. In sample mode the page shows `SAMPLE_SSO_CONNECTIONS` (`packages/sample-data`) read-only with every action disabled.



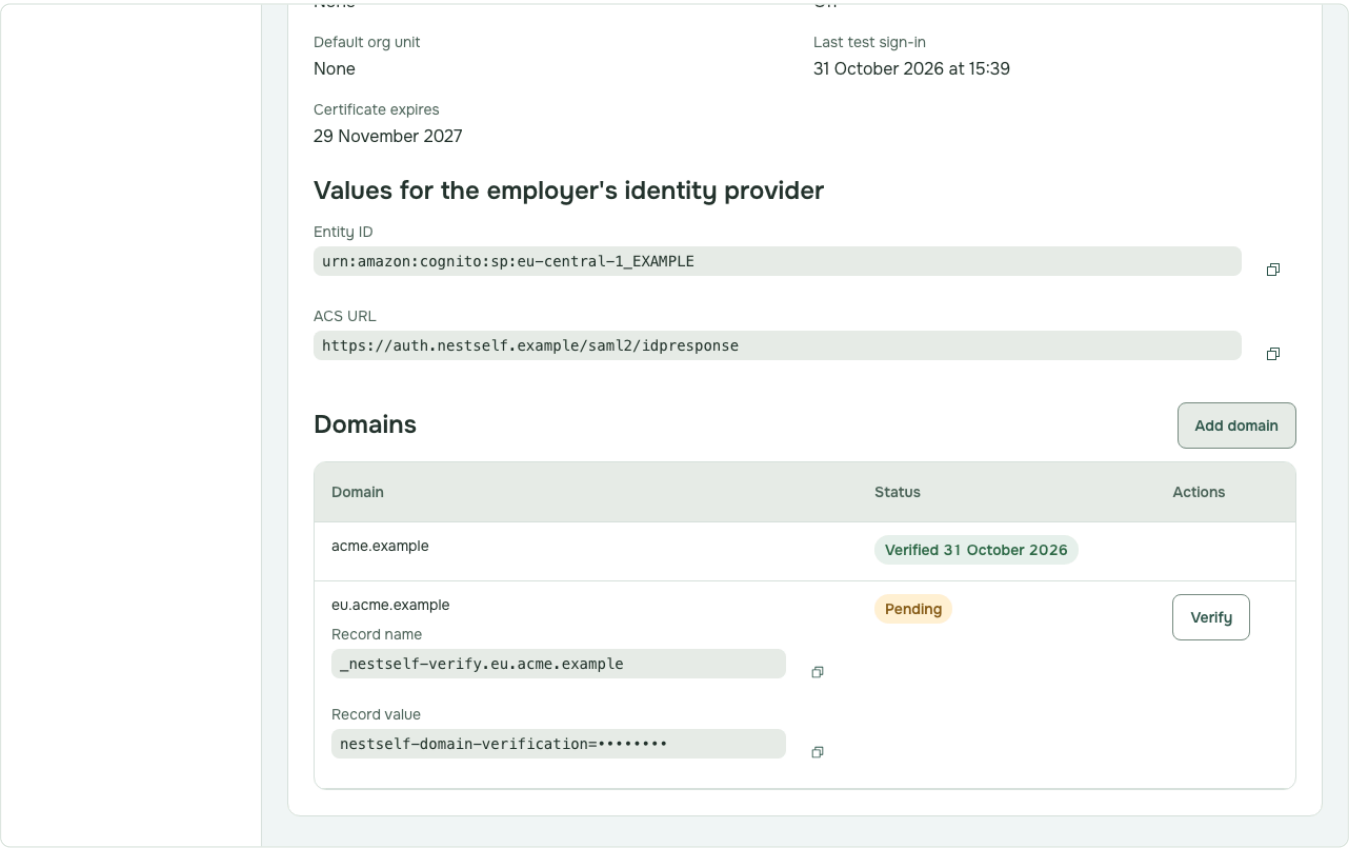
The connections list: an active SAML connection with one of its two domains verified, last tested today, and a draft OIDC connection, both for the fictional sample organization. Created through the stub API for the screenshot; the active one went through a real test sign-in against the fake identity provider. The certificate date, 29 November 2027, is the expiry of the test-only metadata fixture's certificate.

The list and the detail name an organisation beyond the first page of clients with one read of that client (`useOrganization`), else its id. The detail panel shows the settings (the OIDC secret only as "Write-only, never shown", the last test sign-in and the certificate expiry), the service-provider values to hand to the employer

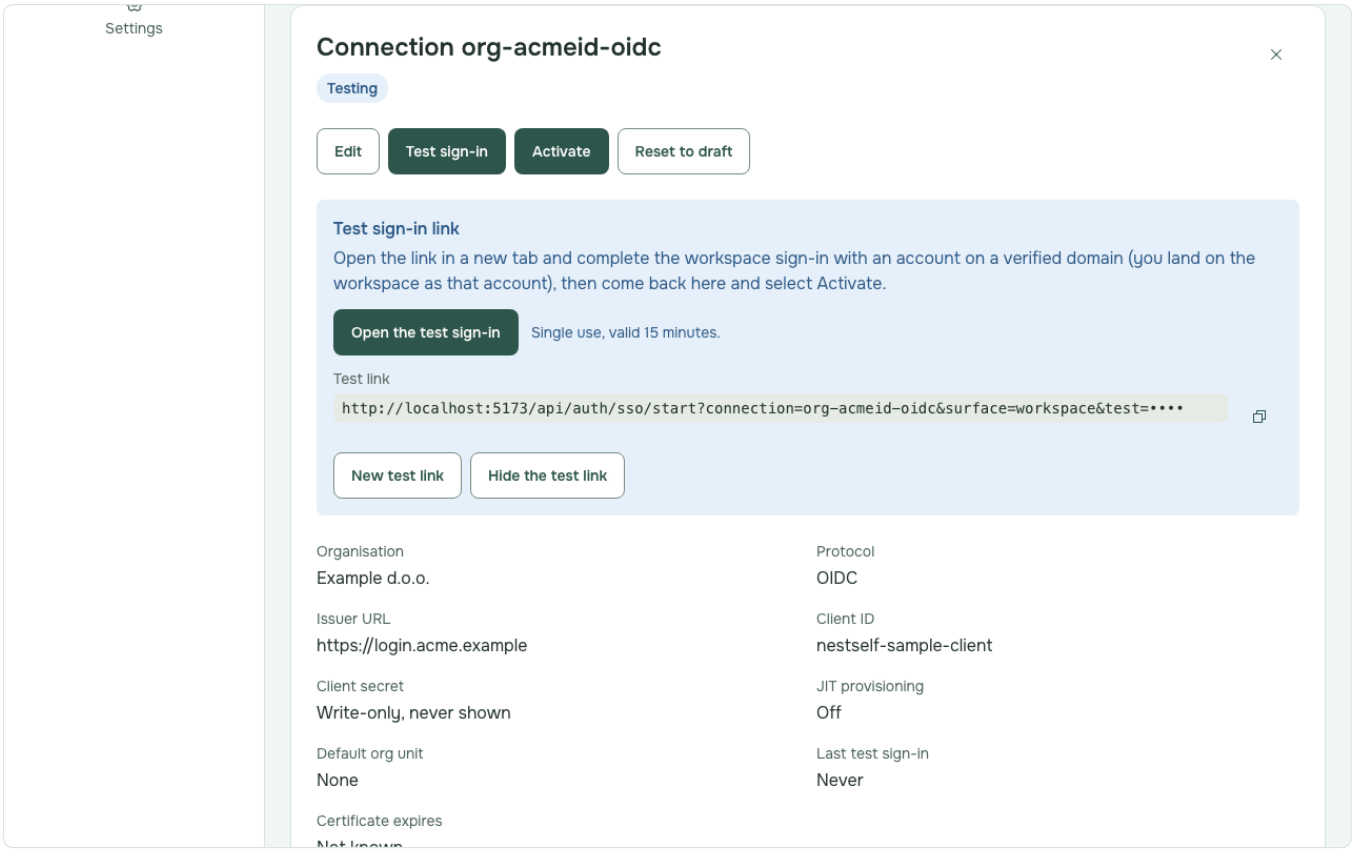
(entity ID `urn:amazon:cognito:sp:<pool id>` and ACS URL `<COGNITO_DOMAIN>/saml2/idpresponse`, each with a copy button), and the domains. A pending domain shows its TXT record name and value with copy buttons and a Verify button. The actions depend on the status:

- **Test sign-in** (`DRAFT`, `TESTING`) mints the single-use link and shows it as a link to open ("Open the test sign-in", `target="_blank" rel="noopener"`) and as a value to copy, with "Single use, valid 15 minutes."; the page opens no tab itself. "New test link" mints another (and, like every test, clears the last test sign-in); "Hide the test link" closes the panel.
- **Activate** (`TESTING`), **Suspend** with a confirmation (`ACTIVE`), **Reactivate** (`SUSPENDED`).
- **Reset to draft** (`TESTING`), behind a confirmation that says it deletes the provider registered at the identity provider and clears the last test sign-in, and that an OIDC connection needs its client secret again before the next test.
- **Edit**: only the changed fields are sent; the secret can be replaced, never read, and leaves the form and the mutation state after every request that carried it, refused or not (`reset()` in a `finally`). The protocol can change while the connection is a `DRAFT` (the new protocol's settings go in the same PATCH) and is locked otherwise. For OIDC the dialog has the "Require a verified email claim from the provider" switch, with a hint that it is turned off only after the client confirms its email claim is the directory's verified address. On an `ACTIVE` connection the dialog warns that changing the provider settings takes it back to testing, on a `SUSPENDED` one that it moves to testing, and the toast confirms it when it did.

A create refused with `provider_name_taken` shows "Another organisation already uses this slug" at the slug field.

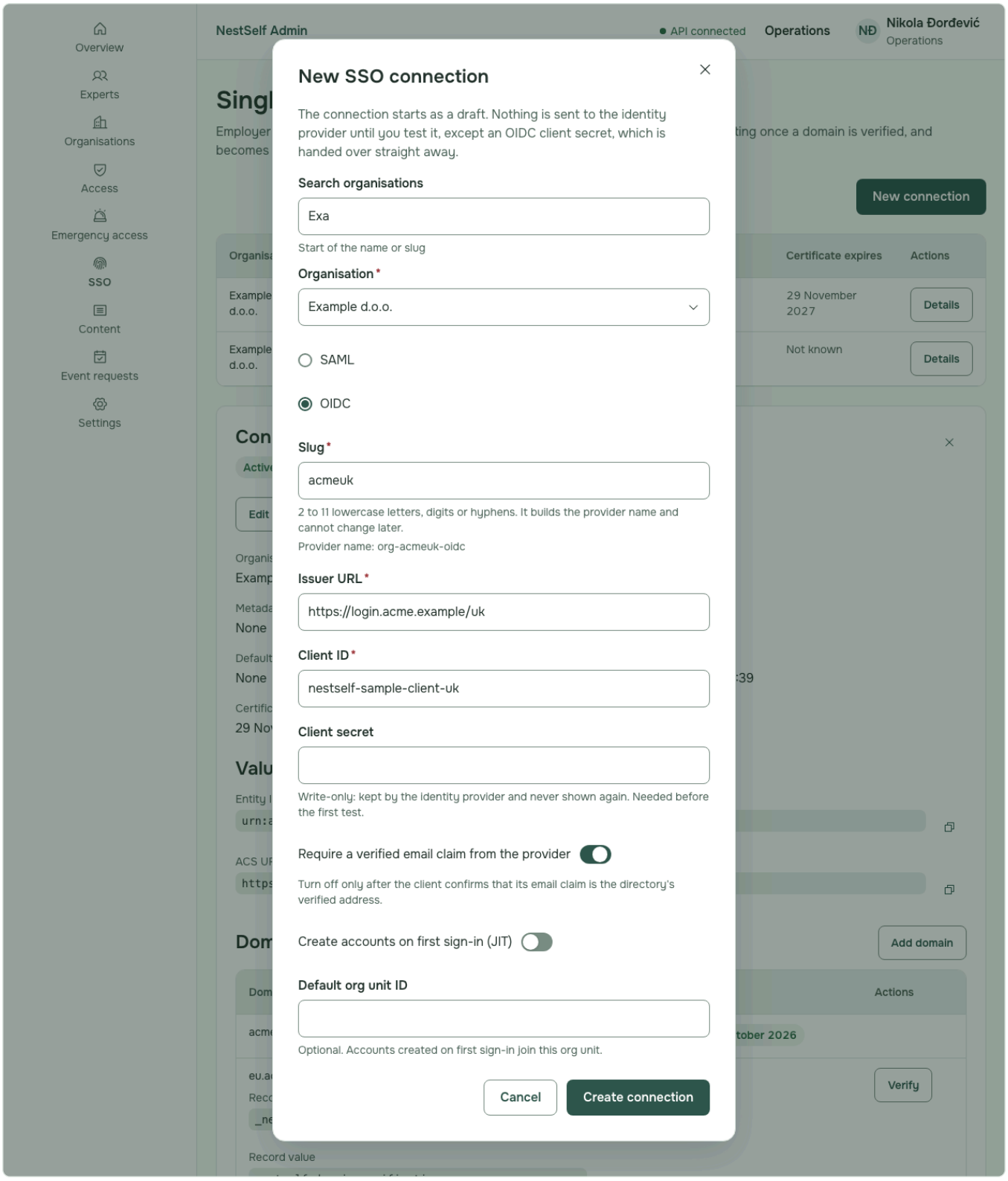


The detail of the active connection, scrolled to the service-provider values and the domains: `acme.example` is verified, `eu.acme.example` waits for its TXT record (the verification token in the record value is masked in the screenshot). The pool id and domain are fictional values the Playwright stack sets for the stub API.



The test-link panel of the OIDC connection in `TESTING`: the single-use link to open or copy, with its note (the nonce in the link is masked in the screenshot), and Reset to draft next to Activate. The screenshot's tester then signed in through that very link.

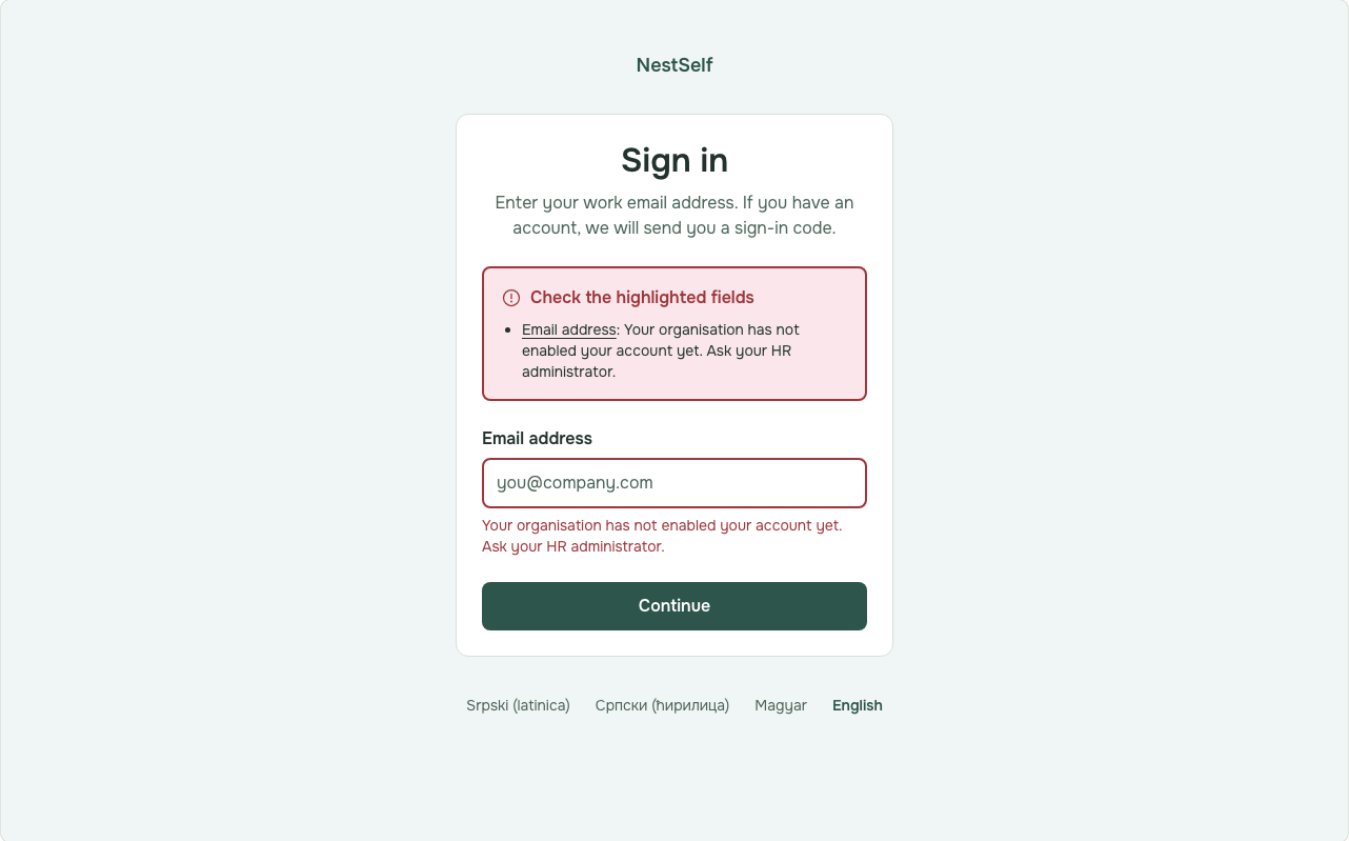
The create dialog takes the organisation through `OrganizationPicker` (`features/organizations/organization-picker.tsx`: a search field whose text, debounced for 300 ms, goes to `GET /ops/organizations?search=` as a prefix of the name or slug, the first 25 matches in the `Select` and "Load more" for the next page; the chosen organisation stays when a later search no longer returns it; Enter in the search field does not submit the form), then the protocol, the slug (with a live preview of the provider name), SAML metadata from a URL or pasted XML, or the OIDC issuer, client ID and write-only secret with the verified-email-claim switch (on by default), the JIT switch (with a warning) and the default org unit. `idpInitiatedAllowed` is not offered.



The create dialog with the organisation found through the search field and the whole OIDC form, the claim switch included, filled in and cancelled: nothing was created. This capture is 1280×1500, taller than the others, because the form does not fit in 800 pixels.

22.9 What an employee sees

An employee of a connected employer types the work address on `/sign-in`, sees the redirect line (screenshot in 22.4), signs in at the employer, and lands on the page they came for. When the employer's OIDC provider did not confirm the address, the page asks once for the code NestSelf mailed to it (the confirm-email step, 22.4) and then continues. Without an invitation (and with JIT off) they see the neutral denial:



`/sign-in?reason=sso_denied`: what the callback shows when the identity provider signed someone in whom NestSelf cannot bind. The same neutral message covers an expired or replayed link.

Too many starts or callbacks from one address land on `/sign-in?reason=sso_rate_limited`, which says to wait a minute (22.3). Federated users step up with an email code to their work address (section 9). On mobile, a federated sign-in binds through the API first (section 8); an OIDC address the provider did not confirm is confirmed once on the web.

22.10 Audit actions

Action	Written by
`sso_connection.created`, `.updated`, `.changed`, `.tested`, `.activated`, `.deactivated`	Connection routes (details `{ from, to }` for status changes, `{ fields }` for updates, plus `oidcEmailVerifiedClaimRequired` with its new value when that switch changed; `.deactivated` with `{ from: 'ACTIVE' }` whenever a connection leaves `ACTIVE`, by a suspension or by a provider change that takes it back to `TESTING`, since discovery stops routing to it either way); `.changed` with `{ reason: 'certificate_expiring', expiresAt, connectionId, daysLeft, job: 'sso.certificate-expiry' }` by the nightly job, one row per night per connection until the certificate is replaced
`sso_domain.added`, `sso_domain.verified`	Domain routes (`{ connectionId, domain }`)
`identity.linked`, `identity.created`, `user.created`, `membership.activated`, `invitation.accepted`	Bindings of the callback, the confirm-email step (`via: 'sso_email_code'`) and the mobile `bind-token`, in the binding transaction (`invitation.accepted` after it)
`identity.verification_started`	The callback, when an OIDC sign-in waits for the confirm-email step (`{ emailNormalised, connectionId, emailVerifiedClaim, testSignIn }`)
`identity.denied`	Binding refusals (`no_invitation`, `connection_inactive`, `email_off_domain`, `email_unverified`; `via: 'sso_email_code'` when NestSelf had confirmed the address)
`auth.sign_in.failed`	Start and callback refusals (`sso_connection`, `sso_state`, `sso_browser`, `sso_idp_error`, `sso_exchange`, `sso_provider_mismatch`, `sso_binding`, `sso_session`); the bind routes and `bind-token` (`sso_provider_mismatch`, `sso_binding`, `sso_session`, `sso_email_unverified`); a refused federated bearer (`sso_provider_mismatch`, once per connection and pool user in 600 s, section 8)
`auth.step_up.completed`, `auth.step_up.failed`	Step-up, with `method: 'email_otp'` and, on completion, `issuer: 'cognito' | 'nestself'` (section 9)

All are ADMIN category except `auth.*` (SECURITY). Details never carry tokens, codes, the verifier, the state, the nonces, the pending-bind handle, a challenge token, the OIDC secret, the verification token or metadata XML (`codeVerifier`, `verifier`, `state`, `nonce`, `testNonce`, `browserNonce`, `handle`, `challengeToken`, `oidcClientSecret` and `verificationToken` are forbidden keys), and the logs never carry the metadata URL or document.

22.11 Certificate expiry and the metadata fetch

The nightly `sso.certificate-expiry` job (section 14) looks at every `ACTIVE` connection; a `TESTING` or `SUSPENDED` connection's expiry is refreshed when staff test it or change its metadata (22.2), not by the job (a ruling of phase 4: the job keeps the scope it had). Where the expiry comes from:

- **A metadata-URL SAML provider**: the API fetches the metadata itself, every night and at every test and metadata PATCH, and stores the earliest `notAfter` of its signing certificates (`certificateExpiryFromMetadata` , `src/identity/saml-metadata.ts` ; encryption certificates ignored). Cognito cannot help: for such a provider `DescribeIdentityProvider` returns the `MetadataURL` , not the file. Certificates rotate, so the nightly fetch keeps the stored date current.
- **A metadata-XML SAML provider**: the expiry is read from the pasted XML at test and PATCH; while it is unknown, the job asks `describeProvider` each night and stores what the provider reports.
- **OIDC**: no SAML signing certificate, so no expiry.

The metadata fetch (`HttpsMetadataFetcher` , `src/sso/metadata-fetcher.ts` , behind the `METADATA_FETCHER` token that `IdentityModule` provides with `AUTH_MODE=cognito`) is the one place the API requests a URL a tenant supplied, so it follows the outbound rule of `CLAUDE.md` before any connection is made (`assertPublicHttpsUrl`): `https:` only and no credentials in the URL; the host is resolved and every address must be public unicast (`isPublicAddress` , `src/sso/public-address.ts` : loopback, private, link-local with the cloud metadata address, carrier-grade NAT, unspecified, multicast, reserved and IPv4-mapped equivalents are refused, so a host answering with one public and one private address is refused too). The request has no redirects (`redirect: 'manual'` , any 3xx is a failure, so metadata URLs must serve the document directly), no cookies or credentials, a 5-second deadline for the lookup, the request and the body together (`METADATA_FETCH_TIMEOUT_MS`) and a 1 MiB cap on the body, enforced while it streams (`METADATA_MAX_BYTES`). Failures carry a fixed reason and are logged as `metadataFailureLabel` (the error's name and reason), never with the URL, whose query may hold a tenant identifier, or the body. The same check refuses a bad URL at create, PATCH and test (22.2). DNS rebinding between the check and the connect is an accepted residual: pinning the checked address would need a custom connector. In stub mode and tests `FakeMetadataFetcher` (`src/sso/fake-metadata-fetcher.ts`) never touches the network: an unseeded host resolves to a documentation address, an unseeded URL serves the test-only fixture `SAMPLE_SAML_METADATA` , whose certificate expires on 2027-11-29, so every stub-mode SAML connection shows that date, and tests seed failures and private answers.

Warning and alert. A certificate that expires within 30 days (`SSO_CERTIFICATE_WARNING_DAYS`), or already has, is audited as `sso_connection.changed` (actor `JOB` , the connection's organization, details `{ reason: 'certificate_expiring', expiresAt, connectionId, daysLeft, job: 'sso.certificate-expiry' }`) in its own transaction and logged as a warning with the provider name and date only, one row per night per connection until the certificate is replaced (new metadata registered). Only once that row has committed is `ops_certificate_alert` mailed to `OPS_ALERT_EMAIL` , in English, with the organization, the provider name, the date, the days left and a link to the admin `/sso` page (section 23), at most once per connection in 7 days (`SSO_CERT_ALERT_TTL` , `sso_cert_alert:{connectionId}` , taken before the send so overlapping runs cannot both mail, released when the send fails so the next run retries). Without `OPS_ALERT_EMAIL` the run audits and logs only and warns once that alert mail is off. A refresh that fails (the fetch, the provider lookup or the write) or fetched metadata without a signing certificate is counted in `failed` and logged, and the connection is still checked against its stored expiry, so an outage at the IdP never hides a known expiring date; an audit transaction or an alert that fails is counted in `failed` too. Everything failed is retried the next night. Staff then follow runbook 10.2 (strategy) with the employer.

22.12 Tests

- Unit: `sso-connections.service.test.ts` , `provider-name.test.ts` , `dns-txt-resolver.test.ts` , `sso-callback.service.test.ts` (real fake IdP and in-memory store), `federated-binding.service.test.ts` , `sso-bind.service.test.ts` (the pending binding and `bind-token`), `pending-bind-store.test.ts` , `id-token-verifier.test.ts` , `nestself-otp.service.test.ts` , `metadata-fetcher.test.ts` , `public-address.test.ts` , `fake-metadata-fetcher.test.ts` , `throttles.test.ts` , `app-throttler.guard.test.ts` , `key-value-throttler-storage.spec.ts` , `fake-sso-authorize.controller.test.ts` (the route is absent or 404 outside stub mode), `cognito-sso.spec.ts` , `fake-identity-provider.sso.spec.ts` , `saml-metadata.test.ts` , the `sso.certificate-expiry` cases in `maintenance-jobs.service.test.ts` ; in both web apps `sso-bind-screen.test.tsx` and the sign-in route tests; in the admin app `sso-errors.test.ts` , `sso-screen.test.tsx` , `connection-detail.test.tsx` , `create-connection-dialog.test.tsx` , `edit-connection-dialog.test.tsx` and `organization-picker.test.tsx` ; on mobile `sso.test.ts` and `sign-in-error.test.ts` .
- Integration (`TEST_DATABASE_URL`): `test/sso.e2e.spec.ts` (suffix `sso` : the lifecycle, the move back to `TESTING` on a provider change of an `ACTIVE` or `SUSPENDED` connection, blocklisted and duplicate domains, provider-name rules and the slug reservation across organizations, `idpInitiatedAllowed` , the secret and the token absent from responses and audit rows, the claim switch, `http:` refused by the schema and non-public hosts by the service (Review Focus 4), the certificate expiry fetched at test, a secret staged on a `DRAFT` , non-staff 403), `test/sso-auth.e2e.spec.ts` (suffix `ssoauth` : discovery and its timing, start and `returnPath` , every callback path on both surfaces (`surface=admin` runs on the admin origin: its authorize `redirect_uri` , the callback and a session `/auth/me` accepts; the fake refuses a foreign `redirect_uri`), the verified-domain rule, login CSRF with two agents, single-use test links, the binding connection and `cognito:username` , federated step-up and foreign challenges, the pending binding end to end (a connection suspended or a domain unverified in between: Review Focus 2), `bind-token` and the federated bearer (a provider name of another organization: Review Focus 3), the throttle buckets per connection and the redirect on 429, an unknown connection writing no audit row), `test/db/sso-repository.spec.ts` (suffix `ssorepo` , as the application role so RLS applies).
- Browser: `e2e/auth/sso-sign-in.spec.ts` (staff create, verify, test and activate a SAML connection through the API; an invited tester completes the test sign-in; an invited employee signs in from the workspace sign-in screen through the fake IdP; a federated HR administrator steps up with a NestSelf code; and an OIDC connection whose provider does not confirm the address, where the staff test and an employee finish through the confirm-email step).

22.13 What the live IdP test (T-3.4) needs from the pilot organisation

Everything above runs against the fake identity provider. Before the first real connection goes `ACTIVE` :

1. **On the AWS side first.** Deploy the stage with the Cognito custom domain (`CDK-setup.md` section 8): the ACS URL and the OIDC redirect URI the employer registers derive from it, so it must be final before the employer configures anything (section 17.2). Set `OPS_ALERT_EMAIL` (`-c opsAlertEmail`) so certificate alerts reach a mailbox, check that `SES_FROM_EMAIL` can send (production access), and confirm that the load balancer refuses direct traffic, because the per-IP throttles rely on it (`CDK-setup.md` 8.6).
2. From the employer: their IdP metadata URL or XML (OIDC: issuer, client ID and secret; the metadata URL must be public `https:` and serve the document without a redirect, 22.11), the email domains they own and access to publish TXT records on them, written confirmation that their IdP enforces MFA (the session records the second factor as delegated), for OIDC whether the ID token carries `email_verified` (and, if it does not, written confirmation that the `email` claim is the directory's verified address before staff turn the claim switch off, 22.4), their offices' egress addresses if many employees share one (the start throttle allows 60 a minute per address and connection, 22.3), and one test account whose address is on a verified domain.
3. From NestSelf: the entity ID and ACS URL from the detail panel, an invitation for the test account (or JIT on), `WORKSPACE_URL` , `ADMIN_URL` and `API_PUBLIC_URL` set, and the callback `/api/auth/sso/callback` of both SPA origins registered on the web client (the CDK stack already lists them, 22.3).
4. Then on the live pool: create, add and verify the domains, Test sign-in (the link from the detail panel; for an OIDC provider without `email_verified` the test finishes through the confirm-email step, 22.4), check the member list, the `external_identities` row (`cognito:<provider>` , the Cognito `sub` , `sso_connection_id` set) and the `identity.linked` and `invitation.accepted` audit rows, activate. Also confirm there what the fake cannot show: that `SupportedIdentityProviders` keeps every other client setting, that `DescribeIdentityProvider` for a metadata-URL provider returns only the `MetadataURL`

(the API fetches the metadata itself either way, 22.11), the attribute mapping of the employer's IdP, Cognito's error pages for a refused assertion, and whether the ID token's `auth_time` follows the employer IdP's own authentication or Cognito's federation (section 9: the condition for reconsidering an IdP round trip at step-up).

5. If employees will sign in on mobile, test one federated sign-in in the app: `bind-token` binds it (section 8), and with an OIDC provider that omits `email_verified` the person confirms the address once on the web first.

22.14 Follow-ups

Phase 4 closed every follow-up this section listed (section 21 names the task that closed each): the per-IP throttle on start and callback, the binding connection on the external identity, the certificate expiry of metadata-URL providers (the API now fetches the metadata itself: `https:` only, every resolved address public unicast, 5 seconds, 1 MiB, no redirects; `http:` refused by the request schema with 400 and a stored one by the service with 422 `scheme`; a host that does not exist 422 `host`, a non-public one 422 `private_address`, a resolver that does not answer 503 `unavailable`; the nightly sweep covers `ACTIVE` connections, 22.11), provider changes on a `SUSPENDED` connection, the binding after the email code, federated users on mobile, federated step-up, the address in the unverified redirect, audit rows outside the binding transaction, the alert email, the custom Cognito domain in CDK and the admin screen's gaps. The old note that the admin test link's fallback spent a second nonce was wrong: it reused the first one.

What is left for SSO needs the owner or the pilot and is listed, with its reason, in section 21:

- **pilot**: the live IdP test (22.13), including the `auth_time` question; the claim switch for the pilot's OIDC provider; a sweep of unbound pool users left by abandoned pending bindings (22.15 for the manual clean-up); the throttle budgets against real traffic.
- **owner decision**: IdP-initiated sign-in stays off (with the replay cache and a risk acceptance); SCIM provisioning.
- **AWS**: deploying the custom domain and the CloudFront-only load balancer, before the first employer configures its IdP (section 17.2).

22.15 Operating notes

- **Re-binding identities after migration `0008`.** A federated identity linked after `0008` ran but by code that did not yet record the binding connection (none is expected outside a developer database) has `sso_connection_id` null, and path 1 refuses it (`auth.sign_in.failed`, `sso_provider_mismatch`). Run the migration's backfill again, limited to those rows:

  ```
  UPDATE external_identities AS ei SET sso_connection_id = c.id FROM sso_connections AS c WHERE ei.provider = 'cognito:' ||
  c.cognito_provider_name AND ei.sso_connection_id IS NULL;
  ```

- **Unbound pool users.** A pending binding that is abandoned or expires (15 minutes) leaves a federated pool user `<provider name>_<subject>` in Cognito with no `external_identities` row. It can do nothing, and the next sign-in with the same subject reuses it. Until a sweep exists, list the pool users of a provider by that username prefix and delete those whose `sub` has no `external_identities` row under `cognito:<provider name>`; never delete one that has a row, because its identity would stop matching (Cognito would create a new user with a new `sub`).
- **A reload of the confirm-email page.** Within a minute of the first code, a reload shows the countdown instead of mailing a new code, and the code from the first mail no longer works, because its challenge token lived only in the page. Users wait for the countdown and ask for a new code; support can say so.
- **Turning the claim switch off.** Only for an OIDC provider that never emits `email_verified`, and only after the client confirms in writing that its `email` claim is the directory's verified address (22.4). The `sso_connection.updated` row records who changed it and its new value; `identity.verification_started` rows with `emailVerifiedClaim: null` show that a provider sends no claim at all.
- **Refused federated bearers.** A mobile pool user whose sign-in was never bound (or whose membership ended) is refused on every request, but `auth.sign_in.failed` is written only once per connection and pool user in 10 minutes (`bearer_mismatch_audited:*`, section 8); count the requests in the access log, not in the audit table.

23. Email

Phase 4 (Task 2) gave the API one mailer for everything it sends itself: invitations, the codes NestSelf issues (section 6) and the operations alert of the certificate job. Cognito still sends its own mails (the native sign-in and password codes, through the CustomMessage trigger). The module is global, like the audit and Redis modules, so the worker process, which boots the same `AppModule`, has the same mailer.

The port (`apps/api/src/email/mailer.ts`). `Mailer` is an abstract class, so it doubles as the NestJS injection token, with one method: `send({ to, locale, template, params })`. `template` is one of `invitation`, `step_up_code`, `sso_bind_code` and `ops_certificate_alert`; `params` fill the template's placeholders. `send` rejects when the mail could not be handed over: with a `MailDeliveryError` when the provider refused it, or with the render error when the message itself is wrong. Callers decide whether a mail that did not go out is a log line (an invitation: the invitation stands and can be revoked and sent again) or a failed request (a code: 503 `unavailable`, because the user is waiting for it). `MailDeliveryError` keeps only the provider's error name (when it is a plain identifier) and the HTTP status, with a fixed message: the SDK's own message can quote the recipient's address (SES `MessageRejected` does), so `String(error)` is safe to log.

The rule: a mail goes out after the transaction that justifies it has committed, never inside it. A rolled-back change must not leave a mail behind, so callers collect inside `withOrgScope` and send after it has returned. The invitation import is the pattern (`InvitationsService.import`: the created rows are collected in the transaction and mailed after it); the single invitation mails after its commit; the step-up and bind codes are not tied to a transaction and are mailed before their challenge is stored (section 6); the certificate alert is mailed only once its audit row has committed (section 14). `CLAUDE.md` carries the rule.

The templates (`EMAIL_TEMPLATES` in `src/email/email-templates.ts`), each with a subject and a plain text in the four locales:

Template	Params	Sent by
`invitation`	`link`, `organizationName`, `inviterDisplayName`	`InvitationMailer` (`src/invitations/invitation-mailer.ts`), with the link `{WORKSPACE_URL}/invitations/{token}`; always in `sr-Latn` for now, because an invitation carries no locale
`step_up_code`	`code`, `minutes`	`NestselfOtpService` for the step-up of a federated user, in the principal's locale
`sso_bind_code`	`code`, `minutes`	`NestselfOtpService` for the confirm-email step, in the locale the sign-in page sent

Template	Params	Sent by
`ops_certificate_alert`	`organizationName`, `providerName`, `expiresAt`, `daysLeft`, `adminUrl`	The nightly `sso.certificate-expiry` job, to `OPS_ALERT_EMAIL`

The tone follows the Cognito CustomMessage trigger (`infra/cdk/lib/triggers/custom-message.ts`): the code, how long it lasts, "if you did not request this, ignore this email"; no promises and no marketing. `minutes` is what the code has left, at least one. **Locales**: `sr-Cyrl` is the `sr-Latn` text run through the catalogs' transliteration (`packages/i18n/scripts/transliterate-core.mjs`), never edited by hand, and `email-templates.test.ts` checks that the two agree; the Latin copy is a draft pending a native speaker's review, like the catalogs. **The ops alert is English in every locale slot**, because the operations mailbox reads English, and the job sends it with `locale: 'en'`. An unknown locale falls back to `sr-Latn`.

Rendering (`renderMail`, `src/email/render-mail.ts`). `{{name}}` placeholders are filled from `params`; a placeholder without a param throws, naming the template and the placeholder and never a value, so a misnamed param fails in development and in tests instead of leaving a hole in a mail. The HTML is minimal: one `<p>` per line of the text, every value escaped, and a value that starts with `https://` written as a link. The text is split into lines before the values go in, so a value cannot add paragraphs, and the subject is collapsed to one line, so a line break in a typed organization name cannot reach a header.

The adapters, chosen by `createMailer` (`src/email/email.module.ts`):

- **`SesMailer`** (`src/email/ses-mailer.ts`) when `SES_FROM_EMAIL` is set and `NODE_ENV` is not `test`: `@aws-sdk/client-ses` `SendEmailCommand` with the text and HTML bodies, from `SES_FROM_EMAIL`, in the region `COGNITO_REGION` (else `eu-central-1`, data residency). It logs one line per mail with the template, the locale and the recipient's domain. The API and worker task roles hold `ses:SendEmail` on the account's identities (`infra/cdk/lib/constructs/ecs-task-roles.ts`). The client has no explicit request timeout and uses the SDK's defaults (section 21).
- **`LoggingMailer`** (`src/email/logging-mailer.ts`) otherwise (local development, tests, a stage without a sender): it sends nothing. It renders every message anyway, so a broken template fails here too, and logs at debug level the template, the locale and the recipient's domain only: never the address, and never the params, which hold the codes and the activation links. Only under `NODE_ENV=test` and in stub auth mode does it keep the messages in `sent`, so unit tests, the API e2e suites and the Playwright journeys can read the code or link that "arrived"; anywhere else a code held in memory would be a secret in memory.

Masked addresses (`maskEmail`, `src/email/mask-email.ts`): the first character of the local part, three stars and the domain (`i***@acme.example`; a one-character local part becomes `*@...`), for the step-up and confirm-email answers that tell the user where the code went. `emailDomain` gives the domain for log lines.

Environment. `SES_FROM_EMAIL` (the verified sender; set per stage by the compute stack, `no-reply@staging.nestself.rs` and `no-reply@nestself.rs`) and `OPS_ALERT_EMAIL` (the operations mailbox; from `-c opsAlertEmail` in AWS, unset meaning the certificate job audits and logs only). Both are optional e-mail addresses in `apps/api/src/config/env.ts` (an empty value counts as unset); a malformed one stops the API at start. Locally, with neither set, nothing is sent: an invitation's link can be built from `devToken` (section 11), and the codes NestSelf issues are fixed by `FAKE_IDP_FIXED_OTP` in stub mode (section 2).

Tests. The module is covered by `email-templates.test.ts` (every template in every locale, the Cyrillic agreement), `render-mail.test.ts`, `logging-mailer.test.ts` (no address, code or link in the log line; `sent` only where allowed), `ses-mailer.test.ts` (the command and the address-free `MailDeliveryError`), `email.module.test.ts` (the adapter choice), `mask-email.test.ts` and `invitation-mailer.test.ts`; `test/invitations.e2e.spec.ts` checks that an import mails only its created rows.